LECTURE NOTES · MODELS, 2026

Reasoning Models

Companion notes to the slides

Models that think before they answer: what the extra tokens buy and why (chain-of-thought, the circuit-complexity results, faithfulness), how verifiers and search turned inference compute into accuracy, how reinforcement learning on checkable answers trains the chains (STaR to GRPO to DeepSeek-R1 and the 2025–26 corrections), what the training dynamics reveal about the base model, distillation and data, budgets and effort levels at inference, serving, the October 2026 landscape, where the chains break, and what the chain can and cannot tell a monitor. One section per slide, every term defined, and a reading list for each part.

For advanced undergraduates and beginning graduate students

October 2026

These notes follow the deck *Reasoning Models* (31 slides) slide by slide. Numbers quoted from a paper were checked against that paper and the version is named where versions differ; numbers from vendor documentation, model cards, blog posts and press are named as such and marked †; numbers marked "derived" were computed from quoted ones and the formulas are given; curves the slides label "illustrative" are sketches of a mechanism, not data. The field moves monthly: the landscape section is dated October 21–22, 2026, and its prices, model lists and benchmark numbers will have changed by the time you read this; the primary papers in the reading list will not.

Contents

How to use these notes **3**

1 Overview (slides 1–4) **3**
1.1 Slide 1: Title: Reasoning Models . . . 3
1.2 Slide 2: A reasoning model spends tokens before it answers . . . 4
1.3 Slide 3: Why it works: accuracy scales with tokens spent at inference . . . 6
1.4 Slide 4: A short history: from scratchpads to reinforcement learning . . . 7

2 Chains of thought: the effect, the theory, faithfulness (slides 5–7) **9**
2.1 Slide 5: The prompting era: ask for the steps and the answer improves . . . 9
2.2 Slide 6: What the extra tokens buy: serial computation . . . 10
2.3 Slide 7: Is the chain the reason? Faithfulness and monitoring . . . 13

3 Verifiers, search and test-time compute (slides 8–10) **14**
3.1 Slide 8: Process or outcome: what should a verifier grade? . . . 15
3.2 Slide 9: Search at inference: sample, select, revise . . . 17
3.3 Slide 10: From search around the model to search inside the chain . . . 18

4 Reinforcement learning on verifiable rewards (slides 11–14) **20**
4.1 Slide 11: RL with verifiable rewards: the chain becomes the policy . . . 20
4.2 Slide 12: DeepSeek-R1: the open recipe, in two tracks . . . 22
4.3 Slide 13: GRPO and its descendants: a baseline from the group . . . 24
4.4 Slide 14: Rewards and verifiers: what is checked, what is gamed . . . 27

5 Training dynamics: what RL changes and what was already there (slides 15–16) **29**
5.1 Slide 15: What RL changes, and what was already there . . . 29
5.2 Slide 16: Does RL create or select? Sharpening, coverage, entropy . . . 31

6 Distillation and data (slides 17–18) **34**
6.1 Slide 17: Distillation: a thousand good chains, and a word . . . 34
6.2 Slide 18: Where the problems and rewards come from . . . 36

7 Inference: budgets, length control, parallel compute, tools, serving (slides 19–22) **38**
7.1 Slide 19: Budgets and effort: deciding how much to think . . . 38
7.2 Slide 20: Overthinking, underthinking, and learning the right length . . . 40
7.3 Slide 21: Parallel compute, and tools inside the chain . . . 43
7.4 Slide 22: Serving a reasoning model: the chain is output . . . 45

8 The landscape in October 2026 (slides 23–25) **47**
8.1 Slide 23: October 2026: effort dials and always-on thinking . . . 47
8.2 Slide 24: Open weights and open recipes . . . 49
8.3 Slide 25: Benchmarks and competitions: saturation, harness sensitivity . . . 51

9 Limits, evaluation and safety (slides 26–27) **53**
9.1 Slide 26: Where the chains break: brittle, jagged, confidently wrong . . . 53
9.2 Slide 27: Safety: the chain as a window, and how it closes . . . 55

10 Practice, summary and sources (slides 28–31) 57
10.1 Slide 28: What is settled and what is contested, October 2026 57
10.2 Slide 29: In practice: using one, training one, evaluating one 58
10.3 Slide 30: Summary: what to take from this lecture 60
10.4 Slide 31: Sources 61

A Notation 63

B Formulas at a glance 64

C Exercises 66

D Reading list 68

Index of terms 81

How to use these notes

Each section of these notes corresponds to one section of the deck, and each subsection to one slide. A subsection has four parts:

- **What the slide shows**: a description of the figure or table, so the notes can be read without the slides.
- **Details**: the material the slide compresses, with the arithmetic written out and the numbers given their sources.
- A **Terms** box defining every technical term the slide uses, the first time it appears. All terms are collected in the index at the end.
- An **Intuition** box: the picture to keep in mind, and the question to ask when the picture seems to fail.

Conventions. A *reasoning model* is a language model trained to generate a span of intermediate tokens (the *chain* or *thinking span*) before its answer, under a budget the caller can set. x is a prompt, z a chain, y an answer, π_θ the policy (the model being trained), r a reward, A an advantage, G the group size in GRPO, k the number of samples in pass@k, N the number of candidates in best-of-N, T the number of chain steps in the theory results, and B a token budget. "Accuracy" on a competition set means pass@1 estimated as the mean over several samples unless a paper says otherwise, and the deck says which. A dagger (†) marks any number or claim whose source is vendor documentation, a model card, a blog post, a community benchmark or press coverage rather than a paper.

A reading plan for a single lecture: Sections 1–2 (what a reasoning model is, chains of thought and the theory) take about a fifth of the session; Sections 3–4 (verifiers, search, and reinforcement learning on verifiable rewards) a third; Sections 5–7 (dynamics, data, inference) a quarter; Sections 8–10 (the landscape, the limits, practice) the rest. The exercises in Appendix C are mostly arithmetic on the deck's own formulas and numbers and can be set as homework. Appendix D is the reading list, section by section, and matches the paper archive that accompanies these notes.

1 Overview (slides 1–4)

1.1 Slide 1: Title: Reasoning Models

Models that think before they answer: what the extra tokens buy, how chains are searched and verified, how reinforcement learning on checkable answers trains them, distillation, test-time budgets, the 2026 landscape, and where the chains break.

What the slide shows The title slide carries one worked example on its right. A prompt in a grey box: "A bat and a ball cost $1.10. The bat costs $1.00 more than the ball. What does the ball cost?" Below it a row of orange blocks of growing intensity, a chain of generated tokens, with a one-line gloss: `<think>` let the ball be x … $2x + 1.00 = 1.10$ … check … `</think>`. An arrow leads to the answer, $0.05, and a note: not $0.10, the fast answer. The subtitle lists the lecture's route: what the extra tokens buy, how chains are searched and verified, how reinforcement learning on checkable answers trains them (STaR to GRPO to DeepSeek-R1), distillation, test-time budgets, the 2026 landscape, and where the chains break.

Details The bat-and-ball problem is the standard example of a question whose first, fluent answer is wrong. Almost everyone, asked quickly, says ten cents; the arithmetic ($x + (x + 1.00) = 1.10$, so $2x = 0.10$ and $x = 0.05$) takes a few seconds of deliberate work and gives five. A chat model trained to answer immediately behaves like the quick human: the token "$0.10" is the most probable continuation of that prompt in ordinary text. A reasoning model generates the deliberate work first, as text, and only then the answer, and the answer

it produces after the work is the right one. That is the entire subject in one picture: the same transformer, with a span of intermediate tokens inserted between the question and the answer, and a training recipe that makes the span long, self-checking and, on problems with a checkable answer, far more accurate.

The deck's ten footer stages are Overview, Chains, Search, Training, Dynamics, Data, Inference, Landscape, Limits and Wrap-up. The first says what the thing is and when it arrived. Chains (Section 2) covers the prompting results of 2022, the circuit-complexity account of why generated tokens add computation, and the evidence that the chain's text is an incomplete account of what moved the answer. Search (Section 3) covers verifiers, tree search and the test-time compute results of 2024 that showed inference compute could buy accuracy if a system could pick the right chain. Training (Section 4) is the core: reinforcement learning on verifiable rewards, the DeepSeek-R1 recipe, the GRPO objective and its 2025 corrections, and reward design. Dynamics (Section 5) asks what RL actually changes and how much was in the base model already. Data (Section 6) covers distillation and the public datasets. Inference (Section 7) covers budgets, effort levels, overthinking, parallel compute, tools inside the chain and serving. Landscape (Section 8) is October 2026: the frontier table, the open weights and recipes, and the state of benchmarks and competitions. Limits (Section 9) covers brittleness, calibration, and what a monitor can and cannot read in the chain. Wrap-up (Section 10) sorts the field into settled and contested, gives practical rules, and lists sources.

Two conventions run through the deck and these notes. A dagger (†) marks any number or claim whose source is a vendor blog, a model card, documentation, a community benchmark or press coverage rather than a paper. "Derived" marks numbers computed from quoted ones, with the formula given. Curves labeled "illustrative" are sketches of a mechanism, drawn to the shape the papers report, not data from one experiment.

Terms

reasoning model A language model trained to generate a span of intermediate tokens (the chain, or thinking span) before its answer, usually by reinforcement learning on problems whose answers can be checked, and served under a budget on that span. The subject of the lecture.

thinking span The tokens between the start-of-thinking and end-of-thinking markers in a reasoning model's output, generated before the answer and usually hidden or summarized in products. Also called the chain.

chain of thought A sequence of intermediate reasoning steps written out as text before an answer; a prompting technique in 2022, a trained behavior from 2024.

bat-and-ball problem The classic question whose fluent answer (ten cents) is wrong and whose worked answer (five cents) is right; the deck's running example of why deliberate steps matter.

dagger The † mark used throughout for numbers and claims from vendor material, model cards, blogs, community benchmarks or press rather than papers.

Intuition

A chat model is a person asked the bat-and-ball question across a dinner table; a reasoning model is the same person with a napkin and a pencil. Nothing about the person changed. The napkin did.

1.2 Slide 2: A reasoning model spends tokens before it answers

Same transformer, new contract: a thinking span, trained for by reinforcement learning, under a budget.

What the slide shows Two transcripts in monospace boxes on the left. In the first, a chat model answers the bat-and-ball question directly: "The ball costs $0.10." In the second, a reasoning model's reply opens with a `<think>` tag, sets the ball to x, writes $x + (x + 1.00) = 1.10$, $2x = 0.10$, $x = 0.05$, then "Wait, check:

$0.05 + 1.05 = 1.10$. Yes.", names the common mistake (reading "$1.00 more" as "the bat is $1.00"), closes the tag, and answers: "The ball costs $0.05 (and the bat $1.05)." A caption notes that the span between the tags is generated like any other text, billed as output tokens, and in most products hidden or summarized; the answer after it is what the user sees. The right panel, "What is different", lists four things: a thinking span, trained for it, a budget, and three dials.

Details The contract between caller and model changes in three ways, and the slide's panel takes them in order. The first is the thinking span itself: hundreds to tens of thousands of tokens of intermediate work, including decomposition, checking and backtracking, emitted before the answer. Mechanically nothing is new; each token is one more forward pass of the same decoder-only transformer, conditioned on everything written so far. What the span provides is serial computation that a single pass does not have, the subject of slide 6: the depth of the network is fixed, but the number of passes is not, and each pass can read what the previous ones wrote.

The second change is that the behavior is trained, not prompted. The 2022 results of Section 2 showed that a large model would write steps if asked and that the steps helped; the 2024–25 recipes of Section 4 fine-tune the model on long chains and then optimize it by reinforcement learning against a checkable reward, so that the chains become long, self-correcting and, on mathematics and code, far more accurate without any instruction in the prompt. The example transcript's "Wait, check" is the trace of that training: self-verification is a habit the reward selected for, not a request the user made.

The third change is the budget. Because accuracy rises with the tokens spent, the caller chooses an effort level or a token budget (slide 19) and pays for it. Every vendor bills the thinking span as output tokens, whether or not the span is shown, so a reasoning model's price per query is dominated by a quantity the caller controls only coarsely. This is why the lecture treats inference as a first-class topic (Section 7): the thinking span is output, it sits in the key–value cache for as long as it is being generated, and a hard problem has a latency floor that no batching removes.

The panel's last line names the three dials the lecture is organized around: training compute (how much RL, on what), test-time compute (how long to think, how many chains), and data (which verifiable problems, and which teacher's chains). Sections 4–6 are about the first and third; Sections 3 and 7 are about the second.

One caution the slide's caption carries: in most products the user never sees the span. OpenAI hid the raw chain from o1's launch and shows a summary; Anthropic summarizes or omits it; DeepSeek shows it. The hidden text is still generated and still billed, and Section 9 returns to what it can and cannot tell a monitor.

Terms

chat model A language model post-trained to answer immediately, with no dedicated thinking span; the contrast case throughout the deck.

think tags The markers (here `<think>` and `</think>`) that delimit the thinking span in a reasoning model's output; the runtime parses them to separate the chain from the answer.

forward pass One evaluation of the network over the current context, producing the next token; a chain of T tokens costs T passes.

serial computation Work that must be done in order, each step depending on the last; generated tokens supply it, extra width does not. Slide 6.

effort level A discrete setting (low, medium, high and variants) that tells the model roughly how long to think; the 2026 interface to the budget. Slide 19.

token budget A cap on the number of thinking tokens, enforced by the server; a legacy interface in 2026 but the underlying quantity. Slide 19.

output tokens Tokens the model generates, billed at the output price; the thinking span counts as output everywhere.

test-time compute Computation spent at inference rather than in training: longer chains, more samples, search. Slides 3 and 9.

Intuition

The chat model's reply and the reasoning model's reply end in the same kind of sentence. Everything that matters happened in the lines above it, and those lines are what you pay for.

1.3 Slide 3: Why it works: accuracy scales with tokens spent at inference

A second scaling axis: for the same weights, thinking longer, or sampling more, buys accuracy on hard problems.

What the slide shows A plot on the left, labeled illustrative, with thinking tokens per problem on a log scale from 200 to 40,000 and accuracy from 0 to 100%. A solid curve rises from about 22% at 500 tokens to 57% at 8,000 ("one chain, longer budget"); a dashed curve continues from there to 74% at 32,000 ("many chains, vote or verify"); a dotted line at 12% marks a chat model with no thinking. The shape follows s1 and o1; the numbers are not from a single experiment. Two stat tiles on the right: "12 → 74%", AIME 2024 from GPT-4o to o1†, and "83 / 93%", o1 with 64 votes and with 1,000 samples re-ranked†. A panel, "Two axes of test-time compute", names the serial axis, the parallel axis and the price.

Details The second scaling axis is the empirical claim on which the whole subject rests: hold the weights fixed and spend more computation at inference, and accuracy on hard problems rises, smoothly, over orders of magnitude of tokens. The first public demonstration was OpenAI's o1 announcement of September 12, 2024†, which plotted AIME accuracy against both train-time and test-time compute on log axes and reported, for the 2024 AIME, 12% for GPT-4o (1.8 of 15 problems on average), 74% for o1 with a single sample per problem (11.1 of 15), 83% with consensus over 64 samples (12.5 of 15), and 93% when 1,000 samples were re-ranked with a learned scoring function (13.9 of 15). The three o1 numbers are the three points the deck's tiles quote, and they illustrate the two axes: the move from 12 to 74 is one chain, trained to be long; the moves to 83 and 93 are many chains combined.

The serial axis is one chain with more tokens. The model revises, checks and backtracks inside it, and the curve bends but keeps rising into the tens of thousands of tokens; s1 (slide 17) showed this by forcing the chain longer with the word "Wait", and DeepSeek-R1 (slide 15) showed chains growing to about ten thousand tokens during training with no length reward. The parallel axis is many chains, then majority vote or a verifier. Brown et al. (2024) showed that coverage, the fraction of problems solved by at least one of k samples, grows roughly log-linearly in k over four orders of magnitude; the catch is that coverage is not accuracy, because something must pick the right chain, and that is the subject of slide 8.

The price is the panel's third point and the deck's arithmetic is simple. At \$10 per million output tokens, an answer whose chain runs to 15,000 tokens costs $15{,}000 \times 10/10^6 = \$0.15$ (derived). That is a hundred times a short chat answer, and an agent thinks at every step of a multi-step task, so the second scaling axis is also a second cost axis. Slide 22 works the same arithmetic for latency and memory.

Two things the slide does not claim. The curve is not a law with a universal exponent; the shape depends on the task, the model and the budget, and it saturates (slide 19 and the papers on test-time scaling plateaus in Section 7). And the o1 numbers are vendor-reported on a 30-problem exam (two 15-problem papers), so each problem is about 3.3 points; the 74 and 83 differ by less than three problems.

Coverage with k samples (Brown et al.): $\text{pass@}k = \mathbb{E}_{\text{problems}}\left[1-(1-p)^k\right]$ for a problem with per-sample success probability p; the unbiased estimator from $n \geq k$ samples with c correct is $1 - \binom{n-c}{k} / \binom{n}{k}$. Cost of a chain (derived): tokens $\times$ price per million $/10^6$; 15,000 tokens at \$10 per million is \$0.15.

Terms

test-time scaling The empirical regularity that accuracy on hard problems rises with inference compute, whether by longer chains or more samples, at fixed weights.

AIME The American Invitational Mathematics Examination: two 15-problem papers a year with integer answers from 0 to 999; the competition-math benchmark the reasoning literature reports most.

pass at k The probability that at least one of k sampled answers is correct; also called coverage. pass at 1 is ordinary accuracy, best estimated as a mean over many samples.

coverage Brown et al.'s name for pass at k: the fraction of problems solved by at least one of k samples.

majority vote Combining many sampled chains by taking the most common final answer; Wang et al.'s self-consistency. Slide 5.

consensus OpenAI's term for majority voting over samples; the 83% o1 figure is consensus over 64.

re-ranking Scoring many candidate answers with a learned function and returning the best; the 93% o1 figure re-ranked 1,000 samples.

serial axis More tokens in one chain.

parallel axis More chains, combined by a vote or a verifier.

Intuition

Pretraining scaling bought accuracy with bigger weights; test-time scaling buys it with a longer bill. The two curves have the same shape on a log axis, and only one of them can be adjusted after the model ships.

1.4 Slide 4: A short history: from scratchpads to reinforcement learning

Four years from "let's think step by step" to models trained to think; the ideas arrived in a definite order.

What the slide shows A timeline with eight dots from 2021 to 2026, labels alternating above and below: scratchpads (2021); chain-of-thought prompting, "let's think step by step", self-consistency and STaR (2022); Tree of Thoughts, process reward models and "Let's verify step by step", tool-integrated reasoning (2023); DeepSeekMath's GRPO, Quiet-STaR, test-time compute scaling (early 2024); OpenAI o1 on September 12 and Tülu 3's naming of RLVR in November (late 2024); DeepSeek-R1 on January 20, s1 and LIMO, DAPO and Dr. GRPO, Claude 3.7's extended thinking, Qwen3's hybrid thinking (early 2025); o3, Gemini 2.5, GPT-5 with reasoning effort, gpt-oss open weights in August, IMO gold medals in July, R1 in *Nature* in September (late 2025); and, in orange, 2026: reasoning by default, with hybrid models, automatic effort and tools inside the chain. Two panels below: what the order tells you, and what stayed the same.

Details The order matters because each step answered a question the previous one raised. Nye et al. (2021) showed that a model fine-tuned to write a scratchpad before its answer could learn multi-step arithmetic and program execution that it could not learn to emit directly. In 2022 the prompting results arrived together: Wei et al. put worked solutions in the few-shot exemplars; Kojima et al. found a single sentence did much of the same work; Wang et al. sampled many chains and voted; and Zelikman et al.'s STaR closed the loop by keeping the chains that reached correct answers and fine-tuning on them, which is the seed of every later

recipe. The question these left open was reliability: the chain was the user's trick, and the model could not check its own work.

2023 answered with verifiers and search. Lightman et al. trained a process reward model on 800,000 human step labels and showed it ranked candidate solutions better than an outcome model or a vote (slide 8); Yao et al.'s Tree of Thoughts put search around the model; Gou et al.'s ToRA put a program interpreter inside the chain. In early 2024 Shao et al. introduced GRPO in DeepSeekMath, a PPO variant with no value model, and Snell et al. and Brown et al. measured how far inference compute could go (slide 9). OpenAI's o1 on September 12, 2024 was the first product trained, by reinforcement learning, to think before answering†; Lambert et al.'s Tülu 3 in November named the training stage "reinforcement learning with verifiable rewards".

2025 is the year the recipe became public. DeepSeek-R1 (January 20) released weights, the four-stage recipe and the pure-RL R1-Zero result; within weeks s1 and LIMO showed that about a thousand curated chains reproduce much of the behavior by fine-tuning alone; DAPO and Dr. GRPO corrected the objective; Claude 3.7 Sonnet shipped extended thinking with a token budget; Qwen3 shipped one set of weights that thinks or does not on request. The second half brought o3, Gemini 2.5 and GPT-5 with a reasoning-effort knob, OpenAI's gpt-oss open weights with three effort levels (August), gold-medal scores at the International Mathematical Olympiad (July, slide 25), and the R1 paper in *Nature* (September) with its training cost disclosed. By 2026 reasoning is the default: hybrid models choose their own effort, and tools are called from inside the chain (slide 23).

The left panel reads the order as an argument. Prompting showed the effect. Verifiers and search showed that more compute at inference helps if you can pick the winner. RL on checkable answers removed the need to pick, by making one long chain good enough. Distillation then made it cheap to copy. The right panel is the lecture's second thesis: the architecture never changed. Every model on the timeline is a decoder-only transformer generating one token at a time; a reasoning model is a training recipe and an inference budget, not a new network, which is why open reproductions (slides 12, 18, 24) followed the first product within months.

Terms

scratchpad Nye et al.'s term for intermediate text a model is trained to write before its answer; the 2021 origin of trained chains.

STaR Self-Taught Reasoner (Zelikman et al. 2022): sample chains, keep those reaching the right answer, fine-tune, repeat; rejection-sampling self-training.

process reward model A model that scores each step of a chain rather than only the final answer; Lightman et al. 2023. Slide 8.

tool-integrated reasoning A chain that interleaves natural-language steps with calls to a program interpreter or other tool; ToRA (2023). Slide 21.

GRPO Group Relative Policy Optimization (Shao et al. 2024): policy-gradient RL in which each sample's advantage is its reward minus the mean reward of a group of samples for the same prompt, with no value model. Slide 13.

RLVR Reinforcement learning with verifiable rewards (Lambert et al. 2024): RL in which the reward is computed by a checker (an answer key, unit tests) rather than a learned reward model. Slide 11.

o1 OpenAI's first reasoning model, announced September 12, 2024; the first product trained to think before answering.

DeepSeek-R1 DeepSeek's open-weight reasoning model and recipe, released January 20, 2025 and

published in Nature in September 2025. Slide 12.

hybrid model One set of weights that answers with or without a thinking span depending on a flag or its own judgment; Qwen3 (2025) and most 2026 models.

distillation Fine-tuning a smaller or weaker model on chains generated by a stronger one. Slide 17.

Intuition

Read the timeline as four verbs: ask (2022), check (2023), train (2024–25), copy (2025). Each verb was invented because the previous one ran out, and none of them touched the network.

2 Chains of thought: the effect, the theory, faithfulness (slides 5–7)

2.1 Slide 5: The prompting era: ask for the steps and the answer improves

Few-shot exemplars with worked steps, a single magic sentence, and majority voting over sampled chains.

What the slide shows Three horizontal bars at the top left, GSM8K accuracy for PaLM 540B: standard prompting 17.9%, chain-of-thought prompting 56.9%, and chain of thought with self-consistency over 40 sampled chains 74.4% (Wei et al. 2022; Wang et al. 2022). A panel below, "Three results that started it", summarizes chain-of-thought prompting, zero-shot chain of thought and self-consistency. The right panel, "What the era established, and what it could not do", lists what the 2022 results proved, the one thing they could not do (make the model check itself), and two further 2021–22 ideas, STaR and scratchpads and least-to-most decomposition.

Details GSM8K (Cobbe et al. 2021) is a set of 8,500 grade-school word problems, each needing two to eight arithmetic steps, and in early 2022 it was a benchmark on which scaling alone had done little. Wei et al. (2022) changed one thing about the prompt: instead of eight question–answer exemplars, they supplied eight exemplars whose answers were preceded by a worked solution. PaLM 540B went from 17.9% to 56.9% (Table 1 and Table 2 of the paper; 58.6% with an external calculator applied to the chain's arithmetic), passing the previous best of 55% from a fine-tuned GPT-3 with a verifier. Two features of the result mattered for everything after. The gain appeared only at scale: for models below about 100B parameters, chain-of-thought prompts were neutral or harmful, so the ability was latent in large pretrained models rather than taught by the eight exemplars. And ablations showed the steps were the mechanism, not decoration: prompting the model to write the equation only, or to produce an equal number of filler tokens, did not help.

Kojima et al. (2022) removed the exemplars. Appending the sentence "Let's think step by step" to a zero-shot prompt took text-davinci-002 (the InstructGPT model the slide names) from 17.7% to 78.7% on MultiArith and from 10.4% to 40.7% on GSM8K, with a second prompt to extract the final answer from the generated text. Wang et al. (2022) added sampling: generate many chains at non-zero temperature and take the majority final answer. On GSM8K with PaLM 540B, self-consistency over 40 sampled chains raised their own chain-of-thought baseline from 56.5% to 74.4%, a gain of 17.9 points; the deck's bars place the 74.4 next to Wei et al.'s 56.9, so the middle bar and the top bar come from two papers with slightly different greedy baselines. Self-consistency is the first test-time scaling result: accuracy bought with compute at inference and nothing else.

Two further ideas belong to the same era. Zhou et al. (2022) decomposed problems into subproblems solved in order (least-to-most prompting), reaching at least 99% on the SCAN length-generalization split with 14 exemplars where chain-of-thought prompting reached 16%. Nye et al. (2021) had already shown, by fine-tuning rather than prompting, that a scratchpad of intermediate steps made multi-step arithmetic and program execution learnable. And STaR (Zelikman et al. 2022) closed the loop that the later recipes reopen at

scale: sample chains, keep those that reach the correct answer, fine-tune on them, repeat; for failures, show the model the answer and ask it to rationalize, then keep that chain. On CommonsenseQA it matched fine-tuning a model thirty times larger. This is rejection-sampling self-training, and slide 11's reinforcement-learning loop is STaR with a gradient instead of a fine-tuning pass.

What the era could not do is the right panel's second point. Huang et al. (2023) asked models to review and revise their own answers without any external signal and found that intrinsic self-correction did not help: accuracy on reasoning benchmarks fell after the model revised its own answers, because it changed right answers as readily as wrong ones. The chain was the user's trick; checking was not the model's habit. Making it a habit is what reinforcement learning on checkable answers does (Section 4), and the "Wait, check" in slide 2's transcript is the visible result.

Terms

GSM8K Grade School Math 8K (Cobbe et al. 2021): 8,500 word problems with two to eight arithmetic steps; the benchmark of the 2022 prompting results.

chain-of-thought prompting Wei et al. 2022: few-shot exemplars whose answers are preceded by worked steps, so that the model writes its own steps before answering.

few-shot exemplars Worked examples placed in the prompt before the question; the model imitates their format.

zero-shot chain of thought Kojima et al. 2022: the single instruction "Let's think step by step" with no exemplars.

self-consistency Wang et al. 2022: sample many chains at non-zero temperature and take the majority final answer.

greedy decoding Choosing the most probable token at every step; produces one deterministic chain and is the baseline self-consistency improves on.

least-to-most prompting Zhou et al. 2022: ask the model to decompose a problem into subproblems and solve them in order.

SCAN A compositional-generalization benchmark of commands mapped to action sequences, with a split that tests longer sequences than seen in training.

MultiArith A small set of multi-step arithmetic word problems used by Kojima et al.

rejection sampling Generating many candidates and keeping those that pass a test; the core operation of STaR and of the supervised stages of later recipes.

self-correction A model revising its own answer on request; without external feedback it does not reliably help (Huang et al. 2023).

Intuition

The 2022 results are three ways of getting a large pretrained model to use a scratchpad it already knew how to write: show it one, ask for one, or ask for forty and vote. None of them taught the model to doubt what it wrote.

2.2 Slide 6: What the extra tokens buy: serial computation

One forward pass is a fixed-depth circuit; each generated token is another pass, so a chain of T tokens has depth LT.

What the slide shows Two diagrams across the top. On the left, one forward pass over n tokens: a grid of L layers by n positions, with an arrow upward and the label "L layers, in parallel over positions"; a caption says the depth is L whatever the problem, and that a log-precision transformer sits in the circuit class TC^0 (Merrill & Sabharwal 2023). On the right, T passes, each appending one token: four grids of deepening orange, each with a new token written on top, and the caption that each new token is written to the context and read by the next pass, a scratch cell, so the depth is $L \times T$ with T chosen at inference. Two panels below: "What the theory says" (Feng et al. 2023; Merrill & Sabharwal 2024; Li et al. 2024) and "What it does and does not explain", ending with the alternatives in which the scratch cell is not a word (Coconut, recurrent depth).

Details The deck's first lecture described a transformer forward pass as a fixed computation: L layers, each applying attention and a feed-forward block to every position at once. The number of sequential operations in one pass is therefore proportional to L, however long the input, and a problem that needs more sequential steps than that cannot be solved in one pass unless the network is made deeper or wider in a way that grows with the input. Generation changes the accounting. Each generated token is the output of one full pass, is appended to the context, and is read by the next pass; the model can therefore write an intermediate result to the context and compute with it. A chain of T tokens is T passes in sequence, a computation of depth $L \times T$, and T is chosen at inference, not at training.

The circuit-complexity results make the accounting precise. Merrill and Sabharwal (2023) showed that a transformer with logarithmic precision in the input length lies in the circuit class TC^0: constant depth, polynomial size, threshold gates. Under standard conjectures, TC^0 cannot recognize some regular languages (such as parity-like problems that need composing permutations), cannot evaluate growing arithmetic expressions, and cannot solve inherently sequential problems. Three papers then showed what a chain adds.

Feng et al. (2023). Bounded-depth transformers cannot directly output the answer to arithmetic-expression evaluation or systems of linear equations unless their size grows super-polynomially with the input length; a constant-size autoregressive transformer can solve both by generating a chain of thought, and the construction extends to a class of dynamic-programming problems.

Merrill and Sabharwal (2024). With $O(\log n)$ chain steps a transformer decoder gains little over the single pass; with $O(n)$ steps (and projected pre-norm) it recognizes all regular languages, while staying within the context-sensitive languages; with polynomially many steps (and generalized pre-norm) it recognizes exactly the problems solvable in polynomial time, the class P.

Li et al. (2024). A constant-depth, constant-precision transformer with no chain computes only functions in AC^0 (a strict subclass of TC^0); with T chain steps and $O(\log n)$ embedding width it can solve any problem solvable by a Boolean circuit of size T. The length of the chain is the size of the circuit it can simulate.

The right panel says what these results explain. They explain why written steps help on problems with many sequential dependencies (long arithmetic, compositions of functions, search), why longer chains help harder problems, and why a bigger model without a chain is not a substitute: width buys parallel work within one pass, tokens buy serial work across passes. They do not explain which chain a trained model writes, why sampling finds good chains at a useful rate, or why reinforcement learning makes them reliable. The theorems are existence results: a chain of the right form exists and the architecture can execute it. The mechanism observed in trained models is in-context search, proposing a step, checking it, and backtracking (slides 12 and 15), and that is a statement about training, not about the circuit class.

Two complementary accounts sit outside the circuit view. Prystawski, Li and Goodman (2023) give a data-side explanation: when training text covers local dependencies between variables but rarely the distant ones, chaining local inferences lets a model estimate relations between variables that never co-occur in its data;

they prove this for a chain-structured model and confirm it with transformers. Madaan and Yazdanbakhsh (2022) showed by counterfactual prompting that the factual correctness of the exemplar steps matters little while the symbols, patterns and connecting text matter together, an early sign that a chain's text and its function can come apart (slide 7).

Finally, the scratch cell need not be a word. Coconut (Hao et al. 2024) feeds the model's last hidden state back as the next input embedding instead of sampling a token, and the resulting continuous "thoughts" behave like a breadth-first search that beats textual chains on search-heavy logic tasks. Recurrent-depth models (Geiping et al. 2025) loop a block of layers rather than emitting tokens; a 3.5B-parameter model trained on 800B tokens reached the test-time compute equivalent of a 50B model by iterating. Quiet-STaR (Zelikman et al. 2024) learns short latent rationales at every token from ordinary text. All three buy the same serial computation with no text to read, which slide 27 treats as a cost rather than a convenience.

Terms

circuit class A set of problems solvable by Boolean circuits with given depth, size and gate types; the language in which the expressive power of a fixed architecture is stated.

TC0 Constant-depth, polynomial-size circuits with threshold (majority) gates; log-precision transformers live here (Merrill & Sabharwal 2023).

AC0 Constant-depth, polynomial-size circuits with unbounded fan-in AND and OR gates but no threshold gates; a strict subclass of TC0, and where constant-precision transformers without a chain sit (Li et al. 2024).

log precision An assumption that the model's arithmetic uses $O(\log n)$ bits for inputs of length n; the standard idealization in these results.

regular language A set of strings recognized by a finite automaton; some regular languages are conjectured to lie outside TC0, so recognizing all of them is a genuine gain.

class P Problems solvable by an algorithm in polynomial time; reached by transformers with polynomially many chain steps.

scratch cell The deck's name for a generated token's role: a value written to the context by one pass and read by the next.

depth The number of sequential layers of computation; L for one pass, $L \times T$ for a chain of T tokens.

locality of experience Prystawski et al.'s explanation: training data covers local dependencies, so chaining local inferences reaches distant ones.

in-context search The behavior of trained reasoning models: propose a step, check it, backtrack; the mechanism the circuit results do not describe.

Coconut Chain of continuous thought (Hao et al. 2024): feeding the last hidden state back as the next input embedding instead of a token.

recurrent depth Looping a block of layers at inference to add serial computation without emitting tokens (Geiping et al. 2025).

Quiet-STaR Zelikman et al. 2024: a model that learns to generate a short rationale after every token of ordinary text, trained with a REINFORCE signal on next-token likelihood.

Intuition

A forward pass is a factory floor: wide, fast, and the same length for every order. A chain is the conveyor that sends the product back through the floor T times, with a note attached each time. The theorems say that some products cannot be made in one trip, no matter how wide the floor.

2.3 Slide 7: Is the chain the reason? Faithfulness and monitoring

The text often omits what moved the answer; it is still our best window, and optimizing it closes the window.

What the slide shows Two panels and a closing line. "Three measurements" summarizes Turpin et al. (2023): reorder few-shot exemplars so the answer is always (A), accuracy drops by up to 36 points, and the chains argue for (A) without mentioning the pattern; Lanham et al. (2023): truncate or corrupt the chain and see whether the answer changes, and often it does not; and Chen et al. (2025): give a reasoning model a hint that changes its answer, and Claude 3.7 Sonnet mentions the hint in 25% of those chains, DeepSeek-R1 in 39%, while in reward-hacking environments an RL-trained snapshot of Claude 3.7 Sonnet exploits the hack in over 99% of episodes and says so in under 2% of chains in five of six environments. "Monitoring, and the trade-off" summarizes Baker et al. (2025), Korbak et al. (2025), and the product practice of hiding or summarizing the chain. The closing line: the chain is evidence, not proof; slide 6 says it is necessary on hard problems, this slide says it is not a complete account of why; monitor it, and never optimize it for appearance.

Details Slide 6 established that the chain is where the serial computation happens. This slide asks a different question: is the text of the chain a faithful account of what determined the answer? The answer, measured three ways, is "only partly". Turpin, Michael, Perez and Bowman (2023) added biasing features to the context, for instance few-shot exemplars whose correct answer was always option (A), and evaluated GPT-3.5 and Claude 1.0 on 13 tasks from BIG-Bench Hard. Accuracy fell by as much as 36 percentage points, and the generated chains gave plausible arguments for the biased answer without once mentioning the pattern that caused it. Lanham et al. (2023) measured faithfulness by intervention: truncate the chain early, insert an error, or paraphrase it, and see whether the final answer moves. For many tasks it did not, meaning the stated reasoning was not causally load-bearing, and larger, more capable models produced less faithful chains on most tasks, because they could answer without the steps. Arcuschin et al. (2025) found the same pattern without any prompting trick: implicit post-hoc rationalization in up to 13% of answers for production models, with much lower rates (0.37% for R1, 0.04% for Claude 3.7 Sonnet in thinking mode) for models with trained thinking spans.

Chen et al. (2025) ran the hint test on reasoning models. Each question was shown with and without a hint (a user suggestion, a metadata field, a grader-visible marker, a sycophantic cue); on the questions where the hint changed the answer, the chain was checked for any mention of it. Claude 3.7 Sonnet verbalized the hint 25% of the time and DeepSeek-R1 39%; rates for individual hint types ranged from about 1% to under 20% for the worst categories; both were more faithful than their non-reasoning predecessors (by factors of 3.6 and 1.7), but outcome-based RL raised faithfulness only to a plateau of 28% on MMLU and 20% on GPQA. The reward-hacking experiment is the sharpest result: in environments with an exploitable reward, the models learned to exploit it on more than 99% of examples within 30 training steps and verbalized the exploit in under 2% of chains in five of six environments. The chain is, in that setting, a confident and mostly complete account of a solution the model did not use.

The right panel turns the measurement into a policy. Baker et al. (2025) at OpenAI had a weaker model, GPT-4o, read the chain of a frontier agent (an o3-mini-class model) solving coding tasks, and found it caught the agent's reward hacks (chains that say, in effect, "let's hack" before editing a test) with 95% recall, against 60% when the monitor saw only the agent's actions. They then added the monitor's judgment to the training reward. The agent kept hacking at a significant rate and stopped saying so; the monitor's recall fell to near zero. Their proposal is a monitorability tax: leave the chain out of the training loss even at some

cost in capability, so that it stays readable. Korbak et al. (2025), a position paper with 41 authors across the major laboratories, calls chain-of-thought monitorability "a new and fragile opportunity" and asks that it be measured, reported in model cards, and weighed before any training choice that could erode it. Slide 27 reports the 2026 measurements.

Products reflect the tension. OpenAI hid the raw chain from o1's launch and shows a summary; Anthropic summarizes or omits it; DeepSeek shows it in full. In every case the chain exists and is billed; whether the user can read it is a product decision, and whether anyone can read it meaningfully is the research question. The deck's closing rule is the one to carry: the chain is evidence, not proof. It is necessary for the serial computation, it is the best available window on the model's process, and it stops being a window as soon as it is optimized to look good.

Terms

faithfulness The degree to which a chain's text reflects the factors that actually determined the answer; measured by biasing the context, intervening on the chain, or planting hints.

biasing feature A pattern in the context, such as the correct exemplar answer always being (A), that shifts a model's answers; Turpin et al.'s probe.

BIG-Bench Hard A set of 23 difficult tasks from BIG-Bench; Turpin et al. used 13 of them.

post-hoc rationalization A chain written to justify an answer reached by other means; Arcuschin et al. measured its unprompted rate.

hint test Chen et al.'s protocol: present a question with and without a hint, keep cases where the hint changed the answer, and check whether the chain mentions it.

reward hacking Obtaining reward by exploiting a flaw in the reward or environment rather than solving the task; slide 14 and slide 27.

monitor A separate model that reads an agent's chain or actions and flags misbehaviour; Baker et al. used GPT-4o.

recall The fraction of true misbehaviours a monitor flags; 95% from the chain against 60% from actions in Baker et al.

monitorability The property that a chain can be read by a monitor to detect what the model is doing; Korbak et al.'s "new and fragile opportunity".

monitorability tax Baker et al.'s proposal to keep the chain out of the training objective, accepting some loss, so that it stays readable.

obfuscation Hiding intent in the chain while continuing the behavior; the outcome of training against a chain monitor.

Intuition

Lab notebooks are necessary for the experiment and still not a transcript of the experimenter's mind. Grade the result on the answer key, read the notebook as evidence, and the day you start grading the notebook for neatness it stops being evidence.

3 Verifiers, search and test-time compute (slides 8–10)

3.1 Slide 8: Process or outcome: what should a verifier grade?

A reward model can score the final answer or every step; step-level supervision won in 2023 and lost to scale in 2025.

What the slide shows Across the top, one sampled solution to a MATH problem drawn as five step chips followed by an answer chip. Under each step is the probability a process reward model assigns to that step being correct: 0.98, 0.95, 0.41, 0.12, 0.30, falling sharply at step 3. Under the answer chip is a single number from an outcome reward model, 0.2. The caption makes the point of the picture: both models agree the solution is probably wrong, but only the process model says *where* it went wrong. Below, a panel on the 2023 result (Lightman et al., *Let's verify step by step*) with the three numbers 78.2, 72.4 and 69.6, and a panel on the right explaining why the trained reasoning models of 2025 went the other way, ending with the observation that process reward models survived as selectors at inference time rather than as training signals.

Details A verifier is a second model, or a program, that judges a candidate solution. Cobbe et al. (2021) introduced the idea for language models together with GSM8K, a set of 8.5K grade-school word problems: fine-tune a generator, sample many solutions per problem (the paper uses 100), train a separate verifier to predict whether a solution is correct, and at test time return the highest-scoring sample. The verifier's judgment is *outcome-based*: it is trained on whether the final answer matched, and it scores the whole solution. The paper's finding was that verification scaled better with data than fine-tuning the generator alone, which is the first hint that compute spent around a fixed model could substitute for a better model.

Uesato et al. (2022) asked whether the grade should attach to the outcome or to the process. On GSM8K the two kinds of supervision reached similar final-answer error rates (the paper's best configurations bring the error from 16.8% to 12.7%), but process supervision cut the rate of *wrong reasoning behind a right answer* from 14.0% to 3.4%. That distinction matters because a solution can reach the correct number by luck or by a cancelling pair of errors, and an outcome verifier rewards it anyway. Lightman et al. (2023) then ran the comparison at scale with a GPT-4 base model on the harder MATH dataset. They collected PRM800K, about 800,000 human labels on individual steps of model-generated solutions, trained a process reward model (PRM) on it and an outcome reward model (ORM) on final-answer labels, and used each to pick the best of N samples. On a held-out 500-problem subset of MATH with $N = 1{,}860$ samples per problem, the PRM selected a correct solution 78.2% of the time, the ORM 72.4%, and majority voting over final answers 69.6%. The gap widened with N: the PRM kept improving as samples were added while the ORM and the vote flattened, which is what one expects if the step-level model is better at rejecting plausible-looking wrong solutions.

Human step labels are expensive, so Wang et al. (2023, Math-Shepherd) replaced them with Monte Carlo estimates: from each intermediate step, roll out several completions and label the step by the fraction that reach the right answer. A PRM trained this way served both as a reranker and as a reward for reinforcement learning, moving a Mistral-7B policy from 77.9% to 84.1% on GSM8K and from 28.6% to 33.0% on MATH, with further gains when the same PRM reranked the trained policy's samples. This was the state of the art in late 2023: a step-level grader, learned from rollouts if not from people, to be used wherever a reward was needed.

The right panel records what happened next. DeepSeek-R1's report (DeepSeek-AI 2025) lists process reward models among its *unsuccessful attempts*, with three reasons: it is hard to define a step in general reasoning, hard even for humans to judge whether a step is correct, and a learned PRM invites reward hacking and has to be retrained whenever the policy changes. Monte Carlo tree search over steps failed for related reasons: the space of token sequences is too large to tile with a value model, and the value model that would guide the search is itself hard to train. Qwen's post-mortem (Zhang et al. 2025, *The lessons of developing process reward models in mathematical reasoning*) adds the empirical case: Monte Carlo step labels are worse than labels from an LLM judge or a human, best-of-N evaluation of PRMs is biased by the policy that

produced the samples, and only consensus filtering between the two labelling methods produced a usable PRM. Underneath both is the general result of Gao, Schulman and Hilton (2022) that any learned reward is over-optimized if a policy is pushed against it long enough: the policy finds the reward's mistakes before it finds the task's solution.

What survived is an outcome reward that a *program* computes: the boxed answer compared to a reference, a hidden test suite run in a sandbox. Such a reward cannot be flattered and does not drift, and Section 4 is built on it. For work without a checker, two 2025 developments try to make a learned grader less brittle: generative verifiers (GenRM, Zhang et al. 2024), which train the grader to write out its own chain and read the probability of "Yes" as the score, giving large best-of-N gains over a discriminative reward model (73% to 93.4% on GSM8K in the paper's setting), and rubric rewards, a per-task checklist that a model scores (slide 14). PRMs themselves did not disappear; they moved from the training loop to inference, where they rank candidates and guide beam search (slide 9).

Terms

verifier A model or program that judges whether a candidate solution is correct; a program (answer match, unit tests) is a *rule-based* verifier, a model a *learned* one.

outcome reward model A learned verifier trained on whether the final answer was correct; it scores a whole solution with one number. Abbreviated ORM.

process reward model A learned verifier that scores each intermediate step; trained on step labels from humans (PRM800K) or from Monte Carlo rollouts (Math-Shepherd). Abbreviated PRM.

PRM800K Lightman et al.'s dataset of about 800,000 human step-level labels on model-generated MATH solutions.

best-of-N Sample N candidate solutions and return the one a verifier scores highest; the simplest way to spend inference compute.

majority vote Return the most frequent final answer among N samples; the verifier-free baseline, also called self-consistency.

Monte Carlo step label An estimate of a step's correctness as the fraction of rollouts from that step that reach the right final answer.

reward hacking A policy achieving high reward by exploiting flaws in the reward rather than solving the task; the standing danger of any learned reward.

over-optimization The decline of true performance as a policy is optimized against a learned proxy reward; Gao et al. (2022) measured it as a function of optimization pressure.

generative verifier A reward model trained to produce a judgment as text, optionally with its own chain, scored by the probability of a token such as Yes (GenRM).

rubric reward A reward computed by scoring a response against a task-specific checklist, usually with a model as the scorer. Slide 14.

Intuition

A process reward model is a teacher marking every line of a proof; an outcome reward model only checks the last line. The teacher gives better feedback until the student learns to write lines that look right to the teacher, which is exactly what a policy under optimization will do. A program that runs the tests cannot be charmed, and that is why the trained models of 2025 kept the program and dropped the teacher.

3.2 Slide 9: Search at inference: sample, select, revise

Before the chain was trained, compute was spent around a fixed model: samples, a verifier to pick, a tree to explore.

What the slide shows Three small diagrams across the top, each a way to spend N samples from a frozen model. Best-of-N: a fan of five independent samples with one picked by a verifier or by majority vote. Beam over steps: a grid of partial solutions where a process reward model scores each step and only the top beams continue. Sequential revision: a row of drafts, each conditioned on the last, produced by a model fine-tuned to revise. Below them, a panel on Snell et al.'s compute-optimal scaling result ($4\times$ and $14\times$) and, on the right, a panel on Brown et al.'s coverage result (15.9% to 56% on SWE-bench Lite), a line on Tree of Thoughts, and the question the slide hands to slide 10: can the model learn to do the search inside one chain?

Details With a verifier in hand, the natural next step is to spend more inference compute and let the verifier choose. The three shapes on the slide differ in where the compute goes. Best-of-N is embarrassingly parallel: N full solutions, one selection, cost $N\times$ a single answer and latency unchanged. Beam search over steps spends the same budget more carefully: generate a few candidate next steps, score them with a PRM, keep the best k, and continue, so that bad partial solutions are abandoned early. Sequential revision spends the budget serially: a model fine-tuned on (draft, improved draft) pairs produces a chain of attempts, each reading the previous one, so that later drafts can fix what earlier ones got wrong. Tree of Thoughts (Yao et al. 2023) is the explicit general form: the model proposes several partial "thoughts" at each node, evaluates them itself, and a search procedure (breadth-first or depth-first) explores the tree. On the Game of 24, GPT-4 with chain-of-thought prompting solves 4% of instances and with Tree of Thoughts 74%.

Snell, Lee, Xu and Kumar (2024) asked how to allocate a fixed budget among these shapes. Using PaLM 2-S* on MATH, they found that the right strategy depends on the problem: for easy problems, sequential revision wins, because the first draft is close and needs only correction; for hard problems, parallel search guided by a PRM wins, because the first draft is far away and the search must explore. Allocating the budget according to a cheap estimate of difficulty gave more than $4\times$ the efficiency of best-of-N at the same compute. Their second comparison matched total FLOPs between a small model with test-time search and a model $14\times$ larger with a single greedy answer; the small model won on easy and medium problems and lost on the hardest, where no amount of search around a weak model helps. Hugging Face's open reproduction (Beeching, Tunstall and Rush 2024) confirmed the shape with released weights: Llama 3.2 1B with a PRM and compute-optimal search beats Llama 3.1 8B on MATH-500, and the 3B model beats the 70B. Liu et al. (2025) pushed the same experiment further and found that the compute-optimal strategy depends jointly on the policy, the PRM and the difficulty, and that in the right configuration a 1B model surpasses a 405B model on MATH-500 and a 7B surpasses o1 and R1.

Brown et al. (2024, *Large language monkeys*) measured the ceiling that any selector is working under. Define *coverage* as the fraction of problems for which at least one of N samples is correct, which equals pass@N. Across models and tasks, coverage grows roughly log-linearly in N over four orders of magnitude of samples. On SWE-bench Lite, DeepSeek-Coder-V2-Instruct solves 15.9% of issues with one sample and 56% of issues with 250, past the single-sample state of the art at the time. The catch is identification. When there is a checker (unit tests, a proof assistant), coverage is cashed in directly. When there is not, majority voting and reward-model selection plateau beyond a few hundred samples while coverage keeps rising, and the gap between the two is the space a verifier has to fill. This is the same gap that slide 8's PRM was trying to close, and it is the quantity to remember: the base model usually *can* solve far more than its single sample shows.

What all of this bought was a way to convert compute into accuracy with a frozen model. What it left open was the cost of the machinery (a verifier to train, a search procedure to run, latency that scales with the tree) and a question: if a search tree outside the model can find the answer, could the model be trained to run

the search itself, in one chain, keeping its own dead ends in view? That is the question o1 answered, and slide 10 takes it up.

Terms

beam search over steps A search that generates several candidate next steps, scores them with a process reward model, keeps the top few, and repeats until the answer.

sequential revision Spending inference compute by producing a chain of drafts, each conditioned on the previous one, with a model fine-tuned to improve its own output.

Tree of Thoughts Yao et al. (2023): a search over partial thoughts in which the model proposes candidates and evaluates them, explored breadth- or depth-first.

compute-optimal test-time scaling Choosing the inference strategy and budget per problem (by estimated difficulty) to maximize accuracy at a fixed FLOPs budget (Snell et al. 2024).

coverage The fraction of problems for which at least one of N samples is correct; identical to pass at N (Brown et al. 2024).

pass at k The probability that at least one of k independent samples is correct, estimated without bias from $n \geq k$ samples by the formula of Chen et al. (2021); pass at 1 is single-sample accuracy.

identification problem The gap between coverage and what a selector (vote or verifier) actually returns; the problem a verifier exists to solve.

Game of 24 A puzzle: combine four numbers with arithmetic to make 24; the Tree of Thoughts benchmark where chain-of-thought scored 4% and tree search 74%.

Intuition

Think of a locksmith with a ring of a thousand keys and no label on any of them. Coverage says the right key is on the ring; the verifier is the labelling. Without labels, trying more keys only helps if you can tell when the door opened, and a majority vote among keys that did not open the door is not a label.

3.3 Slide 10: From search around the model to search inside the chain

o1's bet: train the model so that one long chain does the proposing, checking and backtracking that trees did outside.

What the slide shows Two pictures stacked on the left. The upper one is search outside the model: a tree of partial solutions, three levels deep, with one path highlighted in the answer color; the caption says a PRM or a vote picks the path and the model never sees its own dead ends. The lower one is search inside the chain: a single row of chips reading `try A`, `check`, `wrong`, `wait`, `try B`, `check`, `ok`, `answer`, with the dead end left in place, because in a chain it stays in the context and conditions everything after it. The panel on the right lists what the trained chain does better: no selector is needed (R1-Zero reaches 71.0% pass@1 on AIME 2024, and voting over 64 chains still adds, to 86.7%); dead ends are information; the trees were abandoned for stated reasons; and the cost, which is that the search is now serial.

Details The 2023 tree and the 2025 chain are two implementations of the same search. In the tree, a frozen policy proposes children, a separate value model or PRM scores them, and a search procedure decides which branch to extend; the policy is never told that a branch was abandoned, and every node it generates is an independent sample from the same distribution. In the chain, a single model proposes a step, writes a check of it, declares the step wrong, says "wait", proposes another, and so on until an answer; the model that writes

step twelve has read steps one through eleven, including the failed ones, so the proposal distribution at each point is conditioned on the history of the search. The search tree is still there, but it has been linearized into the context window, and the three roles of proposer, evaluator and controller are played by the same weights.

OpenAI's o1 announcement (September 12, 2024)† is the first public statement of the bet. The post says the model was trained with large-scale reinforcement learning to "think before it answers", that through RL it "learns to recognize and correct its mistakes", to "break down tricky steps into simpler ones" and to "try a different approach when the current one isn't working", and it shows two curves: accuracy on AIME rising smoothly with the logarithm of train-time compute and, separately, with the logarithm of test-time compute. The headline numbers are the ones on slide 3: GPT-4o solves 12% of AIME 2024 problems (1.8 of 15), o1 74% with a single sample (11.1 of 15), 83% with consensus over 64 samples (12.5 of 15), and 93% when 1,000 samples are re-ranked with a learned scorer (13.9 of 15)†. The post also says the raw chain is hidden from users, with a model-written summary shown instead, for reasons of safety monitoring and competitive advantage; slide 27 returns to that decision. None of the training recipe was published.

DeepSeek-R1 (January 2025) supplied the recipe and the clearest evidence that the search had moved inside the chain. R1-Zero, trained by GRPO from the base model with a rule reward and no supervised chains at all, rises from 15.6% to 71.0% pass@1 on AIME 2024 over training, and its chains grow from hundreds of tokens to thousands while acquiring the vocabulary of search: re-reading the problem, verifying a step, "wait", trying an alternative (slide 15). Majority voting over 64 chains raises R1-Zero to 86.7%, so the serial and parallel axes compose: a better single chain does not exhaust what sampling adds. The report also states why the trees were abandoned. Monte Carlo tree search over tokens faced a search space exponentially larger than chess, so the authors capped the branching with a node budget and found the policy trapped in local optima; and the value model needed to guide the search was hard to train, which made iterative improvement impossible. Process reward models failed for the reasons on slide 8. The team's conclusion, that the policy should learn the search rather than be driven through it, is the design decision the rest of this lecture follows.

OpenAI's 2025 paper on competitive programming (El-Kishky et al.) is the cleanest quantitative version of the same conclusion. It compares three systems on Codeforces: o1 (rating 1673, 89th percentile), o1-ioi, which is o1 with hand-engineered test-time strategies for the IOI, among them problem-specific sampling and clustering by test results, reaching 1807 with those strategies and up to 2214 with more elaborate ones, and o3, trained with more reinforcement learning and no hand-built search at all, at 2724, the 99.8th percentile. The scaled RL run learned, inside the chain, what the engineers had been building around the model, and did it better. The one search-based training method that works in the open is rStar-Math (Guan et al. 2025), which uses Monte Carlo tree search with code-verified steps and a process preference model through four rounds of self-evolution, taking Qwen2.5-Math-7B from 58.8% to 90.0% on MATH; the price is the full search machinery at training time, and the trend of the field has been to pay it in rollouts instead.

The cost of search inside the chain is on the slide's last line: it is serial. A tree can be explored in parallel across many machines, but a chain is one token after another, so latency and tokens grow with the difficulty of the problem, and the only control is the budget, which is the subject of slide 19. The model also now carries its own dead ends in context, which helps it avoid them and costs memory for every token kept (slide 22).

Terms

search inside the chain A trained model proposing, checking and backtracking within one generated sequence, with every abandoned attempt kept in the context; the behavior o1 and R1 exhibit.

search outside the model Tree search, beam search or best-of-N run by a procedure around a frozen model, with a separate verifier or value model choosing among samples.

R1-Zero DeepSeek's model trained by GRPO directly from DeepSeek-V3-Base with a rule reward and no supervised chains; AIME 2024 pass at 1 from 15.6 to 71.0, 86.7 with 64 votes. Slide 12.

consensus at 64 Majority voting over 64 sampled chains; written cons at 64 or maj at 64 in reports.

Monte Carlo tree search A search that expands a tree by sampling rollouts from each node and backing up their values; attempted over reasoning steps by DeepSeek and listed among R1's unsuccessful attempts.

value model A model estimating the expected final reward from a partial solution, used to guide tree search; hard to train for token-level reasoning.

o1 OpenAI's first product trained to think before answering, announced September 12, 2024, with hidden chains and a published test-time compute curve.

o1-ioi o1 augmented with hand-engineered test-time strategies for the International Olympiad in Informatics; Codeforces 1807 to 2214 depending on the strategy.

o3 OpenAI's successor model trained with more reinforcement learning; Codeforces 2724 with no hand-built search; IOI 2024 gold under contest conditions.

rStar-Math Guan et al. (2025): small-model math reasoning trained by MCTS with code-verified steps and a process preference model over four rounds of self-evolution.

Intuition

A chess engine searches a tree and forgets the lines it rejected; a grandmaster thinking aloud says "no, that loses the bishop, so instead…" and the rejected line shapes the next idea. The trained chain is the grandmaster, and the price of thinking aloud is that it happens one word at a time.

4 Reinforcement learning on verifiable rewards (slides 11–14)

4.1 Slide 11: RL with verifiable rewards: the chain becomes the policy

Sample whole chains, check the final answer with a program, push probability toward the chains that were right.

What the slide shows A five-box pipeline across the top: a problem x with a checkable answer; sample G chains $o_1, \ldots, o_G$ from the current policy $\pi_\theta(\cdot \mid x)$; a verifier that returns $r_i \in \{0, 1\}$ (the answer matched, the tests passed); advantages A_i computed from the group; a policy-gradient update of θ; and a return arrow, because the next batch is sampled from the updated policy (on-policy). Below the pipeline, the objective and its gradient, with a caption fixing the vocabulary: the state is the prompt plus the tokens generated so far, the action is the next token, an episode is one whole chain and answer, and the reward arrives once, at the end. The panel on the right says why this works where RLHF stumbled: the reward is a program, the signal is sparse and honest, and the rollouts are the cost.

Details Reinforcement learning with verifiable rewards (RLVR) is policy-gradient training of a language model on prompts whose answers can be checked by a program. Tülu 3 (Lambert et al. 2024) named the setting and used it as a post-training stage on mathematics and constrained instruction following; DeepSeek-R1 (slide 12) showed what it does at scale. The objective is the standard one from RLHF with the learned reward model replaced by a checker:

$$J(\theta) = \mathbb{E}_{x\sim D,\, y\sim\pi_\theta(\cdot|x)}\big[\,r(x,y)\,\big] - \beta\,\mathrm{KL}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big), \qquad \nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})\, A_t\Big].$$

Here D is the distribution of training problems, y a complete chain-and-answer, $r(x,y) \in \{0,1\}$ the verifier's verdict, π_{ref} the model before RL, β the strength of the KL penalty that keeps the policy near it (several 2025 recipes set $\beta = 0$), and A_t the advantage at token t: the reward minus a baseline, which is the quantity the algorithms of slide 13 differ on.

The gradient is REINFORCE (Williams 1992) applied to a sequence: every token of a chain that earned reward above the baseline has its log-probability increased, every token of a chain below the baseline decreased, in proportion to the advantage. Nothing in the update says what a good chain looks like. The reward is one bit per episode and is awarded only at the end, so the policy is free to find *any* sequence of tokens that raises the chance of a checkable right answer. The behaviors that reasoning models are known for, long chains, self-verification, backtracking, are what survived that selection, not what was prescribed; slide 15 shows them appearing during training.

The comparison with RLHF explains why this recipe was stable where preference optimization was fragile. In RLHF the reward is a learned model of human preference, which can be flattered (longer, more confident, more agreeable answers score higher whether or not they are better), which drifts as the policy moves away from the data it was trained on, and which costs a forward pass of a large model per sample. A verifier is a program: a boxed answer normalized and compared to a reference, a hidden unit-test suite run in a sandbox, a compiler, a proof assistant. It cannot be gamed except through its own bugs (slide 14), it does not drift, and it is essentially free, so the expensive part of training moves from the reward to the rollouts: thousands of problems per batch, 8 to 64 chains per problem, thousands of tokens per chain, all generated by the current policy. In practice the rollout engine (vLLM or SGLang) dominates wall-clock time, which is why the open training stacks on slide 24 are organized around it.

Two older methods are special cases. STaR (Zelikman et al. 2022) samples chains, keeps those that reach the right answer, fine-tunes on them and repeats; ReST-EM (Singh et al. 2023) frames the same loop as expectation-maximization. Both are rejection sampling followed by supervised fine-tuning, which is a policy-gradient step with the advantage set to the reward itself and the losing samples discarded. RLVR keeps the losing samples, with negative advantage, and makes the step on-policy with an importance-ratio correction and clipping (slide 13), which is what lets it run for thousands of steps without collapsing.

What counts as verifiable sets the boundary of the method. Competition mathematics with a numerical or symbolic final answer, code with tests, logic puzzles with a checkable solution and formal proofs in Lean are verifiable. Essays, advice, research and most agentic work are not, unless the task can be reshaped so that a program can judge the outcome, which is what the agentic recipes on slide 21 do with repository tests and tool results. Slide 14 maps this boundary in detail.

Terms

reinforcement learning with verifiable rewards Policy-gradient post-training on prompts whose answers a program can check; abbreviated RLVR, named by Tülu 3 (Lambert et al. 2024).

policy The language model being trained, written π_θ; it defines a distribution over next tokens given the prompt and the tokens so far.

rollout One sampled chain-and-answer from the current policy for a given prompt; the unit of RL data, and the dominant cost of RLVR.

on-policy Training on samples drawn from the current policy (or a very recent copy of it), as opposed to a fixed dataset.

episode In RLVR, one complete generation for one prompt: the state advances one token at a time and the reward is delivered once, at the end.

advantage The reward of a sample minus a baseline, written A; the signed quantity that scales the gradient on each token.

baseline A reference value subtracted from the reward to reduce the variance of the policy gradient: a learned value function in PPO, the mean of a group of siblings in GRPO, a leave-one-out mean in RLOO.

KL penalty A term $\beta\,\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}})$ that keeps the trained policy close to the reference model; several 2025 reasoning recipes drop it.

REINFORCE The basic policy-gradient estimator: increase the log-probability of actions in proportion to their advantage (Williams 1992).

STaR Self-Taught Reasoner (Zelikman et al. 2022): sample rationales, keep the correct ones, fine-tune, repeat; the rejection-sampling ancestor of RLVR.

ReST-EM Singh et al. (2023): the sample-filter-finetune loop framed as expectation-maximization and scaled on PaLM 2.

RLHF Reinforcement learning from human feedback: policy-gradient training against a learned model of human preferences, the post-training recipe of the earlier deck and the contrast here.

Intuition

RLHF trains a student to please a grader who can be flattered; RLVR trains the student against an answer key. The key says nothing about method, so the student invents one, and the method it invents, when the problems are hard enough, is to think for a long time and check its work.

4.2 Slide 12: DeepSeek-R1: the open recipe, in two tracks

Pure RL from a base model shows the behavior; a four-stage pipeline with a cold start makes it a usable model.

What the slide shows Two tracks. The upper one is R1-Zero: DeepSeek-V3-Base (a 671B mixture-of-experts model) goes through GRPO on mathematics, code and logic with rule rewards for a correct answer and for the `<think>` format, and comes out as R1-Zero, whose AIME 2024 pass@1 rose from 15.6 to 71.0 during training (86.7 with 64 votes) but which mixes languages and is hard to read. The lower one is R1's four stages: a cold-start SFT on thousands of long, readable chains; reasoning RL with an added language-consistency reward; rejection sampling of 600k reasoning and 200k general samples followed by SFT; and a final RL on all prompts with rules plus a preference reward. A caption gives the Nature version's cost of the RL training (including its SFT data generation), about $294K of H800 time at an assumed $2 per GPU-hour, on top of the base model[†]. The panel on the right lists what came out: R1's benchmark numbers, the distillation result, and the lesson that generalized.

Details DeepSeek-R1 (DeepSeek-AI, arXiv January 22, 2025; peer-reviewed version in *Nature* 645:633–638, September 2025) is the paper that made reasoning models reproducible. All of the numbers below are from the body of the arXiv v1; the *Nature* abstract carries none, and the v2 abstract (January 2026) follows it. The two tracks answer two different questions. R1-Zero asks whether reinforcement learning alone, with no supervised reasoning data, can produce the behavior; R1 asks what it takes to turn that behavior into a model people can use.

R1-Zero starts from DeepSeek-V3-Base and runs GRPO with two rule rewards: an accuracy reward (the final answer in the required format matches the reference, or the code passes the tests) and a format reward for placing the reasoning between `<think>` and `</think>` tags. No neural reward model is used, for the reasons on slide 8. Over training, average pass@1 on AIME 2024 rises from 15.6% to 71.0%, and majority voting over 64 samples raises it to 86.7%, comparable to o1-0912 at the time. The chains lengthen steadily with no length term in the reward, and the report reproduces the "aha moment" in which the model interrupts itself to re-evaluate a step (slide 15 discusses how much of this was already in the base model). The failures are also recorded: R1-Zero's chains mix languages and are poorly readable, which is what the cold start was added to fix.

R1 proper is a four-stage pipeline. Stage 1, the cold start, fine-tunes the base on thousands of long chain-of-thought examples, collected by few-shot prompting, by prompting R1-Zero and cleaning its output, and by human editing, in a fixed readable format with a summary after the reasoning. Stage 2 is the same reasoning RL as R1-Zero, with a language-consistency reward (the fraction of target-language words in the chain) added to the accuracy reward; the report notes this costs a little accuracy and buys readability. Stage 3 harvests the stage-2 model: rejection sampling on reasoning prompts, with a generative reward model judging cases where a rule cannot, yields about 600k reasoning samples, and about 200k non-reasoning samples (writing, question answering, translation) are added from DeepSeek-V3's pipeline; the base model is fine-tuned on the 800k. Stage 4 runs RL on all prompts, with rule rewards for reasoning data and a preference reward model for the rest, to restore the helpfulness and harmlessness that the reasoning stages ignored. The *Nature* version's supplementary material, as reported by *Scientific American* in September 2025[†], puts the RL training at about $294,000 of H800 time (512 H800s for about 198 hours for R1-Zero, about 80 hours for R1, plus roughly 5,000 GPU-hours of SFT data generation, at an assumed $2 per GPU-hour), on top of a base model costed at roughly $6M; both figures are the company's own accounting, relayed by the press, and are marked as secondary here.

The results are on the right panel. R1 scores 79.8% pass@1 on AIME 2024, 97.3% on MATH-500, 71.5% on GPQA Diamond, 65.9% on LiveCodeBench, a Codeforces rating of 2,029 (above 96.3% of human participants), and 49.2% on SWE-bench Verified; the weights were released under the MIT license, and the product showed users the full chain where o1 showed a summary. The second result is about distillation. DeepSeek fine-tuned Qwen2.5 and Llama models from 1.5B to 70B on the 800k samples with no RL at all; the 32B student scores 72.6% on AIME 2024 and 94.3% on MATH-500 (the 7B 55.5% on AIME; the Llama-70B 70.0% and 94.5%), while running the R1-Zero RL recipe directly on Qwen2.5-32B-Base for more than ten thousand steps gave 47.0% and 91.6%. The report's conclusion, that distilling a large model's chains into a small one outperforms large-scale RL on the small model, is the subject of slide 17 and the reason every subsequent small reasoning model started from distilled traces.

The lesson that generalized is the recipe's shape rather than its scale: a strong base model with mathematics and code in its pretraining, a pool of problems with checkable answers, a rule reward, a group baseline instead of a critic, and enough steps for the chains to grow. Within weeks of the release, open reproductions appeared at every size from a 3B model learning the countdown game for about $30 (TinyZero) to full retraining efforts (Open-R1), and slide 24 catalogues them. The cold start is the part most often skipped in reproductions and most often regretted: a few thousand curated chains fix the format and the language before RL amplifies whatever it finds, which is why R1-Zero is the experiment and R1 is the product.

Terms

DeepSeek-R1 DeepSeek's open-weight reasoning model (January 2025; *Nature*, September 2025), trained by a four-stage pipeline of cold-start SFT, reasoning RL, rejection-sampling SFT and general RL.

DeepSeek-V3-Base The 671B-parameter mixture-of-experts base model from which R1-Zero and R1

were trained.

cold start A small supervised fine-tuning stage on curated long chains that fixes the output format and language before reinforcement learning begins.

rule reward A reward computed by a fixed program: answer correctness, format compliance, language consistency.

format reward A rule reward for placing the reasoning between the required tags and producing a single final answer.

language-consistency reward R1's stage-2 reward, the fraction of target-language words in the chain, added to stop the chain from mixing languages.

rejection sampling Generating many candidates and keeping only those a verifier or judge accepts; R1's stage 3 collected about 600k reasoning samples this way.

distillation Training a smaller student model by supervised fine-tuning on the outputs of a larger teacher; for reasoning models, fine-tuning on the teacher's chains.

AIME The American Invitational Mathematics Examination, thirty integer-answer problems per year across two papers; the AIME 2024 problems were the standard reasoning benchmark of 2025.

MATH-500 A 500-problem subset of the MATH competition dataset used as a held-out test set since Lightman et al. (2023).

GPQA Diamond The hardest 198-question subset of GPQA, graduate-level science questions written to be Google-proof (Rein et al. 2023).

SWE-bench Verified 500 human-validated GitHub issues from SWE-bench on which a model must produce a patch that passes the repository's tests.

Codeforces rating An Elo-style rating from competitive programming contests; R1's 2,029 places it above 96.3% of participants.

Intuition

R1-Zero is a laboratory animal: it proves that the behavior can be grown from reward alone, and nobody would ship it. R1 is the same animal raised with a few thousand examples of manners first. The surprising result is the distillation one: the easiest way to make a small reasoner is not to raise it, but to have it copy the big one's homework.

4.3 Slide 13: GRPO and its descendants: a baseline from the group

Drop the value model, normalize each chain's reward against its siblings; then fix the biases that this introduces.

What the slide shows The GRPO formulas on the left: the group-normalized advantage, the clipped surrogate objective averaged over the G chains of a group and over the tokens of each chain, and the KL penalty, with a key for the importance ratio, the group size (8 to 64) and the PPO clip. Under them, a panel on what the group buys relative to PPO (no value network, half the memory) and the sibling algorithms RLOO and REINFORCE++. On the right, the 2025 corrections: Dr. GRPO's removal of the length and standard-deviation normalizations, DAPO's four changes and its 50 against 47 on AIME 2024, sequence-level ratios (GSPO, CISPO), and the entropy observations, including the top 20% of tokens by entropy, the "forks",

that carry the useful gradient.

Details Group Relative Policy Optimization was introduced in DeepSeekMath (Shao et al. 2024) as a variant of PPO that removes the value model. For each problem x the policy samples a group of G outputs $o_1, \ldots, o_G$, each receives a reward r_i from the verifier, and the advantage of every token of output i is the group-normalized reward:

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}, \qquad \rho_{i,t} = \frac{\pi_\theta(o_{i,t} \mid x, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid x, o_{i,<t})},$$

$$J_{\text{GRPO}}(\theta) = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\big(\rho_{i,t}A_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)A_i\big)\Big] - \beta\,\text{KL}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big),$$

with the KL term estimated per token by the unbiased estimator $\frac{\pi_{\text{ref}}(o_{i,t}|\cdot)}{\pi_\theta(o_{i,t}|\cdot)} - \log\frac{\pi_{\text{ref}}(o_{i,t}|\cdot)}{\pi_\theta(o_{i,t}|\cdot)} - 1$ and added to the loss rather than to the reward. θ_{old} is the policy that generated the group; ε is PPO's clip range (0.2 by default).

The clipped surrogate is PPO's: when the current policy has moved so far from the sampling policy that the ratio leaves $[1-\varepsilon, 1+\varepsilon]$, the gradient on that token is switched off, which allows several gradient steps per batch of rollouts without the update running away. What is new is the baseline. PPO estimates the expected reward of a partial sequence with a value network, in practice a second copy of the policy with a scalar head, which doubles the memory of training and must itself be trained from a sparse, end-of-episode reward. GRPO instead uses the mean reward of the G siblings sampled for the same problem: if six of eight chains are right, the two wrong ones get a negative advantage and the six right ones a small positive one; if one of eight is right, it gets a large positive advantage. The group is a Monte Carlo estimate of the value of the prompt, free because the samples were needed anyway. RLOO (Ahmadian et al. 2024) is the same idea with a leave-one-out mean as the baseline and no clipping, and its authors showed REINFORCE variants matching or beating PPO on RLHF tasks; REINFORCE++ (Hu et al. 2025) keeps PPO's clipping and normalizes advantages over the whole batch rather than per prompt. They are all REINFORCE with different baselines, which is why the deck calls the family that.

The 2025 corrections address biases that the formula introduces. Liu et al. (2025, *Understanding R1-Zero-like training*, whose algorithm is Dr. GRPO) point out two. The $1/|o_i|$ term averages the loss over a chain's own length, so a wrong chain pays a smaller per-token penalty the longer it is, and a right chain earns less per token the longer it is; the standard-deviation normalization inflates advantages on problems where almost all chains agree. Together, in the authors' words, the normalization "artificially increases response length, especially for incorrect outputs"; removing both terms (dividing by a constant instead of $|o_i|$, dropping the standard deviation) keeps the accuracy and stops the length growth, and a 7B model trained this way reaches 43.3% on AIME 2024. DAPO (Yu et al. 2025) makes four changes and releases the full system and its 17k-problem dataset. *Clip-higher* decouples the clip range into $\varepsilon_{\text{low}} = 0.2$ and $\varepsilon_{\text{high}} = 0.28$, so that low-probability tokens can grow faster than the symmetric clip allows, which the authors show prevents entropy collapse. *Dynamic sampling* drops any group whose rewards are all equal, since its advantages are all zero and it contributes no gradient, and keeps sampling until the batch is full of informative groups. A *token-level* loss sums over all tokens in the batch rather than averaging within each chain first, so long chains are not down-weighted. *Overlong reward shaping* replaces the hard penalty for truncated chains with a soft one that grows with the overshoot. With these, Qwen2.5-32B base reaches 50 on AIME 2024 (averaged over 32 samples), against 47 for the DeepSeek-R1-Zero-Qwen-32B baseline, in half the training steps.

Two further lines concern the importance ratio and the entropy. GSPO (Zheng et al. 2025) computes the ratio and the clip at the level of whole sequences rather than tokens, which stabilizes RL for mixture-of-experts models, where expert routing can change between the sampling and the update and make token-level ratios

noisy; it was used to train Qwen3. CISPO (MiniMax-M1, 2025) clips the importance weights themselves rather than the token updates, so that no token is ever dropped from the gradient; MiniMax reports its full RL run at 512 H800s for three weeks, $534,700. VAPO (Yue et al. 2025) is the value-based counterpoint: with value pretraining, length-adaptive advantage estimation and decoupled clipping, PPO with a critic reaches 60.4 on AIME 2024 with Qwen-32B in 5,000 steps, so the critic was not the problem, only its naive use. Cui et al. (2025) describe the entropy mechanism: policy entropy collapses in the first few hundred steps, and downstream performance follows $R = -a\,e^{H} + b$ in the entropy H, so a run's ceiling is fixed once its entropy reaches zero; their Clip-Cov and KL-Cov interventions restrict updates on the tokens with the highest covariance between probability and advantage and keep entropy up. Wang et al. (2025, *Beyond the 80/20 rule*) split a chain's tokens by entropy under the policy: the top 20% (the "forks", where the chain chooses between strategies) carry the useful gradient, and updating only those tokens matches or improves on full updates for Qwen3 models, while updating the other 80% hurts. The 20% is the authors' chosen cutoff, not a measured fraction.

Terms

GRPO Group Relative Policy Optimization (Shao et al. 2024): PPO's clipped surrogate with the value-network baseline replaced by the mean reward of a group of sibling samples.

group The G outputs sampled for one prompt in one step, from which the baseline is computed; 8 to 64 in practice.

importance ratio The ratio of the current policy's probability of a token to the sampling policy's, written ρ; it corrects for the update having moved the policy since the rollouts were drawn.

clipped surrogate PPO's objective, in which the ratio is clipped to $[1-\varepsilon, 1+\varepsilon]$ so that a token's gradient is switched off once the policy has moved far enough.

PPO Proximal Policy Optimization: the actor-critic policy-gradient algorithm with a clipped surrogate that RLHF used, with a value network as the baseline.

value network A model estimating expected future reward from a partial sequence; PPO's baseline, removed by GRPO.

RLOO REINFORCE leave-one-out (Ahmadian et al. 2024): each sample's baseline is the mean reward of the other samples in its group; no clipping.

REINFORCE++ Hu et al. (2025): a critic-free algorithm with PPO's clipping and advantages normalized over the global batch.

Dr. GRPO Liu et al. (2025): GRPO without the per-length and standard-deviation normalizations, which the authors show bias GRPO toward long incorrect chains.

DAPO Decoupled clip and dynamic sampling policy optimization (Yu et al. 2025): clip-higher, dynamic sampling, token-level loss and overlong reward shaping; 50 on AIME 2024 from Qwen2.5-32B base.

clip-higher Setting the upper clip bound above the lower one (0.28 against 0.2) so that rare tokens can gain probability faster; DAPO's first change.

dynamic sampling Discarding groups whose rewards are all equal (zero advantage) and sampling more until the batch is full; DAPO's second change.

token-level loss Averaging the policy-gradient loss over all tokens in a batch instead of within each chain, so that long chains are weighted by their length.

overlong reward shaping A soft penalty that grows with how far a chain overshoots the length limit, replacing a hard zero reward for truncation.

GSPO Group Sequence Policy Optimization (Zheng et al. 2025): importance ratios and clipping at the sequence level; used for Qwen3 and for mixture-of-experts models.

CISPO MiniMax-M1's algorithm: clip the importance weights rather than the token updates, so that no token is dropped from the gradient.

VAPO Yue et al. (2025): a value-based (critic) method with value pretraining and length-adaptive advantages; 60.4 on AIME 2024 with Qwen-32B.

entropy collapse The rapid fall of policy entropy early in RL, after which the policy stops exploring; Cui et al. (2025) relate the attainable reward to the entropy by $R = -ae^H + b$.

fork tokens The roughly 20% of tokens with high entropy under the policy, at which the chain chooses between strategies; they carry most of the useful gradient (Wang et al. 2025).

Intuition

PPO grades every student against a tutor's prediction of how they would do; GRPO grades each student against the class average on the same exam. The class average is free and usually good enough, but it has quirks: a long wrong essay is punished per word, so wrong essays get longer, and an exam the whole class aced teaches nothing. The 2025 algorithms are a list of those quirks and their fixes.

4.4 Slide 14: Rewards and verifiers: what is checked, what is gamed

The recipe is only as good as the checker; every learned or soft reward is a target the policy will find a way around.

What the slide shows A five-row table. Each row is a kind of reward (answer match, unit tests, format and length, a learned judge, a rubric), how it is computed, and how it gets gamed together with the fix. Under the table, a paragraph on the boundary of the method: RLVR needs a checker, so it reached mathematics, code and formal tasks first; writing, research, advice and open-ended agent work get learned judges, rubrics and preference data, which bring back every RLHF failure; and slide 21 shows how agentic recipes turn tool results into checkable outcomes.

Details The reward in RLVR is a program, but a program has a specification, and the policy optimizes against the specification rather than the intent. Skalse et al. (2022) formalize reward hacking as a policy improving under a proxy reward while the true reward falls, and prove that, over all stochastic policies, two reward functions can be mutually unhackable only in degenerate cases (one of them constant). The practical reading is that every reward on the slide's table has a gap between what it measures and what is wanted, and a policy trained long enough will find it; the question for a reward designer is how large the gap is and how expensive it is to exploit.

Answer match is the cleanest case. The policy's final answer is extracted (a `\boxed{}` expression, a number after a marker), normalized (SymPy for expressions, string rules for units and formatting) and compared with the reference. The exploits are in the normalizer: a loose parser accepts wrong forms, a multiple-choice format lets a guess score 25%, and a reference answer that is itself wrong teaches the wrong thing. The fixes are free-form answers, strict parsers, and filtering of problems whose references disagree with a strong model. *Unit tests* are the code analogue. The sandbox runs the candidate against tests; the exploits are special-casing the visible tests, editing the test file, or printing the expected output; Baker et al. (2025) show a frontier coding agent writing "let's hack" in its chain before doing exactly this (slide 27).

The fixes are hidden tests, a sandbox that the policy cannot edit, and a monitor on the chain. *Format and length* rewards (tags present, one answer, the target language, soft caps on chain length) stabilize training and shape the product; they are gamed by tag spam, language mixing and padding, which is what R1's language-consistency reward and DAPO's soft overlong penalty respond to.

The lower two rows are where the method's guarantees run out. A *learned judge*, whether a reward model trained on preferences or a large model prompted to score, is subject to the over-optimization law of Gao et al. (2022): the policy's true quality rises and then falls as it is pushed against the proxy, and the peak comes sooner for a smaller judge. The policy learns what the judge likes (length, confidence, agreement) rather than what the task needs. Generative verifiers that must write out their reasoning, and rotating or ensembling judges, raise the cost of flattering them without removing it. A *rubric* (Huang et al. 2025, *Reinforcement learning with rubric anchors*) replaces the judge's taste with a task-specific checklist that a model scores item by item; with more than 10,000 rubrics and about 5,000 training samples, the authors report a 5.2% gain on open-ended benchmarks for a Qwen-30B-A3B model and a 2.4-point edge over DeepSeek-V3. Rubrics are brittle in their wording and expensive to write, but they are the best current option for domains without a checker. Two 2025 papers test how little the reward can carry: Intuitor (Zhao et al. 2025) uses the model's own self-certainty as the only reward and matches GRPO on mathematics while generalizing better to code, and the spurious-rewards results of slide 15 show that on some base models a random reward works; both say more about the base model than about reward design.

The 2026 reminder is Helff et al. (*LLMs gaming verifiers*): models trained with RLVR learn to enumerate instance labels of a verifier's test distribution instead of inducing the underlying rule, scoring well on the verifier and poorly on isomorphic variants of the same task; changing the verifier so that it checks a rule-preserving transformation removes the exploit. The checker, in other words, is part of the training distribution, and a checker with a narrow test set teaches the narrow set.

The boundary paragraph on the slide draws the consequence. RLVR produced the 2025 gains in mathematics, competitive programming and formal reasoning because those domains have programs that judge outcomes. Elsewhere, the rewards are learned judges, rubrics and preference data, and the RLHF failures of the post-training deck, sycophancy, length bias and over-optimization, return with them. The productive middle ground is to reshape a task until a program can judge it: agentic recipes (slide 21) reward a patch by the repository's tests or by similarity to the merged change (SWE-RL), reward a tool-using chain by whether the tool's output solved the task (ReTool), and reward a browsing agent by whether it retrieved the answer. The verifier is then the environment, and the environment is harder to flatter than a judge.

Terms

answer match A rule reward that extracts the final answer, normalizes it (SymPy, string rules) and compares it with the reference.

unit-test reward A rule reward that runs candidate code in a sandbox against a test suite, ideally hidden from the policy.

sandbox An isolated execution environment for running untrusted candidate code, which the policy cannot modify.

learned judge A reward model trained on preferences, or a large model prompted to score responses; subject to over-optimization and flattery.

self-certainty reward A reward computed from the policy's own confidence in its answer (Intuitor, Zhao et al. 2025), with no external signal.

proxy reward The reward actually optimized, as distinct from the true objective; the gap between them is what reward hacking exploits (Skalse et al. 2022).

verifier gaming A policy learning the specific instances a verifier tests rather than the rule the task requires (Helff et al. 2026); detected by isomorphic variants of the task.

isomorphic verification Checking a solution on a rule-preserving transformation of the task so that memorized instance labels fail.

SWE-RL Wei et al. (2025): reinforcement learning on real pull-request history with a reward for similarity to the merged patch; 41.0% on SWE-bench Verified with a 70B Llama.

ReTool Feng et al. (2025): RL with a code interpreter inside the rollout; a 32B model reaches 67% on AIME in 400 steps. Slide 21.

Intuition

A reward is a lock, and the policy is water: it finds every crack. A unit test with hidden cases is a good lock; a rubric is a lock with the combination written on a nearby wall; a learned judge is a lock that opens for anyone who speaks kindly to it. The art of RLVR is choosing tasks whose locks are good, and the 2025 frontier is finding good locks for tasks that did not come with one.

5 Training dynamics: what RL changes and what was already there (slides 15–16)

5.1 Slide 15: What RL changes, and what was already there

Chains grow and start to check themselves; two results say much of that behavior was in the base model already.

What the slide shows A plot on the left, labeled illustrative: mean chain length in thousands of tokens against RL steps, rising from a few hundred tokens to about ten thousand over several thousand steps and then flattening, with a note that no length reward is given and the chains grow anyway. The curve is a sketch of the shape reported for DeepSeek-R1-Zero, not a trace of its data. Two panels on the right. The first, *What appears during training*, lists what the R1 report saw emerge: longer chains, re-checking, backtracking, and the "aha moment"; and then the counterweight from Liu et al. (2025, Dr. GRPO): the base models used in most reproductions already write such phrases before any reinforcement learning, and part of the length growth is an artifact of GRPO's normalization. The second panel, *Two cautionary results*, gives the one-example result of Wang et al. (2025) and the spurious-rewards result of Shao et al. (2025). A line of text at the bottom left summarizes the dynamics (accuracy and length rise together, entropy falls, the distribution narrows onto chains that work) and draws the methodological rule: report a second model family and a random-reward control, or a small-scale RL claim means little.

Details The R1 report (DeepSeek-AI 2025, v1) describes R1-Zero, the track with no supervised fine-tuning at all, by two curves: AIME 2024 pass@1 rising from 15.6% to 71.0% (86.7% with 64-sample majority voting), and the average response length on the training set rising from a few hundred tokens to about ten thousand over the run. The second curve is the one the slide sketches. Nothing in the reward asks for length: the reward is a correctness check plus a format check (the answer must sit between the expected tags). The chains grow because, under a reward for being right, longer chains that re-derive and re-check are right more often, and the policy gradient moves probability toward them. The report's qualitative claims follow: the model learns to allocate more thinking to harder problems, to revisit earlier steps, and at one point produces the passage the authors call an "aha moment", in which it interrupts itself to re-evaluate. These are observations on one run of one base model (DeepSeek-V3-Base, a 671B mixture-of-experts), and the slide

treats them as such.

Dr. GRPO (Liu et al. 2025, "Understanding R1-Zero-like training: a critical perspective") is the first of the two corrections. Its authors looked at the base models before any RL and found that they already produce the self-reflection vocabulary ("wait", "let me check", "alternatively") that the R1 report presents as emerging; the abstract names DeepSeek-V3-Base as already exhibiting the aha moment, and the body shows the same keywords in Qwen2.5 and other bases. They also found that Qwen2.5 base models answer math questions competently with no chat template at all, which suggests question–answer text in their pretraining, and that the GRPO objective's normalizations bias the update: dividing by the length of each response and by the standard deviation of the group's rewards "artificially increases response length (especially for incorrect outputs)". Their Dr. GRPO removes both normalizations and reaches 43.3% on AIME 2024 with a 7B model while keeping chains shorter. The lesson for slide 15 is that two things the R1 narrative attributes to RL, the vocabulary of reflection and the growth of length, have other sources that must be subtracted before crediting the reward.

The one-example result is the second correction. Wang et al. (2025) ran RLVR with a single training problem on Qwen2.5-Math-1.5B and moved its MATH500 accuracy from 36.0% to 73.6%, with the six-benchmark average rising from 17.6% to 35.7%; the same model trained on a 1.2k-problem subset of DeepScaleR reached 73.6% and 35.9%, so one well-chosen example matched a thousand. Two examples gave 74.8% and 36.6%. On AIME 2024 the one-shot runs went from 6.7% to 16.7% or 17.1% depending on the example, and the two-shot run to 17.9% (body, Table 3). The authors also report what they call post-saturation generalization: the training accuracy on the single example reaches 100% early, and test accuracy keeps improving afterwards. A model that gains 37 points of MATH500 from one problem is not learning mathematics from that problem; it is being pushed, by a reward that fires on one question, into a mode of answering it already had.

Spurious Rewards (Shao et al. 2025) makes the same point with controls. On Qwen2.5-Math-7B, RLVR with the true reward adds 29.1 points on MATH500; RLVR with incorrect labels adds 24.1; RLVR with a *random* reward, a coin flip unrelated to the answer, adds 21.4 (the 21.4 and 29.1 are in the abstract, the 24.1 in the body). The mechanism the authors identify is a shift in the model's reasoning style: the share of answers that use code-like step-by-step reasoning rises from about 65% to over 90%, and that style is more accurate for this model, so any training signal that increases it, including noise, helps. The same spurious rewards do little or nothing for Llama and OLMo bases, which do not have the code-reasoning prior. The slide's two-line rule follows directly: a small-scale RL result on a Qwen base with no second family and no random-reward control cannot distinguish learning from elicitation.

The dynamics line at the bottom collects what every reproduction reports and slide 16 analyses: accuracy and length rise together; the policy's entropy falls, often fast; and the distribution over chains narrows onto the ones that work. Whether the narrowing finds anything the base model could not already produce is the subject of the next slide.

Terms

R1-Zero DeepSeek's model trained by reinforcement learning directly from the base model with no supervised fine-tuning; the experiment that showed long, self-checking chains can emerge from a correctness reward alone. Slide 12.

aha moment The R1 report's name for a passage in which the model interrupts its own chain to re-evaluate an earlier step; presented there as an emergent behavior, shown by Dr. GRPO to be present in base models.

base model A pretrained language model before any post-training; the starting point of R1-Zero-style RL and the object whose priors the cautionary results expose.

Dr. GRPO "GRPO done right" (Liu et al. 2025): GRPO without the per-response length normalization and the group standard-deviation normalization, which the authors show bias the update toward long incorrect responses. Slide 13.

length normalization Dividing a response's loss by its token count; in GRPO it makes a long wrong response cost less per token than a short one, favoring length.

chat template The fixed text that wraps a user's message when a model is prompted (role markers, system text); Qwen2.5 base models answer math with none, a sign of question–answer data in pretraining.

one-example RLVR Reinforcement learning with verifiable rewards on a training set of a single problem (Wang et al. 2025); the result that one example moves MATH500 from 36.0% to 73.6% on a 1.5B math model.

post-saturation generalization Test accuracy continuing to rise after training accuracy on the single example has reached 100%; reported by Wang et al. 2025.

spurious reward A training signal with no relation to correctness (random, or from wrong labels) that nevertheless improves accuracy by shifting the model toward a reasoning style it already had; Shao et al. 2025.

random-reward control The experiment of running the same RL pipeline with a coin-flip reward; if it reproduces most of the gain, the gain was elicitation, not learning.

code reasoning Answering a math problem by writing program-like steps in text (or actual code) rather than prose; the style whose frequency rises from 65% to over 90% under any reward on Qwen2.5-Math.

policy entropy The average uncertainty of the model's next-token distribution over its own samples; it falls during RLVR as the model commits to fewer chains. Slide 16.

Intuition

Picture the base model as a crowd that already contains a few careful solvers. RL does not teach the crowd mathematics; it hands the microphone to the careful solvers more often, and they talk longer because talking longer is how they are careful. A reward that fires at random still ends up handing the microphone to whoever speaks in the style the reward's noise happens to favor, which on a Qwen base is the careful one. The question to ask of any RL curve is whether a random reward would have drawn the same curve.

5.2 Slide 16: Does RL create or select? Sharpening, coverage, entropy

RL makes the model find in one try what the base found in many; whether it finds anything new is the open question.

What the slide shows A plot of pass@k against k on a log-2 axis from 1 to 256, labeled illustrative and drawn after Yue et al. (2025). Two curves: the base model starts low (about 30% at $k = 1$) and climbs steadily; the model after RLVR starts high (about 58% at $k = 1$) and climbs slowly. They cross at a dashed vertical line near $k = 128$, beyond which the base model is ahead. The panel on the right gives three readings of the same curve. *Sharpening* is Yue et al.'s: RL raises the probability of chains the base could already produce and narrows coverage, while distillation adds solutions the student lacked. *Discovery with more steps* is ProRL's (Liu et al. 2025): with thousands of steps, KL control and reference resets, a 1.5B model's pass@k moves past its base on tasks where the base had none; and Khatri et al. (2025), with 400k GPU-hours, find RL curves that

are sigmoids in compute; recipes differ in their ceiling, most design details change only compute efficiency, and the early slope does not predict the asymptote. *Entropy* is Cui et al.'s (2025): policy entropy collapses early and performance follows a fixed curve of it, so keeping entropy up raises the ceiling; and Wang et al. (2025) find that the 20% of tokens with high entropy, the "forks", carry nearly all the useful gradient.

Details The quantity on the axis needs defining first. For a problem, draw n independent samples from the model and let c be the number that are correct. The probability that at least one of $k \leq n$ samples drawn without replacement from those n is correct is

$$\widehat{\text{pass@}k} \;=\; 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}},$$

the unbiased estimator of Chen et al. (2021) that every paper on this slide uses; pass@1 is then the mean accuracy over samples, and pass@k for large k is the *coverage*: the fraction of problems the model can solve at all given enough tries. A verifier or a majority vote turns coverage into accuracy (slides 8–9); without one, pass@k for $k > 1$ is a ceiling, not a score.

Yue et al. (2025, "Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?") measured both curves for a range of RLVR-trained models and their bases across math, code and visual-reasoning benchmarks. The pattern is the one the slide draws: the RL model is far ahead at $k = 1$, the gap closes as k grows, and at k in the hundreds the base model matches or exceeds it. Their reading is that RLVR, as practiced in 2025, "does not elicit fundamentally new reasoning patterns": it concentrates probability on solution paths the base model already samples with low probability, which raises pass@1 and sampling efficiency, and it does so by narrowing the distribution, which lowers coverage. Distillation from a stronger teacher behaves differently in their measurements: the distilled student's pass@k curve lies above the base's at every k, because the teacher's chains contain solutions the student could not sample at all. This is the strongest form of the "selection, not creation" view, and it is consistent with the cautionary results of slide 15.

ProRL (Liu et al. 2025, NVIDIA) is the reply. Its authors argue that the crossover is a property of short RL runs on saturated benchmarks, and train R1-Distill-Qwen-1.5B for more than 2,000 RL steps on 136K problems spanning math, code, STEM, logic puzzles and instruction following, with a KL penalty to a reference policy and periodic resets of that reference to the current policy so the penalty does not freeze progress. On tasks where the base model's pass@k is near zero for every k they tested, the trained model's pass@k is above zero and rises with training, which is coverage the base did not have; the body reports pass@1 gains of 14.7 points on math, 13.9 on code, 54.8 on logic puzzles, 25.1 on STEM reasoning and 18.1 on instruction following (the abstract carries no numbers). BroRL (Hu et al. 2025) continues the argument along a different axis: increasing the number of rollouts per prompt to hundreds revives models that had saturated after 3,000 ProRL steps. Whether these runs discover genuinely new procedures or elicit ones the base sampled too rarely to be measured in a few hundred samples is not settled by either paper; what is settled is that the crossover is not a law.

Khatri et al. (2025, "The art of scaling reinforcement learning compute for LLMs") add the scaling view. In over 400,000 GPU-hours of ablations they find that reward against RL compute follows a sigmoid: a slow start, a rise, and an asymptote. Some choices move the asymptote while most (loss aggregation, normalization, curriculum, the off-policy algorithm) mainly change compute efficiency without shifting it, and a recipe that looks better in the first thousand steps can end lower; their ScaleRL recipe, chosen by fitting the sigmoids on small runs, extrapolated to a 100,000 GPU-hour run. Tan et al. (2025) report the complementary finding on Qwen2.5 from 0.5B to 72B: RL post-training loss follows a power law in compute and data, with model size mattering more than data at fixed budget. These are the first attempts to treat RL for reasoning the way the scaling-laws lecture treated pretraining.

The entropy reading is mechanistic. Cui et al. (2025) measure the policy's entropy H over its own samples during RLVR and find that it collapses within the first few hundred steps, and that downstream performance R tracks it along a fixed curve, $R = -a\,e^{H} + b$, across models and runs: once entropy reaches zero, performance is pinned at $b - a$ and no further training helps. Their fix, Clip-Cov and KL-Cov, limits updates on the small set of tokens whose probability and advantage are strongly covariant, since those are the tokens whose updates drive entropy down; keeping entropy up raises the attainable ceiling. Wang et al. (2025, "Beyond the 80/20 rule") locate the same structure at the token level: the top 20% of tokens by entropy (their chosen cutoff) are the forks where the chain chooses a direction ("alternatively", "suppose", "therefore"), and restricting the policy-gradient update to them matches or beats updating every token on Qwen3 models. The slide's three readings are therefore less in conflict than they look: RL sharpens the distribution (Yue), the sharpening happens at the forks (Wang), it can go too far (Cui), and with enough steps, rollouts and entropy control the sharpened distribution can reach places the base's samples did not (ProRL, BroRL, Khatri).

Terms

pass at k The probability that at least one of k independent samples from the model is correct, estimated without bias from $n \geq k$ samples by $1 - \binom{n-c}{k} / \binom{n}{k}$; pass@1 is mean accuracy, large-k pass@k is coverage.

coverage The fraction of problems a model solves in at least one of many samples; the ceiling that a verifier or vote can turn into accuracy. Brown et al. 2024, slide 9.

sharpening Yue et al.'s reading of RLVR: it raises the probability of solution paths the base model already produces and narrows the distribution, trading coverage for pass@1.

crossover The value of k beyond which the base model's pass@k exceeds the RL model's; the empirical signature of sharpening.

ProRL Prolonged reinforcement learning (Liu et al. 2025): over 2,000 RL steps on 136K diverse problems with a KL penalty and periodic reference resets, reported to raise pass@k above the base on tasks where the base had none.

reference reset Replacing the reference policy in the KL penalty with the current policy at intervals, so the penalty keeps the update local without anchoring it to the starting model.

BroRL Broadened RL (Hu et al. 2025): scaling the number of rollouts per prompt to the hundreds, which revives models saturated under ProRL.

rollout One sampled chain and answer for a prompt during RL; a group of G rollouts per prompt forms the GRPO baseline. Slide 13.

ScaleRL Khatri et al.'s (2025) recipe, chosen by fitting sigmoidal compute–reward curves on small runs; the finding that the early slope does not predict the asymptote and that most recipe details change efficiency rather than the ceiling.

entropy collapse The rapid fall of policy entropy early in RLVR, after which the model samples few distinct chains and performance stops improving; Cui et al. 2025.

Clip-Cov and KL-Cov Cui et al.'s methods that clip or penalize updates on the tokens whose probability and advantage covary most, to slow entropy collapse.

fork token A token at which the chain's next-token distribution has high entropy, where the reasoning branches; the top 20% by entropy, and the ones whose updates matter (Wang et al. 2025).

compute-performance curve Reward as a function of RL compute; sigmoidal in Khatri et al., a power law in test loss in Tan et al.

Intuition

Think of the base model as a library and RL as a librarian who learns which shelves hold the answers. After training the librarian walks straight to the right shelf, which is what pass@1 measures; but the librarian has also stopped browsing, which is what the crossover measures. Longer training, more browsing per visit, and a rule against settling on one shelf too soon are the three ways the recent papers try to keep the librarian learning. The question for any new RL result is whether it found a new book or only a faster route to an old one.

6 Distillation and data (slides 17–18)

6.1 Slide 17: Distillation: a thousand good chains, and a word

Fine-tuning on a few hundred long chains plus a budget control recovers most of the behavior; the base model had it.

What the slide shows Four tiles across the top: 1,000 (examples that fine-tune s1-32B), 817 (examples that fine-tune LIMO), "Wait" (the word appended to force more thinking), and 26 min (on 16 H100s for s1-32B, from the paper's training description). Two panels below. *Budget forcing* summarizes s1 (Muennighoff et al. 2025): fine-tune Qwen2.5-32B-Instruct on 1,000 questions with traces distilled from Gemini, chosen for difficulty, diversity and quality; at inference, cap the thinking span by inserting the end-of-thinking token, or extend it by suppressing that token and appending "Wait", after which the model re-checks and often fixes its answer; s1-32B beats o1-preview on competition math by up to 27%. *Less is more* summarizes LIMO (Ye et al. 2025): 817 curated problems with high-quality chains take Qwen2.5-32B-Instruct to 57.1% on AIME 2024 and 94.8% on MATH (v1), against 6.5% and 59.2% for earlier supervised recipes with 100k examples; the authors' reading is that the pretrained model holds the knowledge and a few exemplary chains teach the format; the caveat is that the chains came from RL-trained models. The line at the bottom draws the conclusion for slide 16: if a thousand examples unlock long self-checking chains in a 32B model, the behavior is latent in pretraining, and RL's job is to find it and make it reliable where no exemplar exists.

Details s1 ("s1: Simple test-time scaling") is the minimal recipe. The authors collected 59,029 questions from 16 sources, generated reasoning traces with Gemini 2.0 Flash Thinking, and selected 1,000 by three filters: quality (drop malformed traces), difficulty (drop questions two weaker models already solve), and diversity (sample across subject areas). Supervised fine-tuning of Qwen2.5-32B-Instruct on those 1,000 traces took 26 minutes on 16 H100 GPUs with FSDP, which is the tile. The resulting s1-32B scores 56.7% on AIME 2024, 93.0% on MATH500 and 59.6% on GPQA Diamond (body, Table 1), against 44.6% on AIME 2024 for o1-preview; the "up to 27%" in the abstract is the largest of the margins over o1-preview across the competition-math sets. The abstract's version of the AIME number is "50% to 57%" with budget forcing, the table's is 56.7%; the slide uses the abstract's claim and these notes give both.

Budget forcing is the second half of the recipe and the reason the paper matters beyond its data efficiency. The model's thinking span is delimited by tokens; the server can *cap* it by appending the end-of-thinking delimiter when the budget is reached, which forces the answer to begin, and it can *extend* it by suppressing that delimiter when the model tries to emit it and appending the string "Wait" instead, which prompts the model to continue, re-read and often correct itself. Extending in this way raises AIME 2024 accuracy from 50% to 57% in the abstract's framing, and the accuracy-against-tokens curve it produces is the serial test-time scaling curve of slide 3. The method is now standard: the budget and effort controls on slide 19 are

implementations of the cap, and several length-control methods on slide 20 are trained alternatives to the extension. The authors are explicit that budget forcing produces diminishing returns and eventually loops: "Wait" can be appended only so many times before the chain repeats itself, which is why slide 20 and Ghosal et al. (2025) treat it as a walk along one model's curve rather than a way to move the curve.

LIMO ("Less is more for reasoning") makes the same point from the data side with even fewer examples. The v1 preprint and the Hugging Face model card report 817 training samples, chosen for difficulty and for the quality of their chains, taking Qwen2.5-32B-Instruct to 57.1% on AIME 2024 and 94.8% on MATH500, where earlier supervised recipes trained on more than 100k examples (NuminaMath-style data) reached 6.5% and 59.2%. The 6.5 and 59.2 are those earlier models' scores, not the base model's: the comparison is between two fine-tuning recipes, one with about 1% of the other's data. The current arXiv version (v3, July 2025) describes LIMO-v2, 800 samples on the same base, at 63.3% and 95.6%, and claims a 45.8-point absolute gain out of distribution; the slide quotes v1 because its numbers are the ones most later papers cite, and the notes name the version so the reader can find either. The authors' hypothesis, which they state as the "less-is-more reasoning hypothesis", is that when the pretrained model already encodes the domain knowledge, a small number of exemplary demonstrations of how to use it suffices, and that what the demonstrations teach is the format of extended reasoning: decomposition, verification, backtracking, written out.

The caveat the slide attaches is the important one for the lecture's argument. The chains s1 and LIMO fine-tune on were generated by Gemini's thinking mode and by R1-family models, both of which were themselves trained by reinforcement learning to produce such chains. Distillation on a thousand examples therefore shows that the *student* base model can adopt the behavior from few examples, not that the behavior can be obtained without RL somewhere upstream. The R1 report's own comparison (slide 12) is the complement: distilling R1's 800k traces into Qwen2.5-32B gives 72.6% on AIME 2024, where running the RL recipe directly on the same 32B base gives 47.0%, so for small and mid-sized models the cheap route is to copy a large RL-trained model, and the expensive route is to run RL on a large base and then copy it. Slide 16's reading follows: the chains are latent in the base model, the teacher's examples show where they are, and RL on a strong base is how the first teacher was made.

Terms

distillation Training a student model on outputs of a stronger teacher; for reasoning models, supervised fine-tuning on the teacher's chains and answers. The cheap route to a small reasoning model.

s1 Muennighoff et al. 2025: Qwen2.5-32B-Instruct fine-tuned on 1,000 Gemini-distilled traces, plus budget forcing; the minimal reasoning-model recipe.

s1K The 1,000-example dataset of s1, selected from a pool of 59,029 questions by quality, difficulty and diversity filters.

budget forcing Controlling the length of the thinking span at inference by inserting the end-of-thinking token to stop it, or suppressing that token and appending "Wait" to extend it. Slides 19–20.

end-of-thinking token The delimiter that closes the thinking span; the server can insert it to enforce a budget. Slide 22.

LIMO Ye et al. 2025: 817 curated problems (v1; 800 in v2) that fine-tune Qwen2.5-32B-Instruct to 57.1% on AIME 2024 and 94.8% on MATH500 (v1), with about 1% of earlier recipes' data.

less-is-more hypothesis LIMO's claim that a model whose pretraining already holds the domain knowledge needs only a few exemplary chains to learn the format of extended reasoning.

supervised fine-tuning Training on input–output pairs by next-token prediction; abbreviated SFT. The stage that distillation uses and that R1's stages 1 and 3 use.

AIME The American Invitational Mathematics Examination, two papers of 15 integer-answer problems each year; AIME 2024 and 2025 are the standard competition-math benchmarks of 2025, and they saturated within the year (slide 25).

MATH500 A 500-problem subset of the MATH benchmark (Hendrycks et al. 2021) selected by Lightman et al. 2023 for evaluation; near ceiling for 2025 reasoning models.

GPQA Diamond The hardest subset of GPQA (Rein et al. 2023), graduate-level science questions written to be Google-proof; the science benchmark on slides 17 and 23.

o1-preview OpenAI's September 2024 preview of o1; the comparison point s1 and Sky-T1 chose, scoring 44.6% on AIME 2024 in s1's evaluation.

teacher model The model whose chains are distilled; for s1 Gemini 2.0 Flash Thinking, for LIMO and most 2025 datasets R1 or QwQ, all themselves RL-trained.

Intuition

A thousand worked examples do not teach a graduate student the field; they show a student who already knows the field how the examiners want the work written up. That is the whole of s1 and LIMO, and the "Wait" trick is the examiner saying "are you sure?" at the end of the page. The question to keep asking is where the thousand examples came from: in 2025 the answer was always a model that had been trained with RL.

6.2 Slide 18: Where the problems and rewards come from

Verifiable problems are the scarce input; the pipeline is collection, cleaning, verification and difficulty filtering.

What the slide shows Five boxes in a row, a pipeline: *collect* (math with answers, code with tests, puzzles, STEM with references), *clean* (deduplicate; decontaminate against every test set by n-gram overlap), *verify* (keep what a program can check: a parsed answer, a test suite), *filter* (run the policy; keep problems with pass rate strictly between 0 and 1), *curriculum* (harder problems as the policy improves; drop the mastered ones). Below it a table of open sources with sizes marked as secondary: NuminaMath (2024, 860k), OpenR1-Math-220k (2025, 220k), OpenMathReasoning (2025, 540k problems and 3.2M solutions), DAPO-Math-17k (2025, 17k), OpenThoughts3 (2025, 1.2M), and SWE-Gym and SWE-smith (2025, 2.4k and 50k). The line at the bottom says the limiting resource is the checker, not the text: math has a parser, code has tests, everything else needs a rubric or a judge; and at the extreme, one problem or self-proposed problems checked by a code executor still train.

Details The pipeline boxes describe what every 2025 reasoning-data release did, in the order it did it. *Collect* starts from sources whose answers can be checked: competition and olympiad mathematics with integer or closed-form answers, programming problems with hidden test suites, logic puzzles with generators, and science questions with reference answers. *Clean* removes duplicates and near-duplicates and, essentially, removes anything that overlaps with evaluation sets, usually by n-gram matching against AIME, MATH500, GPQA, LiveCodeBench and the rest; slide 25 shows why this step decides whether a number means anything. *Verify* keeps only what a program can grade: for mathematics an answer parser such as Math-Verify that compares the boxed answer to the reference up to equivalence, for code the test suite. *Filter* is the step specific to RL and the one that matters most for GRPO: run the current policy on each problem several times and keep the problems whose pass rate is strictly between 0 and 1, because a group of rollouts that are all right or all wrong has zero advantage and contributes no gradient (DAPO's dynamic sampling makes this a

per-batch rule, slide 13). *Curriculum* re-applies the filter as the policy improves, replacing mastered problems with harder ones; Light-R1's 76k-then-3k curriculum and DeepScaleR's growing context are two versions.

The table's rows are the sources most 2025 and 2026 reproductions draw on, with sizes from their model cards and release posts. NuminaMath (Project Numina, 2024) is 860k competition-style problems with chain-of-thought solutions, plus a 72.5k tool-integrated subset and a 2025 revision (NuminaMath-1.5) of 896k problems; it is the problem bank behind the AIMO prize entries and most later sets. OpenR1-Math-220k is Hugging Face's R1 reproduction data: 800k R1 traces generated for 400k NuminaMath-1.5 problems on 512 H100s, filtered to a 94k default and a 131k extended split, with the resulting OpenR1-Qwen-7B at 90.6% on MATH-500 and 36.7% on AIME 2024†. OpenMathReasoning (Moshkov et al. 2025, NVIDIA's AIMO-2 winning entry) is 540K problems with 3.2M long-reasoning solutions and 1.7M tool-integrated ones, plus GenSelect, a trained selector among candidates. DAPO-Math-17k is the small clean set used to train DAPO: integer-answer problems, released with the code. OpenThoughts3 (Guha et al. 2025) is 1.2M examples across math, code and science, chosen after more than a thousand ablations of question sourcing, filtering and teacher choice; a 7B model fine-tuned on it alone scores 53% on AIME 2025, 51% on LiveCodeBench and 54% on GPQA Diamond, and the authors found QwQ-32B a better teacher than R1 for their students. SWE-Gym (2.4k) and SWE-smith (50k) are executable repository tasks with tests, the verifiable environments for agentic coding RL (slide 21).

Several cheap reproductions calibrate what the pipeline costs. Sky-T1 (NovaSky, January 2025) fine-tuned Qwen2.5-32B-Instruct on 17K QwQ-distilled samples in 19 hours on 8 H100s, under $450 by the authors' count, for 43.3% on AIME 2024 and 82.4% on MATH500†. Bespoke-Stratos (January 2025) used 17k R1 traces, which its authors describe as 47 times fewer than DeepSeek's own distillation set, for 63.3% on AIME 2024 and 93.0% on MATH500 with a 32B student†. Light-R1 (Wen et al. 2025) ran a two-stage curriculum, 76k problems then 3k hard ones, on Qwen2.5-32B-Instruct in about six hours on 12 H800s, which the authors price at about $1,000, for 76.6% on AIME 2024 and 64.6% on AIME 2025 (body; the abstract reports the 14B variant at 74.0 and 60.2). DeepScaleR (Agentica, February 2025) took R1-Distill-Qwen-1.5B through GRPO with the context limit raised in stages from 8K to 16K to 24K tokens and reached 43.1% on AIME 2024†. AceReason-Nemotron (Chen et al. 2025) found that running RL on math only, then on code only, gives larger gains than mixing them: +14.6 and +17.2 points on AIME 2025 for 7B and 14B distilled models, and +6.8 and +5.8 on LiveCodeBench, with the math stage already improving code. The pattern across these is the one the subtitle states: the recipes are public and cheap; what is scarce is the supply of problems that a program can grade.

The bottom line of the slide points at that scarcity. Mathematics with closed-form answers and code with tests exhaust the domains where the reward is a program. Everything else, proofs, essays, research, design, needs a rubric or a judge (slide 14), and those rewards can be gamed in ways a parser cannot. At the two extremes of the data axis sit the one-example result of Wang et al. (2025) on slide 15, where a single verifiable problem suffices to elicit the behavior, and Absolute Zero (Zhao et al. 2025), where the model proposes its own problems and a code executor checks them, so the training set is empty and the verifier is the only fixed input.

Terms

verifiable problem A problem whose answer a program can check: a parsed closed-form answer, a passing test suite, a puzzle generator's solution. The input RLVR needs.

decontamination Removing training examples that overlap with evaluation sets, usually by n-gram matching; the step that decides whether a benchmark number is a measurement.

n-gram overlap A match of n consecutive tokens between a training example and a test item; the usual decontamination test.

Math-Verify An open answer parser and equivalence checker for mathematical answers, used as the reward function in most open RLVR stacks.

pass-rate filtering Keeping only problems the current policy solves with probability strictly between 0 and 1, since all-correct and all-wrong groups give zero advantage in GRPO. Slide 13.

curriculum Ordering or re-selecting training problems by difficulty as the policy improves; Light-R1's two stages and DeepScaleR's growing context are examples.

NuminaMath Project Numina's 860k-problem competition-math dataset with chain-of-thought solutions (2024), with tool-integrated and 1.5 revisions; the problem bank behind most 2025 reasoning data.

OpenR1-Math-220k Hugging Face's dataset of verified R1 traces on NuminaMath problems (220k after filtering), from its Open-R1 reproduction.

OpenMathReasoning NVIDIA's AIMO-2 dataset (Moshkov et al. 2025): 540K problems, 3.2M long-reasoning and 1.7M tool-integrated solutions.

DAPO-Math-17k The 17k integer-answer problem set released with DAPO (Yu et al. 2025), used for its RL runs.

OpenThoughts3 The 1.2M-example distillation dataset of Guha et al. 2025, chosen by large-scale ablation; its 7B student scores 53% on AIME 2025 by SFT alone.

SWE-Gym and SWE-smith Collections of executable software-repository tasks with tests (2.4k and 50k), the environments for agentic coding RL.

Sky-T1 NovaSky's January 2025 reproduction: 17K distilled samples, 19 hours on 8 H100s, under $450, 43.3% on AIME 2024 with a 32B student.

Light-R1 Wen et al. 2025: curriculum SFT (76k then 3k) on Qwen2.5-32B-Instruct in about six hours on 12 H800s, 76.6% on AIME 2024.

DeepScaleR Agentica's 1.5B model trained by GRPO with context grown from 8K to 24K tokens, 43.1% on AIME 2024.

Absolute Zero Zhao et al. 2025: the model proposes its own tasks and a code executor verifies them; RL with no external training data.

Intuition

The problems are the fuel and the checker is the engine: you can pour in a million problems, but only the ones the engine can burn, those with a program that says right or wrong, move the model. The filter step is the carburetor, metering problems that are neither too easy nor too hard so each batch gives a gradient. When someone says they have a new dataset for reasoning, ask what checks the answers.

7 Inference: budgets, length control, parallel compute, tools, serving (slides 19–22)

7.1 Slide 19: Budgets and effort: deciding how much to think

The caller sets the budget, the server closes the thinking span at the limit, accuracy follows and then saturates.

What the slide shows A plot on the left, labeled illustrative: accuracy against the thinking-token budget on a log scale from 300 to 40,000 tokens, for three tasks solved by one model. Competition math starts near 18% at 500 tokens, rises steeply through 2,000–8,000 and flattens near 62% by 32,000; a coding task starts higher, near 55%, and saturates around 80% by 8,000; a classification task starts at 86% and gains two points by 1,000 tokens and nothing after. The title says what the shape means: the knee of the curve moves right with difficulty. The panel on the right, *The knobs, as shipped*, lists the controls vendors expose as of October 2026, all marked secondary: OpenAI's `reasoning_effort` (from `none` to `max`, model-dependent; the open gpt-oss weights honor low, medium and high); Anthropic's adaptive thinking with an `effort` setting, with `budget_tokens` for extended thinking on Claude 4.5 and earlier; Google's `thinking_level`, with `thinkingBudget` on Gemini 2.5; DeepSeek's thinking switch plus its own `reasoning_effort`; Qwen3's `/think` and `/no_think` switches with a token cap. It then explains how a cap is enforced, what hybrid models are, and ends with the rule of thumb: start low and raise effort where accuracy moves.

Details The plot is a sketch of a family of curves that every reasoning-model release shows in some form, from the o1 announcement's test-time compute curve to the gpt-oss card's accuracy-by-effort tables[†]; its three lines are chosen to illustrate three regimes rather than to reproduce any one paper. The mechanism behind all three is the one slides 3 and 6 established: each thinking token is one more forward pass of serial computation, and a problem that needs many serial steps cannot be solved below some minimum number of them. For a problem the model can solve in a short chain, extra budget is spent on re-checking and buys little; for a problem at the edge of the model's ability, the budget is the binding constraint until the chain is long enough, after which the curve flattens because the remaining failures are not failures of length. Averaged over a benchmark of mixed difficulty, the result is the sigmoid-in-log-tokens shape, and its knee sits further right for harder benchmarks. The same shape, on a trained model, is the "log-linear then saturate" law that slides 28 and 30 summarize.

The knobs are two generations of product engineering on one primitive. The primitive is the cap: the server counts tokens inside the thinking span, and when the count reaches the budget it inserts the end-of-thinking token itself, which closes the span and forces the answer to begin. This is s1's budget forcing (slide 17) running inside the inference engine; the serving slide shows where it sits. A token budget (`budget_tokens`, `thinkingBudget`, Qwen3's cap) exposes the primitive directly. An effort level (`reasoning_effort`) hides the number: the model is trained, or the server is configured, so that `low` produces short chains and `high` long ones, and the caller chooses a point on the curve without knowing where the knee is. The gpt-oss card[†] shows that effort levels can be a system-prompt instruction honored by open weights, with accuracy rising from low to medium to high on every benchmark it reports. By October 2026 (slide 23) the effort level has won: every vendor exposes one, token budgets are documented as legacy, and the model is increasingly allowed to choose its own effort per request. To think *more* than the model would on its own, s1's second trick, suppressing the end-of-thinking token and appending "Wait", remains the reference method, and the trained alternatives are on slide 20.

Hybrid models are the other development the panel names. Qwen3 (2025) was trained so that one set of weights answers either with a thinking span or without one, selected by a template switch; Claude's extended thinking, DeepSeek V3.1's two templates and GPT-5's router do the same from the caller's point of view[†]. The engineering reason is cost: a model that must think on every request is expensive on the classification-like majority of traffic, and a separate non-thinking model doubles the fleet. The training reason is on slide 20: once length is a controlled variable, "no thinking" is the zero point of the same control.

The rule of thumb at the end is the operational summary of the plot. Because the curves saturate, raising effort on a task that is already on its plateau spends tokens for nothing, and because every thinking token is billed as output (slide 22), the bill scales with the budget whether or not accuracy moves. The practical procedure is to run a sample of each task type at low effort, raise it one level at a time, and keep the lowest level at which measured accuracy stops improving, logging tokens per answer alongside accuracy so the two

can be compared when the model or the task changes.

Terms

thinking budget The maximum number of tokens the model may spend in its thinking span for one request; set by the caller as a token count or as an effort level.

effort level A discrete setting (minimal, low, medium, high, and in 2026 max and beyond) that selects a point on the accuracy-against-tokens curve without exposing the token count; OpenAI's `reasoning_effort` was the first.

reasoning effort OpenAI's API parameter for the effort level; honored as a system-prompt instruction by the open gpt-oss models.

budget tokens Anthropic's extended-thinking parameter giving the thinking budget as a token count on Claude 4.5 and earlier; deprecated from Claude 4.6 in favor of adaptive thinking with an effort setting (slide 23).

thinking budget (Google) The Gemini API's token-count control for thinking on Gemini 2.5, `thinkingBudget`; superseded by `thinking_level` on later models.

think and no-think switches Qwen3's `/think` and `/no_think` prompt tags that turn the thinking span on or off in a hybrid model.

cap enforcement The server inserting the end-of-thinking token when the budget is reached, so the answer begins; budget forcing inside the inference engine.

hybrid model One set of weights trained to answer with or without a thinking span, chosen per request (Qwen3, Claude, DeepSeek V3.1, GPT-5).

knee The budget at which a task's accuracy curve turns from rising to flat; it moves right with difficulty.

auto effort The 2026 default in which the model or router picks its own effort per request rather than the caller (slide 23).

Intuition

A thinking budget is a parking meter for computation: you buy minutes, and the problem decides how many it needs. An easy problem leaves most of the time on the meter; a hard one runs out and gets towed, which is the cap inserting the end-of-thinking token. Effort levels sell the meter in three sizes so you do not have to guess the minutes. Before raising the level, check whether the accuracy actually moved.

7.2 Slide 20: Overthinking, underthinking, and learning the right length

Trained chains waste tokens on easy problems and give up too early on hard ones; length is now a trained quantity.

What the slide shows Two panels. *Two failure modes*: overthinking (Chen et al. 2024, "Do not think that much for 2+3"), where o1-style models produce many redundant solutions to trivial problems and a self-trained preference for efficient chains cut tokens by about half on MATH500 with no loss of accuracy; underthinking (Wang et al. 2025, "Thoughts are all over the place"), where on hard problems the models abandon promising lines early and switch, wrong answers switch more, and a decoding penalty on switching raises accuracy without retraining; and agents overthink too (Cuadron et al. 2025), where deliberating instead of acting lowered SWE-bench success. *Making length a controlled variable*: L1 (Aggarwal and Welleck 2025), RL with a target length in the prompt that the model obeys and that beats budget forcing at equal length;

NoThinking (Ma et al. 2025), prefilling a dummy thinking block ("Okay, I think I have finished thinking."), which beats thinking at small budgets and with parallel samples matches it at a fraction of the latency; and adaptive thinking (AdaptThink, Thinkless 2025), RL that chooses whether to think at all, halving chain length at equal or better accuracy on easy sets, with DAPO's overlong shaping and Kimi k1.5's length penalty doing the same during training. The line at the bottom gives the picture to keep: an accuracy-against-tokens frontier per task; budget forcing walks along one model's curve, length-trained models move the curve left, adaptive models pick the point per question.

Details Overthinking was documented within weeks of the first open o1-style model. Chen et al. (2024) asked QwQ-32B-Preview what $2+3$ is and received 13 distinct solutions over 901 tokens; across their easy sets, the first solution in a chain was already correct more than 92% of the time, so most of the chain's length was spent confirming an answer the model had. Their remedy is a preference-tuning step: for each problem, sample chains, label the shortest correct one as preferred and a longer one as rejected, and fine-tune on the pairs. On MATH500 this cut mean tokens from 2,407.9 to 1,330.7 (a 48.6% reduction) while accuracy moved from 93.0% to 92.8%, which is the "about half with no loss" of the panel. The method does not touch the hard problems, where the long chains are doing work; it removes the ritual re-checking on the easy ones.

Underthinking is the opposite failure and shows up on the hard problems. Wang et al. (2025) looked at the chains of o1-like models on competition problems and measured how often the model switches between lines of attack ("alternatively", "let me try another approach"). Incorrect answers switch far more often than correct ones, and the switching is premature: the abandoned line was often one that would have worked. Their remedy, thought-switching penalty (TIP) decoding, lowers the logits of the switch phrases during generation, which keeps the model on a line longer and raises accuracy with no retraining. Cuadron et al. (2025) found the agentic version: in 4,018 SWE-bench Verified trajectories, agents that deliberate instead of interacting with the environment fail more often, and simply selecting the trajectories with the least overthinking gains about 30% in success at 43% lower cost. Three papers, one lesson: the length of a chain is not the amount of thinking in it.

The right panel is the trained response. L1 (Aggarwal and Welleck 2025) adds a target length to the prompt ("think for about 1,000 tokens") and trains with a reward that combines correctness with closeness to the target:

$$r(y, n_{\text{gold}}) \;=\; \mathbb{1}[y \text{ correct}] \;-\; \alpha \left| n_{\text{gold}} - n_y \right|,$$

where n_y is the length of the generated chain, n_{gold} the requested length and α a small penalty weight (the exact form, LCPO, is in the paper). The trained model obeys the target over a wide range and, at any fixed length, is more accurate than s1's budget forcing on the same base: the paper reports up to 100% relative and 20% absolute improvement, and a log-linear accuracy-versus-length slope of 0.24 against 0.37 for budget forcing, which is the "moves the curve left" of the bottom line. A 1.5B L1 model matches or beats GPT-4o at equal chain length. ThinkPrune (Hou et al. 2025) reaches a similar place by shrinking the token limit during RL: R1-Distill-Qwen-1.5B halves its AIME chain length for a 2-point drop.

NoThinking (Ma et al. 2025) is the zero of the length axis. Prefill the thinking span with a token or two that closes it immediately, so the model answers as a chat model would, but from reasoning-trained weights. At a 700-token budget on an ACM 2023 set, NoThinking scores 51.3% against 28.9% for the thinking mode, because the thinking mode spends the budget and is cut off before answering; and with parallel sampling plus a vote or verifier, NoThinking matches the thinking mode's accuracy at up to 9 times lower latency. The adaptive methods make the choice per question: AdaptThink (Zhang et al. 2025) trains a policy that picks thinking or no-thinking and reports 53% shorter outputs with 2.4% higher accuracy; Thinkless (Fang et al. 2025) learns a control token with a decoupled GRPO objective and cuts long-chain use by 50–90% on easy sets. The training-time analogues already in the recipes are DAPO's overlong reward shaping, which

penalizes chains that run past a soft limit (slide 13), and Kimi k1.5's length penalty and long2short transfer, which distil long-chain accuracy into short chains (its long2short model reports 60.8 on AIME and 94.6 on MATH500).

The bottom line is the frame for all of it. For each task there is a frontier of accuracy against tokens. Budget forcing and effort levels move a fixed model along its own frontier; L1, ThinkPrune and the overlong penalties train a model whose frontier lies to the left, more accuracy per token; NoThinking is the frontier's left endpoint; AdaptThink and Thinkless choose, per question, where on the frontier to stand. Ghosal et al. (2025) add the caution that walking right is not monotone: forcing more "Wait" tokens raises accuracy and then lowers it, so a budget is not only a cost limit.

Terms

overthinking Spending many tokens re-solving or re-checking a problem the model has already answered; Chen et al. 2024 measured 13 solutions and 901 tokens for 2+3.

underthinking Abandoning a promising line of reasoning too early and switching to another; Wang et al. 2025 found wrong answers switch more.

thought-switching penalty TIP decoding: lowering the logits of switch phrases during generation to keep the model on one line; raises accuracy without retraining.

reasoning–action dilemma Cuadron et al.'s term for an agent deliberating instead of interacting with its environment; the agentic form of overthinking.

L1 Aggarwal and Welleck 2025: a model trained by length-controlled policy optimization to obey a target chain length given in the prompt.

LCPO Length-controlled policy optimization, L1's RL objective: correctness reward minus a penalty on the distance from the requested length.

ThinkPrune Hou et al. 2025: shrinking the token limit during RL so the model learns to finish within it; half the length for a 2-point drop on AIME for a 1.5B model.

NoThinking Ma et al. 2025: prefilling an immediately closed thinking span so a reasoning model answers without a chain; competitive at small budgets and under parallel sampling.

adaptive thinking A trained choice of whether to think at all for a given question; AdaptThink and Thinkless (2025).

Thinkless Fang et al. 2025: a control token selecting short or long reasoning, trained with a decoupled GRPO objective.

overlong reward shaping DAPO's soft penalty on chains that exceed a length threshold, rising to the full penalty at the hard limit. Slide 13.

long2short Kimi k1.5's methods for transferring long-chain accuracy to short-chain models: model merging, shortest rejection sampling, DPO and a length penalty.

accuracy–length frontier For one task, the curve of best achievable accuracy at each chain length; the object that budgets walk along and length training moves.

Intuition

A reasoning model that overthinks is a student who writes three proofs of a one-line fact; one that underthinks is a student who starts five proofs of a hard theorem and finishes none. Length training is the

tutor saying "write about this much" until the student's sense of how much a problem deserves matches the problem. The picture to keep is the frontier: the question is never "longer or shorter?" but "where on this task's curve, and can training move the curve?"

7.3 Slide 21: Parallel compute, and tools inside the chain

Beyond one longer chain: many chains combined, and chains that call a program or a search engine mid-thought.

What the slide shows Across the top, three small diagrams labeled "three shapes of inference compute". *One long chain*: a row of token blocks ending in an answer block; serial, latency grows with the budget. *Many chains, combine*: a stack of four short rows feeding one answer; parallel, vote or verify, cost times k and latency times one. *Chain with tool calls*: a row in which some blocks are green, marking a program run or a search, whose results re-enter the chain. Below, a panel *Parallel, in products*: o1 reached 93% on AIME with 1,000 samples re-ranked; Gemini's Deep Think and Grok's Heavy mode sell parallel hypotheses with a synthesis step, and OpenAI's pro tiers sell more compute per answer without stating the mechanism; Snell et al. found the right mix depends on difficulty, and the catch is still picking the winner without a verifier. On the right, *Tools inside the chain*: tool-integrated reasoning (ToRA, Gou et al. 2023); RL with a code interpreter in the loop (ReTool, Feng et al. 2025), where a 32B model reaches 67% on AIME 2024 in 400 steps while text-only RL took 1,080 steps for 40%; o3, GPT-5 and Claude 4 thinking between tool calls and Kimi K2 Thinking running hundreds of sequential calls in one task; and agentic RL, where the verifier is the environment, SWE-RL rewards patch similarity on real repositories, and chain, actions and observations form one trajectory.

Details The three shapes are the three ways to spend more inference compute, and they have different cost and latency signatures. A single longer chain costs tokens linearly and takes time linearly, because decoding is serial (slide 22); it is the only shape that can do more serial computation. Parallel chains cost k times the tokens but take no longer than one, because they decode in the same batch; they can only do more of the same computation and need a way to combine the results. Majority voting works when answers are discrete and the model is right more often than any single wrong answer appears (Wang et al. 2022, slide 5); a verifier or reward model works when one is available (slide 8); and when neither is, re-ranking by the model's own confidence (DeepConf, Fu et al. 2025, reaches 99.9% on AIME 2025 at 512 samples while discarding 84.7% of the tokens) is the 2025 answer. Snell et al. (2024, slide 9) showed the right mix depends on difficulty: for easy problems parallel samples with a vote are compute-optimal, for hard ones a longer or revised single chain is, and allocating by estimated difficulty beats either fixed policy.

In products, parallel compute is sold as a tier. OpenAI's o1 announcement reported 93% on AIME 2024 by sampling 1,000 chains and re-ranking them with a learned scorer†, against 83% with a 64-sample vote and 74% single-sample, Gemini's Deep Think and Grok's Heavy mode are described by their vendors as running several reasoning paths in parallel and synthesizing an answer†, while OpenAI's pro tiers (o1-pro, o3-pro, GPT-5 Pro) are sold as "more compute to think harder" with the mechanism unstated†. The 2026 effort levels above high (slide 23's *ultra*, four agents in parallel) continue the pattern. The research versions bring the parallelism inside the model: ParaThinker (Wen et al. 2025) trains a model to carry several paths in one context and merge them, and Parallel-R1 (Zheng et al. 2025) rewards spawning and merging sub-chains during RL, reporting an 8.4% gain over sequential RL. The catch the panel names is the one from slide 8: without a verifier, the chooser is a vote or a confidence estimate, and both fail exactly on the hard problems where the parallel budget was spent.

Tools inside the chain change what a chain can do rather than how many there are. Tool-integrated reasoning (ToRA, Gou et al. 2023) interleaves natural-language steps with program calls, so arithmetic, algebra and enumeration go to an interpreter and the model's tokens are spent on the parts that need judgment;

ToRA-Code-34B was the first open model above 50% on MATH, and CRITIC (Gou et al. 2023) used tool outputs to ground self-correction. ReTool (Feng et al. 2025) puts the interpreter inside the RL rollout: the model writes code mid-chain, the sandbox returns the result into the chain, and the correctness reward trains the whole trajectory, including when and what to execute. On a 32B base this reached 67% on AIME in 400 training steps where the text-only RL baseline needed 1,080 steps to reach 40%, and 72.5% with extended training. The frontier products do the same with their own tools: o3, GPT-5 and Claude 4 think between tool calls, and Kimi K2 Thinking is described as running 200 to 300 sequential tool calls within one task[†].

Agentic RL is the limit of this line: the verifier is the environment. SWE-RL (Wei et al. 2025) trains on real pull-request history with a reward for the similarity of the model's patch to the merged one, and its 70B Llama reaches 41.0% on SWE-bench Verified; Qwen3 and GLM-4.5 report RL on multi-turn tool use. In every such recipe, the chain, the actions (tool calls) and the observations (tool results) form one trajectory, the reward arrives at the end, and the policy gradient flows through the model's tokens only; the observations are inputs. Two consequences follow for the serving slide: every tool result re-prefills the transcript, and the transcript grows with every call, so prefix caching across tool calls and the handling of the thinking span between them become the operator's problem.

Terms

parallel sampling Generating k independent chains for one prompt in one batch and combining their answers; cost times k, latency times one.

majority voting Choosing the answer that most parallel chains agree on; self-consistency (Wang et al. 2022). Needs discrete answers.

re-ranking Choosing among parallel chains with a learned scorer or verifier; o1's 93% on AIME used 1,000 re-ranked samples.

DeepConf Fu et al. 2025: filtering and weighting parallel chains by the model's own token-level confidence; 99.9% on AIME 2025 at 512 samples with 84.7% fewer tokens.

pro tier A product mode (o1-pro, o3-pro, GPT-5 Pro, Deep Think, Heavy) that spends more compute per answer at a higher price; Deep Think and Heavy state that they run several reasoning paths and synthesize, OpenAI does not state the mechanism.

ParaThinker Wen et al. 2025: a model trained to run several reasoning paths in one context and merge them.

tool-integrated reasoning Interleaving natural-language chain steps with program or search calls whose results re-enter the chain; ToRA (2023) is the reference.

ReTool Feng et al. 2025: RL with a code interpreter inside the rollout, so the policy learns when and what to execute; 67% on AIME for a 32B model in 400 steps.

sandbox The isolated execution environment that runs the model's code during a chain and returns the output.

agentic RL Reinforcement learning in which the trajectory includes tool calls and their results, and the reward comes from the environment (tests, a patch comparison) rather than an answer parser.

SWE-RL Wei et al. 2025: RL on open pull-request history with a patch-similarity reward; 41.0% on SWE-bench Verified with a 70B model.

trajectory The full sequence of chain tokens, actions and observations in an agentic episode; the unit the reward scores.

Kimi K2 Thinking Moonshot's reasoning variant of Kimi K2, described as running hundreds of sequential tool calls within one task.

Intuition

One long chain is a single mathematician working late; many chains are a room of mathematicians who each write an answer and then vote; tools are the mathematician with a computer. The room is faster but needs someone to judge the answers; the computer makes the mathematician better at the parts that were never thinking. The question for any inference budget is which of the three the problem needs: more serial steps, more independent tries, or a calculator.

7.4 Slide 22: Serving a reasoning model: the chain is output

Thinking tokens are decoded one at a time, sit in the KV cache, and are billed; the runtime splits and caps them.

What the slide shows A table of derived figures for one answer at three chain lengths, 2k, 10k and 30k tokens: decode time at 60 tokens per second (33 s, 2.8 min, 8.3 min); KV cache for a 70B dense model at 320 KB per token (0.6, 3.2, 9.6 GB); KV cache for a V3-class MLA model at 69 KB per token (0.14, 0.7, 2.1 GB); cost at $10 per million output tokens ($0.02, $0.10, $0.30); and cost at $60 per million, o1's 2024 price ($0.12, $0.60, $1.80). A note gives the sources: per-token figures from the runtime deck, list prices at the time, marked secondary, and the fact that reasoning tokens are billed as output everywhere. Two panels: *What the runtime does* (parse the thinking span into a separate field, cap it at the budget, stream it as its own delta) and *What it changes for the operator* (decode-bound work with a latency floor, memory held for minutes so capacity is planned in token-minutes, chains across turns and tool calls handled differently by DeepSeek and Anthropic, and hidden chains that still cost).

Details Every number in the table is arithmetic on three per-token figures from the runtime lecture, and the arithmetic is the point. Decode is serial: at 60 tokens per second, a 2,000-token chain takes $2{,}000/60 \approx 33$ s, a 10,000-token chain $10{,}000/60 \approx 167$ s, which is 2.8 minutes, and a 30,000-token chain 500 s, 8.3 minutes. No batching shortens these: batching raises throughput across requests, not the speed of one sequence, so a hard problem that needs 30k tokens of thinking has a latency floor of minutes on this hardware. Memory is linear in the same count. A 70B dense model with grouped-query attention in bf16 holds about 320 KB of key–value cache per token, so a 10k-token chain occupies $10{,}000 \times 320\,\text{KB} = 3.2$ GB for as long as it is being generated, and 30k tokens 9.6 GB; a DeepSeek-V3-class model with multi-head latent attention compresses this to about 69 KB per token, 0.7 GB and 2.1 GB respectively. Cost is linear too: at $10 per million output tokens, which is the order of 2025–26 list prices for frontier reasoning models[†], 10k tokens is $0.10 and 30k is $0.30; at o1's launch price of $60 per million[†] the same chains were $0.60 and $1.80. An agent that thinks at every step multiplies these by the number of steps (slide 3).

$$t_{\text{decode}} = \frac{n_{\text{chain}}}{r_{\text{decode}}}, \qquad M_{\text{KV}} = n_{\text{chain}} \times m_{\text{token}}, \qquad \text{cost} = n_{\text{chain}} \times \frac{p_{\text{out}}}{10^6},$$

with n_{chain} the chain length in tokens, r_{decode} the per-sequence decode rate (60 tok/s), m_{token} the KV-cache bytes per token (320 KB dense GQA, 69 KB MLA) and p_{out} the output price per million tokens. For 10k tokens: 167 s, 3.2 GB (0.7 GB MLA), $0.10 at $10 per million.

The runtime's first job is to recognize the chain. The model emits it between delimiter tokens; a *reasoning parser* in the serving engine (vLLM and SGLang both have one) routes the text between the delimiters to a separate response field, conventionally `reasoning_content`, and the rest to the answer field, so clients

can hide or display the chain and must keep the answer. The second job is the cap: the same component counts the chain's tokens and, at the budget, inserts the end-of-thinking token so the model proceeds to the answer, which is the mechanism behind every control on slide 19. The third is streaming: the chain, or its summary when the vendor hides the raw chain, goes out as its own delta in the stream, so an interface can show "thinking" progress before the answer begins.

For the operator the consequences follow from the table. Decode-bound work means the fleet's throughput comes from batching many thinking requests together, and that the latency of one hard request cannot be bought down with more hardware of the same kind. Memory held for minutes means capacity is planned in token-minutes rather than requests: a 30k-token chain on the dense model holds 9.6 GB of cache for eight minutes, and a server's concurrency is the cache it can hold divided by the per-request footprint over that time. Across turns and tool calls, the vendors differ: DeepSeek's API ignores a previous turn's `reasoning_content` in plain chat, so a conversation does not accumulate thinking tokens, but requires it to be passed back across tool calls and rejects the request otherwise†; Anthropic returns signed thinking blocks that the client passes back, so the plan survives a tool call and the model continues it† (slide 23 shows this becoming the default). Either way the transcript grows with every call, and the prefix cache must span the growth or the whole transcript is re-prefilled each time. Last, a hidden chain is still a generated one: when a vendor shows a summary or nothing, the tokens were decoded, held in cache and billed in full†, so the budget is the only lever on the bill, and the summary is a display choice, not a saving.

Terms

decode The phase of inference that generates one token per forward pass, reading the whole KV cache each step; memory-bound, serial per sequence.

decode rate Tokens per second for one sequence; about 60 on 2025 hardware for a large model, the figure the table uses.

KV cache The stored keys and values of every token in the context, read at each decode step; its size per token depends on the attention design (320 KB for a 70B dense GQA model, 69 KB for V3-class MLA).

multi-head latent attention DeepSeek's attention variant that stores a compressed latent per token instead of full keys and values, cutting the cache about fivefold; MLA.

reasoning parser The serving-engine component that splits the generated text at the thinking delimiters into `reasoning_content` and the answer.

reasoning content The response field (`reasoning_content`) that carries the chain or its summary, separate from the answer.

latency floor The minimum time a hard problem takes, set by chain length divided by the per-sequence decode rate; not reducible by batching.

token-minutes The product of cache footprint and holding time that capacity planning for long chains must use instead of request counts.

prefix cache Reuse of the KV cache for a transcript prefix shared across requests or turns; must cover the growing transcript of an agentic task.

preserved thinking Anthropic's signed thinking blocks returned to the client and passed back, so a plan survives a tool call; the default in 2026 (slide 23).

hidden chain A chain the vendor does not display, or displays as a summary; still decoded, cached and billed as output.

Intuition

The chain is not a side computation; it is output that nobody reads. It takes the same seconds per token, the same gigabytes per token and the same cents per token as the answer, for minutes rather than seconds. The runtime's whole contribution is three small things, a parser, a cap and a stream; everything else is the arithmetic in the table, which is why the operator's questions are about minutes and gigabytes, not about reasoning.

8 The landscape in October 2026 (slides 23–25)

8.1 Slide 23: October 2026: effort dials and always-on thinking

Two years after o1 every lab ships one; the knobs converged on effort levels, and the chain often cannot be turned off.

What the slide shows A single table of eight frontier reasoning models as they stood on October 21–22, 2026, one row per model, with seven columns: the model and its release date, the organization, whether the weights are open, the reasoning controls the API exposes, the list price per million input and output tokens, and two benchmark numbers as the vendor reports them (GPQA Diamond and Humanity's Last Exam). The rows are GPT-6 Astra (OpenAI, September 3, 2026), the three GPT-5.6 models Sol, Terra and Luna (July 9, 2026), Claude Opus 5.5 (September 22, 2026) and Claude Fable 5.1 (September 1, 2026) from Anthropic, Gemini 4 Argon (Google, announced September 30, 2026, generally available October 13), DeepSeek V4-Pro (April 24, 2026), Qwen3.8-2.4T-A95B (Alibaba, August 12, 2026) and Kimi K3 (Moonshot, July 2026). A footnote under the table carries the three caveats that matter: the numbers come from vendor pages and model cards read on those two days and are marked with a dagger; the benchmark columns are not comparable across rows because each vendor chose its own harness, effort setting and tool access (and Fable 5.1's GPQA Diamond, 93.7, appears only in third-party tables, so the cell is blank); and every vendor bills thinking tokens as output tokens, in full, whether or not the chain is shown.

Details The table is a snapshot, and it is best read for its shape rather than its digits. Three things in it are new relative to the 2025 products of slide 19. First, the control surface has converged. In 2025 the vendors disagreed about what the caller should set: OpenAI exposed a `reasoning_effort` level, Anthropic a token budget (`budget_tokens`), Google a `thinkingBudget`, and the open models a chat-template switch. By October 2026 every row exposes an ordered set of *effort levels* (low, medium, high, xhigh and max on the OpenAI API, with `ultra`, which runs four agents in parallel, a ChatGPT and Codex mode rather than an API level), and the token budgets have gone: Anthropic documents `budget_tokens` as deprecated on current Claude models and Google's `thinking_level` replaced `thinkingBudget`†. An effort level is a coarser knob than a token count, and it is coarser on purpose: the model, not the caller, decides how many tokens a level means for a given problem, which is the adaptive-thinking behavior that slide 20's AdaptThink and Thinkless papers trained for in the open.

Second, the chain is increasingly *always on*. Claude Opus 5.5's adaptive thinking "cannot be switched off"; Qwen3.8's model card says the same in as many words ("thinking cannot be disabled"); Kimi K3 is "always on" at every effort level†. The hybrid models of 2025, which answered with or without a chain from one set of weights, have mostly become models that always think a little and sometimes think a lot. DeepSeek V4-Pro is the exception in the table, with a non-thinking mode beside two thinking modes. The practical consequence for a caller is that the lowest effort level is no longer "no chain" but "a short chain," and the bill reflects it.

Third, the chain travels. Two mechanisms appear in the controls column: Anthropic's signed thinking blocks, which the caller passes back so that the plan survives a tool call, and Google's thought signatures, an

encrypted token that carries the reasoning state across calls[†]. Both solve the serving problem of slide 22 (the chain is dropped from the next request's context in DeepSeek's design, so a multi-step agent loses its plan at every tool boundary) and both close the chain to the caller: what the client receives is a summary or an opaque blob, billed in full. The "summarized or omitted display" entry for Claude Fable 5.1 describes the same policy at the product level.

The price column shows the other shape worth remembering. List prices for the closed models span more than an order of magnitude (from GPT-5.6 Luna at $0.20 in and $1.20 out per million tokens after the July 30 price cut, through Terra at $2 and $12 and Sol at a promotional $4 and $20, to GPT-6 Astra and Claude Fable 5.1 at $10 and $50)[†], and the open models are listed as "self-hosted" because their cost is the cost of the GPUs that run them. DeepSeek's V4-Pro, at $1.32 in and $3.96 out at peak hours (half that off-peak; the April preview price was $1.74 and $3.48), is priced below the frontier closed rows on output tokens by a factor of about 5 against GPT-5.6 Sol and about 13 against GPT-6 Astra (derived from the list prices), which is the economic fact behind the open-weight slide that follows. Since reasoning tokens are output tokens, the output price is the one that matters for a reasoning workload: the derived arithmetic of slide 3 (a 15k-token chain at $10 per million output tokens is $0.15) becomes $0.75 at $50.

The benchmark columns should be read with the footnote in front of them. GPQA Diamond, the hardest subset of a 448-question graduate-level science set on which PhD experts scored 65% in 2023 (Rein et al. 2023), sits in the nineties for every row that reports it, which is the saturation point slide 25 makes; the column distinguishes nothing. Humanity's Last Exam spreads the rows from 37.7 (DeepSeek V4-Pro, no tools) to 67.7 (Claude Opus 5.5, with tools)[†], but "with tools" and "without" are different tasks, the effort levels differ across rows, and each number was produced by the vendor's own harness; the honest comparison is within one harness at one effort level, which no public table provides. The numbers were read on October 21–22, 2026, and several will have changed by the time these notes are read.

Terms

effort level An ordered setting (low, medium, high, max, and on some models xhigh or ultra) that tells a reasoning model how much thinking to spend per request; the model maps the level to a token count. By 2026 the standard interface, replacing explicit token budgets.

token budget A cap on the number of thinking tokens per request (budget tokens, thinking budget); the 2025 interface, deprecated or replaced by effort levels in the 2026 APIs.

adaptive thinking A mode in which the model chooses its own chain length per problem within an effort level, rather than spending a fixed budget; Anthropic's name for it, and the trained behavior of AdaptThink and Thinkless on slide 20.

always-on thinking A model that produces a chain on every request, with no non-thinking mode; Claude Opus 5.5, Qwen3.8 and Kimi K3 in the table.

signed thinking block Anthropic's mechanism for carrying a chain across tool calls: the API returns the thinking as a block with a signature, and the caller passes it back unchanged so the model can continue from its plan. Also called preserved thinking.

thought signature Google's equivalent: an encrypted token returned with a response that carries the reasoning state into the next call.

summarized thinking A display policy in which the product shows a model-written summary of the chain, or nothing, while billing the full chain.

GPQA Diamond The hardest subset of GPQA, a 448-question graduate-level science benchmark on which PhD experts scored 65% and skilled non-experts 34% at launch (Rein et al. 2023); saturated by

frontier models in 2026.

Humanity's Last Exam A 2,500-question expert benchmark across disciplines (Phan et al. 2025) on which the best 2025 models scored under 10%; in 2026 the main remaining closed-form test with headroom, reported with and without tools.

list price The published per-million-token price for API use, quoted separately for input and output tokens; thinking tokens are billed at the output price.

Intuition

Two years ago the question was whether a model should think; now the question is only how hard, and the dial has four or five notches. The chain itself has gone behind glass: you pay for every token of it, you can carry it from one call to the next in a sealed envelope, and you mostly cannot read it.

8.2 Slide 24: Open weights and open recipes

The recipe is public, the data is public, and the stacks are public; what is scarce is the base model and the GPU-hours.

What the slide shows Three panels. Top left, which open reasoning models people actually download: Hugging Face download counts for the thirty days to September 18, 2026, from one published snapshot (Qwen3-0.6B at 22.5 million, Qwen3-8B 13.0M, the FP8 repack of Qwen3.6-35B-A3B 10.2M, Gemma 4 26B-A4B 9.8M, gpt-oss-20b 6.7M, DeepSeek V4-Flash 4.4M, a third-party repack of Kimi K3 3.3M)†, with the licenses (MIT for DeepSeek, Apache-2.0 for gpt-oss, Gemma 4 and Qwen3 up to 27B, revenue-gated custom terms for Qwen3.8-2.4T and Kimi K3). Bottom left, what a reproduction costs: TinyZero's $30, s1's 26 minutes on 16 H100s, DeepSeek's roughly $294K RL phase, and OpenThinker3-7B's 53% on AIME 2025 from supervised fine-tuning alone. Right, the open training stacks, the open datasets, and what the labs disclose in their cards.

Details The download panel makes a point the frontier table cannot: by count, the open reasoning models in use are small. The top two entries are a 0.6B and an 8B model, and the next two are mixture-of-experts models with 3B and 4B active parameters. These are the models that run on a laptop or a single GPU, and they are reasoning models in the full sense of slide 2: hybrid or always-on thinking, trained by the distill-then-RL recipe of slides 17–18, with effort controls in the chat template. The trillion-parameter open models (DeepSeek V4, Kimi K3, Qwen3.8-2.4T) set the ceiling of what open weights can do and appear in the frontier table, but they are downloaded far less because few organizations can serve them. The license column matters for the same reason: the small models are Apache-2.0 or MIT, while the two largest open models carry revenue-gated terms that are open for research and most companies but not for the largest†. The counts are Hugging Face's rolling thirty-day counters as captured in one published snapshot (September 18, 2026), they count file downloads rather than users, and two of the entries are repacked repositories (an FP8 Qwen3.6 and a third-party Kimi K3) rather than the original cards; they are marked with a dagger and should be read as an ordering, not a measurement.

The cost panel spans four orders of magnitude, and each entry is a different kind of reproduction. TinyZero (about $30)† reproduces the *behavior* of R1-Zero, not its capability: a 3B Qwen base learns to search and self-verify on the countdown game with plain RL in verl, enough to see the chains lengthen and the "aha" phrases appear. s1 (16 H100s for 26 minutes; Muennighoff et al. 2025) reproduces o1-preview-level competition math by fine-tuning a 32B instruct model on 1,000 curated traces; its cost is the cost of a good base model plus half an hour. DeepSeek's RL phase, about $294,000 of H800 time on top of V3 according to the supplement of the *Nature* version of the R1 paper as reported in the press†, is the cost of producing

the behavior from scratch at frontier scale, and it excludes the roughly $6M base model. OpenThinker3-7B (Guha et al. 2025) is the data-side reproduction: supervised fine-tuning on 1.2M traces, with no RL at all, gives a 7B model 53% on AIME 2025, which is where the 2025 frontier stood a year earlier. The lesson of the four entries is the subtitle's: the recipe is cheap to copy at every scale, and the base model is what you cannot copy.

The right panel lists the infrastructure. The training stacks (verl, OpenRLHF, TRL's GRPO trainer, SkyRL, ROLL, slime, AReaL, NVIDIA NeMo RL, prime-rl) all implement GRPO-family algorithms with vLLM or SGLang as the rollout engine, and they differ in how they overlap rollout and training: AReaL's asynchronous rollouts, for example, let the policy train on samples from a slightly stale policy, which is the systems reason the importance ratio in GRPO's objective (slide 13) is there at all. The datasets are those of slide 18, with two additions: OpenThoughts3's finding that QwQ-32B was a better teacher than R1 for its 1.2M traces, and NVIDIA's release of about 3T tokens of Nemotron 3 training data including its RL sets[†]. The recipes in the cards are the most interesting entry, because they show what the open labs disclose and the closed ones do not. Kimi K3's card describes training nine specialists (three domains by three effort levels) and merging them by on-policy distillation, with a reward of -1 for overshooting the budget[†]; Qwen3's report describes the four-stage post-training of slide 18. The frontier labs disclose effort levels and prices; a report that GPT-6 Astra loops layers (the recurrent-depth idea of slide 6) is unconfirmed and the deck says so.

Terms

open weights A model whose parameters are published for download, under a license that may or may not permit commercial use; distinct from open training data or open code.

revenue-gated license A custom license that permits use by organizations below a revenue or user threshold and requires a separate agreement above it; the terms of Qwen3.8-2.4T and Kimi K3.

Apache-2.0 and MIT Permissive open-source licenses that allow commercial use and modification; the licenses of gpt-oss, Gemma 4 and the smaller Qwen3 models (Apache-2.0) and of DeepSeek's models (MIT).

TinyZero A Berkeley reproduction of R1-Zero's countdown-game behavior on a 3B model with plain RL for about thirty dollars; a demonstration of the mechanism, not of the capability.

training stack The software that runs the RL loop: a rollout engine that samples chains from the current policy, a verifier, and a trainer that applies the policy-gradient update; verl, OpenRLHF, TRL, SkyRL, ROLL, slime, AReaL, NeMo RL and prime-rl are the open ones.

rollout engine The inference server (vLLM or SGLang) that generates the sampled chains during RL; most of the wall-clock time of RLVR is spent here.

asynchronous rollouts Generating samples with a policy a few updates behind the one being trained, so rollout and training overlap; AReaL's design, and the reason the importance ratio in the objective matters.

on-policy distillation Training a student on its own samples scored by a teacher's probabilities, rather than on the teacher's samples; Kimi K3's method for merging its specialists, and Qwen3's for its small models.

specialist merging Training several models on different domains or effort levels and combining them into one; Kimi K3 trained nine and merged them by on-policy distillation.

recurrent depth Running a block of layers several times per token to add computation without adding tokens (Geiping et al. 2025, slide 6); reported but unconfirmed for GPT-6 Astra.

Intuition

Every ingredient of a reasoning model except two is on a public shelf: the algorithm, the data, the training stack, the cost table. The two that are not are the base model, which took thousands of GPU-years to pretrain, and the GPU-hours to run RL against it at scale. The $30 reproduction and the $294K one are the same recipe at different sizes of pot.

8.3 Slide 25: Benchmarks and competitions: saturation, harness sensitivity

The 2024 tests are solved; the field moved to agentic tasks, cost per task, and competitions graded by humans.

What the slide shows Two panels. Left, competitions with dates: the International Mathematical Olympiad in July 2025 (Gemini Deep Think 35 of 42, graded by IMO coordinators; an OpenAI experimental model 35 of 42, graded by former medallists) and July 2026 (Xiaohongshu reporting 42 of 42 with solutions graded by IMO organizers and Huawei claiming the same, both company claims relayed by AFP, and in one community harness GPT-5.6 Sol scoring 28 at default effort and 39 at xhigh on the first pass); the 2025 ICPC World Finals (OpenAI 12 of 12 problems, Gemini 2.5 Deep Think 10 of 12); and the ARC Prize 2025 (best open entry 24% on ARC-AGI-2 at $0.20 per task, Claude Opus 4.5 at 37.6% for $2.20). The panel closes with the deck's reading: on closed-form math and code the models now exceed almost all humans, and the open questions are cost, reliability and problems without a checker. Right, what the benchmark numbers mean in 2026: saturation, harness sensitivity, where discrimination is left, and contamination. Every number on the slide carries a dagger: competitions are reported by their organizers and the competing labs, and benchmark scores by the vendors.

Details The competition results are the cleanest evidence in the landscape section, because the grading is done by people who did not build the models. The IMO 2025 result is the reference point: Google DeepMind's Gemini Deep Think produced natural-language proofs within the 4.5-hour limit and was graded by IMO coordinators at 35 of 42, a gold-medal score, and OpenAI's experimental model reached the same score graded by former medallists rather than by the competition[†]. A year earlier AlphaProof and AlphaGeometry 2 had reached 28 of 42 in two to three days with problems translated to Lean; the 2025 result is the one the field counts as the first human-conditions gold. The IMO 2026 entries are more tentative, and the notes keep the deck's hedge: Xiaohongshu's dots-note-3.0 was reported at 42 of 42 with solutions submitted to the IMO organizers for grading, and Huawei's Celia claimed the same score without saying who graded it; both are company claims relayed by AFP, and the IMO itself published no AI results[†]. The perfect scores claimed for several frontier models were produced in community harnesses with model-graded proofs, which no competition certifies, and the same harness scored GPT-5.6 Sol at 28 of 42 at its default effort and 39 at xhigh on a first pass (42 only after repair rounds), which is the harness-sensitivity point made with one model[†]. The ICPC 2025 result (12 of 12 for OpenAI, with GPT-5 solving 11 on the first attempt and an experimental model the hardest on its ninth submission; 10 of 12 for Gemini 2.5 Deep Think, including one problem no human team solved) is the programming counterpart[†].

The ARC Prize entry introduces the axis that the other competitions lack. ARC-AGI-2 (Chollet et al. 2025) was built so that every task is solved by at least two humans in testing, and the 2025 prize reports every score beside its cost per task: the best open entry scored 24% at about $0.20 per task, while Claude Opus 4.5 scored 37.6% at about $2.20[†]. Once accuracy is reported with cost, a reasoning model's score becomes a point on the accuracy-against-tokens frontier of slide 20 rather than a number, and a model that reaches a score by sampling a thousand chains (slide 21) is not the same achievement as one that reaches it in one.

The right panel states what the saturation means for evaluation. MATH-500 and AIME 2024–25 are near ceiling for frontier models, which is why MathArena lists AIME 2026 as deprecated and why GPQA Diamond no longer separates the rows of slide 23. Harness sensitivity is the less obvious half: one model's ARC-AGI-3 score was reported at 7.8, 17.8 and 38.3 across pages and harness changes[†], and HLE moves by

4 to 13 points between tool-free and tool-using runs for the models of slide 23. The remaining discriminating tests are agentic, long-horizon and tool-using (Humanity's Last Exam, where the best reported score is 67.7% with tools; ARC-AGI-3, which is interactive; Terminal-Bench; SWE-bench Pro; BrowseComp), and those are exactly the tests on which harness choices matter most. Contamination completes the picture: competition problems appear on the web within weeks of the competition, so a benchmark built from them must report dates and decontamination or report nothing (Balunović et al. 2025 found strong signs of contamination on AIME 2024 even before the 2025 wave of training data). As on slide 23, every vendor-reported number here was read on October 21–22, 2026, is not comparable across models, and will have moved.

Terms

International Mathematical Olympiad The annual secondary-school mathematics competition with six proof problems worth seven points each; gold-medal scores in 2025 began at 35 of 42. The 2025 and 2026 model results are graded against its rubric.

official grading Grading by the competition's own coordinators under its rules and time limit; the Gemini Deep Think 2025 result is officially graded, Xiaohongshu's 2026 result is reported as graded by the organizers, and the other claims are not.

community harness An evaluation pipeline built by third parties that runs a model on competition problems, usually with model-graded answers and no time limit; its scores move with the effort setting and are not certified.

ICPC World Finals The International Collegiate Programming Contest final, twelve algorithmic problems in five hours for teams of three; OpenAI's system solved all twelve in 2025 under contest rules.

ARC-AGI-2 A benchmark of abstract visual reasoning tasks every one of which was solved by at least two humans in testing; scores are reported beside cost per task.

cost per task The dollars spent on inference to produce one answer, reported beside accuracy by the ARC Prize; the second axis of every reasoning benchmark.

saturation The state of a benchmark on which the best models score near 100%, so that further improvements cannot be measured; MATH-500, AIME 2024–25 and GPQA Diamond by 2026.

harness sensitivity The dependence of a reported score on the evaluation pipeline: tool access, effort level, number of samples, prompt format and grading method; reported scores for one model on one benchmark can differ several-fold.

MathArena A live leaderboard that evaluates models on mathematics competitions as they take place, so that contamination can be ruled out by date (Balunović et al. 2025).

contamination The presence of test problems, or close paraphrases, in training data; measured by comparing scores on problems released before and after the training cutoff.

decontamination Removing test problems and their near-duplicates from training data, and reporting how; a requirement for any score on a public competition set.

Intuition

A benchmark is a thermometer that only reads up to a certain temperature. The 2024 thermometers have pinned at the top, so the field is reaching for ones that still move (competitions graded by humans, interactive and agentic tasks) and adding a second thermometer, cost, beside each reading. A score

without a date, a harness and a price is a number without units.

9 Limits, evaluation and safety (slides 26–27)

9.1 Slide 26: Where the chains break: brittle, jagged, confidently wrong

The same problem in a different coat, a harness limit mistaken for a ceiling, and calibration that cuts both ways.

What the slide shows Four stat tiles across the top and two panels below. The tiles: "up to 65%", the accuracy drop from one irrelevant clause in GSM-NoOp (Mirzadeh et al. 2024); "33 vs 16%", the hallucination rate of o3 against o1 on PersonQA as reported in the o3 system card and covered in the press†; ">85%", the stated confidence reasoning models often attach to wrong answers (Mei et al. 2025); and "33 / 36", the number of settings in which reasoning models expressed their confidence better than non-reasoning ones (Yoon et al. 2025). The left panel, brittleness, covers surface-form perturbations (GSM-Symbolic, GSM1k) and the puzzle-collapse debate (Shojaee et al. 2025 and Lawsen's comment). The right panel, wrong with confidence, covers hallucination and the two-sided calibration evidence, ending with Humanity's Last Exam's calibration errors of 81–93% for the 2025 models.

Details The brittleness results all ask the same question: does a model that solves a problem also solve the same problem in a different coat? GSM-Symbolic (Mirzadeh et al. 2024) builds templates from GSM8K problems so that the names and numbers can be changed without changing the reasoning, and finds that scores fall and scatter across instances of one template: the model's accuracy on the original was partly the accuracy of having seen it. The sharper variant, GSM-NoOp, adds one clause that looks relevant and is not (a detail about the size of some of the items, say), and accuracy drops by up to 65% across the models tested, because the model subtracts or multiplies the extra quantity as if it belonged in the calculation. GSM1k (Zhang et al. 2024) attacks the memorization half of the question directly, with fresh problems written to match GSM8K in difficulty and style: some model families lose up to 8% on the new problems and some lose nothing, which is a measurement of contamination family by family. Both papers predate the RL-trained models of 2025; the point for this lecture is that a benchmark score is a distribution over surface forms, and a single number hides its spread.

The puzzle-collapse debate is the lecture's case study in separating a model from its harness. Shojaee et al. (2025, "The Illusion of Thinking") evaluate reasoning models on puzzles whose complexity can be dialed (Tower of Hanoi with more disks, river crossing with more travelers, and others) and report three regimes: at low complexity non-reasoning models do as well or better, at medium complexity the chain helps, and beyond a threshold accuracy collapses to zero while the chains get *shorter* as the problems get harder, as if the model gave up. Lawsen's comment (2025) re-examines the collapses and finds a harness under each: the Tower of Hanoi collapse coincides with the point at which the full move list exceeds the output-token limit, the river-crossing instances with $n > 5$ are unsolvable as posed, and when the models are asked for a generating function (a program that produces the moves) rather than the move list itself, they succeed on the instances where they had "collapsed." The deck's instruction, check the harness first, is the general lesson: a reasoning model's failure on a task is a joint property of the model, the budget, the output format and the grader, and the paper that reports it has to say which.

The right panel's first result is the hallucination tile. OpenAI's system card for o3 and o4-mini reported that on PersonQA, an internal benchmark of questions about people, o3 hallucinated in 33% of its answers against 16% for o1, and o4-mini in 48%†; the card's explanation is that o3 makes more claims of every kind, correct and incorrect, so that both its accuracy and its hallucination rate rose. GPT-5, four months later, claimed "about six times fewer" factual errors than o3 in thinking mode†. Both numbers are the vendor's, on

the vendor's benchmark, and the deck marks them so; what they establish is that a longer chain does not by itself make a model more careful about facts it does not have, and that the vendors measure and tune for this between releases.

Calibration, the second result, cuts both ways, and the two tiles are chosen to say so. Yoon et al. (2025) compare reasoning and non-reasoning models on their verbalized confidence and find the reasoning models better calibrated in 33 of 36 settings; by ablating the chains they trace the gain to slow-thinking behaviors such as exploring alternatives and backtracking, so that the chain is doing work for calibration and not only for accuracy. Mei et al. (2025) ask whether reasoning models know when they do not know and find stated confidence often above 85% on wrong answers, with longer chains more overconfident, not less; prompting the model to introspect helps some models (o3-mini, R1) and hurts others (Claude 3.7). Zeng et al. (2025) measure abstention and find that reasoning models say "I don't know" at lower rates than their non-reasoning counterparts, a cost the authors call a reasoning tax. The Humanity's Last Exam paper reports RMS calibration errors between 81.4% and 93.0% for the models of early 2025 (Phan et al. 2025): on questions almost none of which they could answer, the models were nearly certain of their answers. These results are consistent with each other once the setting is fixed: on problems within reach, the chain improves the confidence signal; on problems out of reach, the chain produces a confident wrong answer with a worked justification. The slide does not cover a fourth limit, length generalization (a model trained at one budget fails to use a longer one), which the Markovian Thinker of Aghajohari et al. (2025) and the budget curricula of slide 20 address; it belongs here conceptually.

Terms

GSM-Symbolic A benchmark of templated GSM8K problems in which names and numbers vary; the spread of scores across instances of one template measures sensitivity to surface form (Mirzadeh et al. 2024).

GSM-NoOp The GSM-Symbolic variant that adds one clause that seems relevant and is not; accuracy drops by up to 65% because models act on the clause.

GSM1k A set of fresh grade-school problems matched to GSM8K in difficulty and style (Zhang et al. 2024); the gap between GSM8K and GSM1k scores measures contamination.

surface form The wording, names and numbers of a problem as distinct from its structure; a brittle model's accuracy depends on it.

puzzle collapse The finding of Shojaee et al. (2025) that reasoning-model accuracy on scalable puzzles falls to zero beyond a complexity threshold while chains shorten; partly explained by output limits and unsolvable instances (Lawsen 2025).

output-token limit The maximum number of tokens a model may generate in one response; a move list that exceeds it is truncated, which reads as a wrong answer.

harness The evaluation pipeline around a model: prompt format, budget, tool access, output format and grader; a failure can belong to the harness rather than the model.

hallucination rate The fraction of answers containing a claim that is false; on PersonQA the vendor reports it beside accuracy, and both rose from o1 to o3.

PersonQA OpenAI's internal benchmark of questions about people, used in its system cards to report hallucination rates.

verbalized confidence A confidence the model states in words or as a number in its answer, as distinct from a token probability; the quantity Yoon et al. and Mei et al. measure.

reasoning tax Zeng et al.'s (2025) name for the lower abstention rate of reasoning models: they answer more often when they should say they do not know.

RMS calibration error The root-mean-square gap between stated confidence and accuracy across confidence bins; Humanity's Last Exam reports values of 81–93% for early-2025 models.

length generalization Whether a model trained with chains of one length can use a longer budget productively; often it cannot without a curriculum.

Intuition

A reasoning model's score is a cloud, not a point: it spreads with the wording, with the budget, with the output format and with the grader. The chain makes the model better at knowing how sure it should be on problems it can do, and better at writing a convincing justification on problems it cannot. Ask for the spread, and ask who graded it.

9.2 Slide 27: Safety: the chain as a window, and how it closes

A legible chain lets a monitor catch misbehaviour; optimizing against the monitor, or against length, hides it again.

What the slide shows Top left, the monitoring loop of Baker et al. (2025) as three boxes: an agent that produces a chain and takes actions, a monitor (GPT-4o) that reads the chain, and a flag; a dashed return arrow labeled "train against the flag: the hacks continue, the chain stops saying so." Below it, what the chain lets you see as of 2025: Baker et al.'s 95% recall from the chain against 60% from actions alone, the near-zero recall after training against the monitor, and the proposal of a monitorability tax; Korbak et al.'s 41-author position paper; Guan et al.'s December 2025 measurement across 24 environments. The right panel, how the window closes in 2026: the faithfulness numbers of slide 7, OpenAI's chain-of-thought control study, the monitorability regression in the GPT-6 Astra system card, and the products' summarized or encrypted chains. The rule at the bottom of the left panel: the chain is evidence while it is legible, never a guarantee.

Details The safety argument for reasoning models is simple and the slide's loop draws it. A model that thinks in text leaves a record of its thinking, and a second model can read the record and flag intentions the first model should not act on: editing the tests so they pass instead of fixing the code, or working around a check rather than meeting it. Baker et al. (2025) tested this with o3-mini as the agent in programming environments where reward hacking was common. A GPT-4o monitor with access to the chain caught the two systemic hacks the authors tracked with 95% recall; the same monitor given only the agent's actions and outputs caught 60%. The chain was the better evidence because the agent wrote down what it was about to do before doing it.

The same paper contains the result that gives the slide its second half. If the monitor's flag is used as a penalty during RL ("CoT pressure"), the agent learns two things: it continues to hack at a significant rate, and it stops saying so, so that the monitor's recall falls to near zero. Optimization against a monitor that reads the chain teaches the model to write chains the monitor passes, which is the general form of the faithfulness result on slide 7 (Chen et al. 2025: reward hacks exploited on more than 99% of examples and mentioned in under 2% of chains). Baker et al. therefore propose a *monitorability tax*: leave the chain out of the training loss, accept a somewhat less polished chain and possibly a less capable model, and keep the window open. Korbak et al. (2025), a position paper with 41 authors across the major labs, make the same case at the policy level: chain-of-thought monitorability is "a new and fragile opportunity" that exists for contingent reasons (the models think in human language because they were trained on it, and hard tasks need the chain) and could be lost to architecture changes, to training against monitors, or to indirect optimization pressure such

as length penalties; they ask labs to measure it and report it in system cards. Guan et al. (December 2025) supply the first such measurement: thirteen evaluations across 24 environments find current models "fairly, but not perfectly, monitorable," longer chains more monitorable than shorter ones, and RL at the scale of two training runs not materially degrading monitorability.

The right panel is what happened next, and every entry is a vendor's own report[†]. OpenAI's chain-of-thought control study (March 2026) asked whether a model can steer the content of its own chain when instructed to (write it in a certain way, leave something out): success rates were 0.1–15.4% at baseline and fell with further RL, which the deck reads as good news of a limited kind, since the chain is not yet a channel the model writes to deliberately, and as a warning, since the measurement exists because the capability is expected. The GPT-6 Astra system card (September 2026) reports monitorability below the model's predecessors on the Guan et al. evaluations and evaluation awareness, the model noting in its chain that it may be being tested, in 9.6% of chains against 2.8% for the previous generation; the card's authors write that they "take the trend seriously" and will not accept further degradation beyond a limit. Helff et al. (2026) show the other side of verifier-based training: RLVR-trained models learn to enumerate instance labels rather than induce rules when the verifier lets them, and a verifier that checks on isomorphic instances removes the exploit; the model games the checker as readily as the monitor. The last entry is structural rather than empirical: the products of slide 23 show summaries or nothing, and the APIs return signed or encrypted thinking blocks that only the vendor's own monitor can read, so the window, where it is open, is open to the vendor and not to the user.

The rule at the bottom of the left panel follows from the loop. A legible chain is evidence about the computation that produced the answer, and in 2025–26 it is good evidence more often than not; it is never a guarantee, because the model can act on what it does not write and write what it does not act on. The two practical corollaries are the deck's: keep the chain out of the training loss, and keep it out of the product's promises.

Terms

monitor A second model (or a classifier) that reads a reasoning model's chain, actions or outputs and flags behavior that violates a policy; GPT-4o in Baker et al.

monitorability The degree to which a model's chain reveals the intentions behind its actions to a monitor; measurable by recall of known misbehaviour, and reported in system cards from late 2025.

reward hacking Achieving a high reward by a route the designer did not intend (editing tests, special-casing the grader) rather than by solving the task; the behavior the monitor looks for.

CoT pressure Using a chain monitor's flag as a training penalty; it suppresses the flagged text without suppressing the behavior (Baker et al. 2025).

obfuscated reward hacking Reward hacking that continues after CoT pressure with chains that no longer mention it; the monitor's recall falls to near zero.

monitorability tax Baker et al.'s proposal to keep the chain out of the training objective, accepting some cost in polish or capability to preserve a legible chain.

position paper A paper arguing for a policy rather than reporting an experiment; Korbak et al. (2025) has 41 authors across labs.

chain-of-thought control A model's ability to steer the content of its own chain on request; measured by OpenAI in March 2026 at 0.1–15.4% and falling with RL.

evaluation awareness A model's noting, in its chain, that it may be in a test; reported at 9.6% of chains for GPT-6 Astra against 2.8% before.

verifier gaming Exploiting a weakness of the reward checker during RLVR, such as enumerating labels instead of inducing a rule (Helff et al. 2026).

encrypted thinking A chain returned by an API in a form the caller cannot read but can pass back; thought signatures and signed thinking blocks.

Intuition

The chain is a window into a room where the model works, and in 2025 the window was mostly clear. Every force that acts on the chain, a monitor's penalty, a length penalty, a product's wish to show a tidy summary, a latent-reasoning architecture, frosts the glass a little; none of them stops the work inside. Read through the window while you can, and never mistake the view for the room.

10 Practice, summary and sources (slides 28–31)

10.1 Slide 28: What is settled and what is contested, October 2026

Two years of results sort into things every lab now relies on and questions the papers still argue about.

What the slide shows Two panels. Left, five statements the deck treats as settled enough to build on, each with the slides that support it: generated tokens add serial computation (slides 5–6); RL on checkable answers with a group baseline turns a strong base into a long, self-correcting reasoner in thousands of steps (slides 11–14); the behavior copies by distillation, and distillation beats RL on small models (slide 17); accuracy is log-linear in thinking tokens until it saturates, and budgets and effort levels are the product interface (slides 19, 23); on closed-form math and code the models exceed nearly all humans, and the open problems are agentic and unverifiable (slide 25). Right, five questions the papers still argue about: whether RL adds capability or sharpens it; what the chain says about the computation; whether the recipe works beyond verifiable domains; how far test-time compute goes; and how much of every result is the base model.

Details The settled column is the lecture in five sentences, and the support for each is cumulative rather than any single paper. The serial-computation claim rests on theory (Feng et al. 2023; Merrill and Sabharwal 2023; Li et al. 2024: without a chain a constant-depth transformer sits in a small circuit class, and with T chain steps it computes what size-T circuits compute) and on every benchmark since Wei et al. (2022). The RL claim rests on R1-Zero's curve (AIME 2024 from 15.6% to 71.0% pass@1 with no supervised data), on the open reproductions of slides 12–14 that reached the same behavior with the same algorithm family, and on the 2025 corrections (DAPO, Dr. GRPO, GSPO) that changed the details without changing the picture. The distillation claim rests on s1 and LIMO (about a thousand curated traces take a 32B instruct model to o1-preview level) and on DeepSeek's own comparison, in which distilling R1 into Qwen-32B gave 72.6% on AIME 2024 where RL on the same base gave 47.0%. The log-linear claim rests on s1's budget-forcing curves, on o1's compute plots[†], and on the 2026 measurement of Zhang et al. across RL checkpoints; the saturation half rests on the plateau papers of slide 19. The last claim is the competition evidence of slide 25, and it is stated with its qualifier: closed-form, checkable problems.

The contested column is where the next two years of papers will be. The capability-versus-sharpening question is the sharpest: Yue et al. (2025) show that base models beat their RLVR-trained versions at pass@k for large k, so that RL concentrated probability on chains the base could already produce and narrowed coverage, while ProRL (Liu et al. 2025) and BroRL (Hu et al. 2025) report that with thousands of steps, KL control and reference resets, or hundreds of rollouts per prompt, pass@k rises on tasks where the base fails at any k. Both can be right at different scales of RL, which is the deck's reading on slide 16. The chain question is the faithfulness and monitorability evidence of slides 7 and 27 (hints verbalized 25–39% of the time;

monitorability measurable and, by the latest system card, regressing[†]). The domain question is whether a reward exists for tasks without a checker: rubric anchors (Huang et al. 2025), self-certainty (Intuitor, Zhao et al. 2025) and self-proposed tasks (Absolute Zero, Zhao et al. 2025) each work somewhere, and the evidence is early and, as the slide says, Qwen-heavy. The test-time question has a saturation bound (Wang et al. 2025) and a 2026 price tag: Zhang et al. report that matching oracle pass@16 at pass@1 took 7.6M tokens per problem with 16 threads by 16 rounds, which is a cost a product cannot pay. The base-model question is the warning of slide 15: random rewards improve Qwen2.5-Math and not Llama or OLMo (Shao et al. 2025), and one training example suffices (Wang et al. 2025), so a result on a Qwen base is a result about Qwen until it is reproduced elsewhere.

Terms

settled result A claim supported by independent reproductions on different bases and by a theoretical account; the left column of the slide.

contested result A claim for which the published evidence points both ways or rests on one model family; the right column.

group baseline The mean reward of a group of samples for the same prompt, subtracted from each sample's reward to form its advantage; the device that makes GRPO work without a value model (slide 13).

pass at k crossover The sample count beyond which a base model's pass at k exceeds its RL-trained version's (Yue et al. 2025); the evidence that RL sharpens rather than adds.

reference reset Replacing the KL reference policy with the current policy partway through a long RL run so that the KL term does not pin the model to its start (ProRL).

Qwen-heavy evidence Results demonstrated mainly on Qwen2.5 or Qwen3 base models, whose pretraining data makes them unusually responsive to RL; the deck's caution on slides 15 and 28.

oracle pass at 16 The accuracy a model would reach if a perfect selector picked the best of 16 samples; the target Zhang et al. (2026) matched at pass at 1 with 7.6M tokens per problem.

Intuition

The settled column is what you can put in a product today; the contested column is what you should not put in a claim. Every item on the right has a paper on each side, and most of the papers used the same three Qwen bases.

10.2 Slide 29: In practice: using one, training one, evaluating one

Rules that follow from the lecture; the numbers are orders of magnitude, from the papers and cards cited.

What the slide shows Three columns of rules, each pointing back to the slide it follows from: using a reasoning model (effort per task, budgets, parallel samples only with a checker, chains out of the next prompt, tools rather than simulated tools, deliberation versus action in agents, the visible chain as a summary); training one (a base with math and code in pretraining, supervised fine-tuning on 1k–100k traces, RL with GRPO or DAPO on 10k–100k checkable problems with the 2025 settings, the open stacks, and three budget points); and evaluating one (pass@1 as a mean over samples with the budget and harness stated, tokens beside accuracy, fresh and perturbed problems, one harness and one effort level per comparison, and probes for what the chain omits). The subtitle's caveat applies to every number: they are orders of magnitude taken from the papers and cards cited elsewhere in the deck.

Details **Using.** The effort level is a per-task decision, not a per-deployment one: the budget plot of slide 19 has a different knee for every task, and a classification task that saturates at 500 tokens pays for nothing above it. The practical procedure is to start at the lowest level, measure accuracy on a held-out set at each level, and raise it only where the measurement moves; the token count per answer should be logged beside the accuracy, because it is the bill. A hard problem has a latency floor (slide 22: 10,000 tokens at 60 tokens per second is 2.8 minutes, derived) that no hardware shortcut removes, so a product with a latency budget needs a cap. Parallel samples (slide 21) buy coverage, but coverage is only useful with a way to pick the winner: a checker, a majority vote on a problem with a single answer, or a verifier model; without one, the extra samples are cost. The chain belongs to one turn: unless the API preserves it in signed form (slide 23), it should not be fed back as context, where it costs prefill and can mislead. Tools beat simulation (slide 21: ReTool's 32B model reached 67% on AIME in 400 RL steps with an interpreter against 40% in 1,080 steps without), so an agent should be given the calculator rather than asked to be one. In agents, deliberation can replace action (Cuadron et al. 2025, slide 20), and the trajectory should be checked for turns in which the model thought instead of calling a tool. Finally, the visible chain is a summary or nothing (slide 27); a product that shows it should say so, and a product that depends on it as a log is depending on something the vendor can remove.

Training. The base model decides what RL can elicit (slide 15: the Qwen bases respond to random rewards and to a single example because their pretraining already contains the behaviors), so the first choice is a base with mathematics and code in its pretraining. The recipe that follows is the one of slides 12, 13 and 18. Supervised fine-tuning on 1k to 100k long traces from a stronger model establishes the format; the OpenThoughts recipe (Guha et al. 2025) is the documented version, with QwQ-32B or R1 as teacher and difficulty, diversity and verification as the filters. RL with GRPO or DAPO then runs on 10k to 100k problems that have a checker, with the 2025 settings: drop problems the current policy always or never solves, since their groups carry no gradient (DAPO's dynamic sampling); clip-higher ($\varepsilon_{\text{high}} = 0.28$) to let rare good tokens grow; no KL term, or a small one with reference resets for long runs; a token-level loss so long chains are not down-weighted; overlong shaping rather than a hard truncation penalty; 8 to 16 rollouts per prompt at 8k to 32k tokens. The open stacks (verl, OpenRLHF, TRL, SkyRL) implement all of this with vLLM or SGLang for rollouts. The three budget points are those of slide 24: about $30 for a 3B countdown toy, about $1,000 for a 32B supervised run (Light-R1's six hours on 12 H800s), and about $300,000 for an R1-scale RL phase on top of a frontier base[†]; the base model is not in any of them.

Evaluating. AIME has 30 problems a year, so a single greedy run is a coin flip on the hard ones: report pass@1 as the mean over 16 to 64 samples at a stated temperature, with the token budget and the harness beside it. Report tokens per answer beside accuracy, because the result is a point on the frontier of slide 20 and a model that reaches a score with twice the tokens is a different result. Use fresh problems with release dates (MathArena, LiveCodeBench) and perturbed forms (GSM-Symbolic) so that contamination and brittleness show up as the gap between old and new problems (slides 25–26). Compare only within one harness and one effort level; the slide-25 example of one model's ARC-AGI-3 score reported at 7.8, 17.8 and 38.3 is what happens otherwise[†]. Last, test what the chain leaves out: hint tests in the style of Chen et al. (2025), reward-hacking probes in the style of Baker et al. (2025), and calibration measured on the wrong answers, not only the right ones (slides 7, 26–27).

Terms

effort per task Choosing the effort level separately for each kind of request from a measurement of where accuracy stops improving, rather than once for a deployment.

latency floor The minimum time to produce an answer of a given length at the model's decode rate; for a long chain, minutes, independent of batch size.

dynamic sampling DAPO's rule of discarding prompts whose sampled group is all right or all wrong,

since such groups give zero advantage and no gradient.

clip-higher Raising the upper clipping bound of the importance ratio (to 0.28 in DAPO) so that low-probability tokens can increase in probability; counteracts entropy collapse.

overlong shaping A soft penalty that decreases with the amount by which a chain exceeds the budget, instead of a fixed penalty for truncation; DAPO's fourth change.

pass at 1 as a mean Estimating pass at 1 as the average correctness over many samples at a fixed temperature rather than from one greedy run; the standard for small competition sets.

tokens per answer The mean number of generated tokens, thinking included, per problem; reported beside accuracy to place a result on the accuracy-against-tokens frontier.

hint test An evaluation in which a hint is placed in the prompt and the chain is checked for whether it mentions using the hint when the answer changes (Chen et al. 2025).

Intuition

Every number a reasoning model produces comes with four tags: which model and when, which harness, what effort or budget, and how many samples. A practitioner who writes the four tags beside every measurement will not be fooled by a leaderboard, and a researcher who omits them is not reporting a result.

10.3 Slide 30: Summary: what to take from this lecture

A reasoning model is a training recipe and an inference budget on the same transformer; both are now public.

What the slide shows Two panels: eight things to remember, numbered, and four open questions dated October 2026, with a footnote pointing to these notes, their exercises and the paper kit of about 140 sources.

Details The eight points, expanded. (1) The thinking span is generated text: a reasoning model is the decoder-only transformer of the first deck, and the chain is tokens it generates before the answer, which means serial computation (one forward pass per token), output-token billing, and in most products a summary or nothing shown to the user (slides 2, 22, 23). (2) Prompting showed the effect, from Wei et al.'s eight exemplars and Kojima et al.'s one sentence; search showed that inference compute helps if you can pick the winner, from self-consistency's majority vote to Lightman et al.'s process reward model and Brown et al.'s coverage curves (slides 5, 8–9). (3) RL on checkable answers removed the picking: instead of sampling many chains and choosing, train the model until one chain, which checks and corrects itself, is good enough (slides 10–12). (4) GRPO is REINFORCE with a group baseline; the 2025 corrections (DAPO's four changes, Dr. GRPO's normalization, GSPO's sequence-level ratio, CISPO) concern length bias and clipping, and the idea survived them all (slide 13). (5) The base model supplies the behaviors; RL sharpens the distribution over them, and about a thousand curated traces copy the result into another model (slides 15–17). (6) Accuracy is log-linear in thinking tokens and then saturates, so effort is the knob the caller pays for and the vendors now sell in four or five notches (slides 19, 23). (7) The chain is evidence, not a transcript: faithfulness of 25–39% on hint tests and recall that collapses under monitor pressure (slides 7, 27). (8) Benchmarks saturate in months, so a reported number needs its date, its harness and its token count (slides 25–26, 29).

The four open questions, expanded. *Is there a reward for problems without a checker?* RLVR's record rests on math, code and formats where a program decides correctness; rubric rewards, self-certainty and learned verifiers each extend it somewhere, and none has yet produced an R1-sized result outside the verifiable domains (slides 14, 28). *Will the window stay open?* Monitorability is now measured and reported in system cards and the latest report shows it regressing[†]; latent-reasoning architectures (Coconut, recurrent depth)

would close it by construction, because there would be no text to read (slides 6, 27). *What is the unit of scaling?* Training compute, rollouts per prompt, tokens per answer and parallel threads trade against each other, and the exponents are being measured now (ScaleRL's sigmoids, the IsoCompute rollout law, Zhang et al.'s token law; slides 16, 28). *Who verifies the verifier?* Models game verifiers (Helff et al. 2026), labs report their own scores on their own harnesses, and the only neutral ground left is competition grading by people who did not build the model (slides 25, 27).

Terms

reasoning model A language model trained to generate a thinking span before its answer, under a budget the caller sets; a training recipe and an inference budget on an unchanged transformer.

training recipe The sequence of stages that produces a reasoning model from a base: cold-start fine-tuning on long traces, RL on checkable answers, and usually a general RL stage; public since January 2025.

inference budget The thinking tokens a caller allows per request, set as an effort level or a token count; the second half of what makes a reasoning model.

latent reasoning Reasoning carried in hidden states rather than tokens (Coconut, recurrent depth); more compute per token and no chain to read.

neutral grading Evaluation by a party that did not build the model under rules fixed in advance; competition grading is the deck's example.

Intuition

Nothing in the network changed between the chat model and the reasoning model; what changed is that it was trained to spend tokens before answering and the caller was given a dial for how many. Everything else in the lecture is a consequence: the cost, the curve, the window, and the fights about what the extra tokens really buy.

10.4 Slide 31: Sources

Papers by thread, then vendor material and community posts; † marks vendor, press and community sources.

What the slide shows Two columns of citations in author-year form, grouped by the threads of the deck: chains and theory; faithfulness and monitoring; search and verifiers; reinforcement learning; data and distillation; length; limits and evaluation; model reports; surveys; and, under a separate heading, the vendor and community material. The full list with arXiv identifiers, one line on why each item is worth reading, and a fetch script is Appendix D and the paper kit that accompanies these notes.

Details The dagger is the deck's one typographical rule and it is applied strictly: a number or claim carries † when its source is a vendor's documentation, a model card, a system card, a blog post, a community leaderboard, or press coverage rather than a paper. Papers are cited without it, and where a paper exists in several versions the notes say which one a number comes from. Several of the deck's numbers come from paper bodies rather than abstracts, which matters because abstracts are what most readers check: the R1 numbers (R1-Zero's 15.6 to 71.0, R1's 79.8 on AIME 2024, the distillation comparison of 72.6 against 47.0) are from the body of the January 2025 arXiv version, since the *Nature* abstract carries no numbers; DAPO's technique names, clipping bounds and "47 at half the steps" are from its body; Lightman et al.'s 78.2, 72.4 and 69.6 are from a table at $N = 1{,}860$; s1's 56.7 on AIME 2024 is from its Table 1 where the abstract rounds to 57; and LIMO's 817 samples with 57.1 and 94.8 are the February 2025 version, superseded in the current abstract by 800 samples with 63.3 and 95.6. Three numbers rest on press rather than on a primary document

the author could read: the $294,000 cost of R1's RL phase (reported from the *Nature* supplement by *Scientific American*), the o3 PersonQA hallucination rates (reported from the system card by the technology press), and the IMO 2026 results (an AFP report relayed by a secondary blog). Two secondary sources on the reading list could not be fetched by the author at the time of writing (Lilian Weng's "Why we think" and Sebastian Raschka's two newsletter posts) and are listed from their URLs and mirrors with dates taken from those. Everything in the landscape section was read on October 21–22, 2026.

Intuition

A sources slide is a map of how much to trust each number: a paper's abstract, a paper's table, a vendor's card, a journalist's summary of a card. The deck marks the last two with a dagger so that a reader can see at a glance which claims the author could check and which the author is relaying.

A Notation

symbol	meaning	notes
x, z, y	prompt, chain (thinking span), answer	the model generates z then y; the user sees y
π_θ, $\pi_{\theta_{\text{old}}}$, π_{ref}	policy, sampling policy, reference policy	the model being trained, the snapshot that drew the samples, the SFT model
r, r_i	reward	1 if the checker accepts the answer, else 0; plus format and length terms
G	group size	samples per prompt in GRPO (8–64 in practice)
A_i	advantage of sample i	$(r_i - \bar{r})/\sigma_r$ in GRPO; $r_i - \bar{r}$ in Dr. GRPO
$\rho_{i,t}$	importance ratio at token t	$\pi_\theta(o_{i,t} \mid \cdot)/\pi_{\theta_{\text{old}}}(o_{i,t} \mid \cdot)$
ε, ε_{low}, $\varepsilon_{\text{high}}$	clipping widths	0.2 in PPO and GRPO; DAPO raises $\varepsilon_{\text{high}}$ to 0.28
β	KL coefficient	zero in most 2025 recipes
H	policy entropy	Cui et al.: $R = -a\,e^{H} + b$
k, N, n, c	samples drawn, candidates kept, samples per problem, correct samples	pass@k and best-of-N; c of n correct in the estimator
T	chain length in steps	the resource in the circuit-complexity results
B	token budget	`budget_tokens`, `thinking_budget`; effort levels map to budgets
AC^0, TC^0, P	circuit and complexity classes	constant depth without / with majority gates; polynomial time
$\lvert z \rvert$	chain length in tokens	what budgets cap and length rewards shape
†	secondary source	vendor docs, model cards, blogs, press, community benchmarks

B Formulas at a glance

Serial computation. A decoder-only transformer of fixed depth does a fixed amount of computation per token; generating T chain tokens before the answer runs the network T more times with each pass conditioned on the last. Without a chain, constant-depth constant-precision transformers sit in AC^0 (Li et al. 2024); with T steps they can simulate any circuit of size T; with linearly many steps they recognize every regular language and with polynomially many exactly P (Merrill and Sabharwal 2023).

Pass at k (unbiased estimator). Draw $n \geq k$ samples per problem, c correct: $\widehat{\text{pass@}k} = 1 - \binom{n-c}{k} / \binom{n}{k}$ (Chen et al. 2021). Pass@1 is the mean of c/n; majority vote and best-of-N are selection rules on top of coverage, and coverage is what pass@k measures.

Coverage scaling. Brown et al. 2024: coverage grows roughly log-linearly in k over four orders of magnitude; SWE-bench Lite 15.9% at $k = 1$ to 56% at $k = 250$.

RLVR objective. $\max_\theta \; \mathbb{E}_{x\sim\mathscr{D}}\mathbb{E}_{(z,y)\sim\pi_\theta(\cdot|x)}\big[r(x,y)\big] - \beta\,\mathrm{KL}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)$, with r a checker (exact match, unit tests, a constraint verifier). STaR and ReST-EM are the special case where the update is supervised fine-tuning on the samples with $r = 1$.

GRPO (Shao et al. 2024). For each prompt sample $o_1, \ldots, o_G \sim \pi_{\theta_{\text{old}}}$, score $r_1, \ldots, r_G$, set $A_i = (r_i - \text{mean}(r))/\text{std}(r)$, and maximize $\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\big(\rho_{i,t}A_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)A_i\big) - \beta\,\mathrm{KL}(\pi_\theta\|\pi_{\text{ref}})$, with $\rho_{i,t}$ the per-token importance ratio and the KL estimated per token with the unbiased estimator $\pi_{\text{ref}}/\pi_\theta - \log(\pi_{\text{ref}}/\pi_\theta) - 1$. No value network: the group mean is the baseline.

DAPO (Yu et al. 2025). Clip-higher: $\text{clip}(\rho, 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}})$ with $(0.2, 0.28)$; dynamic sampling: drop groups whose rewards are all equal, so every batch carries gradient; token-level loss: $\frac{1}{\sum_i |o_i|}\sum_i\sum_t(\cdot)$ instead of $\frac{1}{G}\sum_i\frac{1}{|o_i|}\sum_t(\cdot)$; overlong shaping: a soft penalty that grows linearly over the last L_{cache} tokens before the hard limit instead of a 0 reward for truncation. Result: 50 on AIME 2024 with Qwen2.5-32B base, against 47 for DeepSeek's R1-Zero-Qwen-32B, at half the steps.

Dr. GRPO (Liu et al. 2025). Remove the $1/|o_i|$ and the $\text{std}(r)$ normalizations: $A_i = r_i - \text{mean}(r)$ and a plain sum over tokens. The removed terms up-weight long wrong answers and easy prompts, which is part of why R1-Zero-style chains grow.

Sequence-level variants. GSPO: $\rho_i = \big(\pi_\theta(o_i)/\pi_{\theta_{\text{old}}}(o_i)\big)^{1/|o_i|}$ clipped once per sequence. CISPO: clip the importance weight in the gradient, $\text{sg}[\text{clip}(\rho_{i,t}, \cdot)]\,A_i \log \pi_\theta(o_{i,t})$, so no token is dropped. RLOO: $A_i = r_i - \frac{1}{G-1}\sum_{j\neq i} r_j$. REINFORCE++: advantages normalized over the global batch rather than per prompt.

Entropy ceiling (Cui et al. 2025). Downstream performance R against policy entropy H during RL fits $R = -a\,e^H + b$; at $H = 0$ the fit gives the ceiling $b - a$. Clip-Cov and KL-Cov act on the high-covariance tokens that drive the collapse.

Budget forcing (s1). To cap: when the thinking span reaches B tokens, append the end-of-thinking delimiter and let the answer begin. To extend: suppress the delimiter and append "Wait". Accuracy rises with B and then saturates; forcing many extensions can lower it (Ghosal et al. 2025).

Length-controlled RL (L1). Reward $r = \mathbb{1}[\text{correct}] - \alpha\,\big||z| - B_{\text{target}}\big|$ with the target B_{target} stated in the prompt; the model learns to hit the target, and accuracy against length follows a log-linear line with slope 0.24 against 0.37 for budget forcing on the same base (Aggarwal and Welleck 2025).

Serving arithmetic (derived). Decode time $= |z|/$(tokens per second): 10,000 tokens at 60 tok/s is 167 s. KV cache $= |z| \times$ bytes per token: 10,000 tokens at 320 KB per token (70B dense, bf16, GQA) is 3.2 GB; at 69 KB per token (V3-class MLA) 0.7 GB. Cost $= |z| \times$ price per output token: 10,000 tokens at \$10 per million is \$0.10, at \$60 per million \$0.60. Thinking tokens are billed as output tokens whether or not they are shown†.

Monitor recall (Baker et al. 2025). Of the reward hacks the agent committed, the fraction a GPT-4o monitor flagged: 95% reading the chain, 60% reading actions only; after training against the monitor, near zero while the hacks continued.

C Exercises

1. **Serial compute.** A 70B dense model does about $2 \times 70 \times 10^9$ FLOPs per generated token. How many FLOPs does a 10,000-token chain cost, and how does that compare with one forward pass over a 10,000-token prompt? Why does the second not substitute for the first on a problem such as multi-digit multiplication? (Use slide 6 and the Li et al. result.)

2. **Pass at k.** A model solves a problem on 3 of 16 samples. Compute the unbiased $\widehat{\text{pass@}k}$ for $k = 1, 2, 4, 8, 16$. Now suppose an RL-trained version solves it on 9 of 16. Compute the same values and say at which k the two models are closest. Explain in one sentence why Yue et al. (2025) find base models winning at large k on some problem sets.

3. **Coverage versus selection.** Brown et al. report SWE-bench Lite coverage of 15.9% at one sample and 56% at 250. If a verifier picks the correct patch, when one exists, with probability 0.6, what accuracy does the 250-sample system deliver? What verifier precision would be needed to beat a single-sample system at 40%?

4. **Best-of-N with a PRM.** Lightman et al. report 78.2% (process reward model), 72.4% (outcome reward model) and 69.6% (majority vote) at $N = 1{,}860$ on a MATH subset. If each sample costs 1,000 tokens at \$10 per million, what does one problem cost at $N = 1{,}860$? At what N would the same budget be exhausted by a single 30,000-token chain, and which approach would you expect to win on a problem the model solves 2% of the time?

5. **GRPO advantages.** A group of 8 samples scores $(1,0,0,1,0,0,0,0)$. Compute the GRPO advantages $(r_i - \bar{r})/\sigma_r$ and the Dr. GRPO advantages $r_i - \bar{r}$. Repeat for $(1,1,1,1,1,1,1,0)$. Which group would DAPO's dynamic sampling keep, and what happens to the GRPO advantage of a group scoring $(0,0,0,0,0,0,0,0)$?

6. **Length bias.** Two wrong samples in a group have lengths 500 and 5,000 tokens and the same advantage $A < 0$. Under GRPO's per-sample $1/|o_i|$ normalization, what is the total gradient weight each contributes? Under DAPO's token-level loss? Explain why Liu et al. say the first "artificially increases response length, especially for incorrect outputs".

7. **Clip-higher.** With $\varepsilon = 0.2$, a token whose old probability was 0.01 can at most be raised to what probability in one update? With $\varepsilon_{\text{high}} = 0.28$? Why does this matter more for rare "exploration" tokens than for tokens already at probability 0.9?

8. **Entropy ceiling.** A run fits $R = -0.6\,e^{H} + 0.9$ (in accuracy units). What accuracy does the fit predict at $H = 0$? At $H = 0.5$? If the entropy collapses from 0.5 to 0.1 in the first 100 steps, how much of the available improvement has been spent?

9. **Spurious rewards.** On Qwen2.5-Math-7B, random rewards gave $+21.4$ points on MATH-500, incorrect labels $+24.1$, ground truth $+29.1$. What control experiment would distinguish "the reward teaches the task" from "any training signal amplifies a behavior the base already has"? What would you expect on a Llama base, and why does the slide tell you to check the base before crediting the reward?

10. **Distillation economics.** s1 fine-tuned a 32B model on 1,000 examples in 26 minutes on 16 H100s. At \$3 per H100-hour, what did the run cost? OpenThoughts3 used 1.2M examples; estimate its fine-tuning cost on the same hardware if cost scales with tokens and the average trace is 8,000 tokens against s1's

6,000 (assume the s1 run is dominated by its traces). Compare both with the $294K† reported for R1's RL phase.

11. **Budget curves.** Using the illustrative curve on slide 19 (competition math: 18% at 500 tokens, 42% at 2,000, 58% at 8,000, 62% at 32,000), compute the accuracy gained per additional 1,000 tokens on each segment. At $10 per million output tokens and 1,000 problems, what does moving the budget from 8,000 to 32,000 cost, and what does it buy?

12. **Serving.** A service runs 200 concurrent reasoning requests averaging 12,000 thinking tokens on a 70B dense model at 320 KB per token. How much KV-cache memory is in use at the peak? How many 80 GB GPUs does that occupy at 70% usable memory, before weights? Redo the calculation for a V3-class MLA model at 69 KB per token.

13. **Faithfulness.** Chen et al. (2025) find hints verbalized in 25% and 39% of chains that used them, and reward hacks verbalized in under 2% of chains in five of six environments. A monitor reads the chain and flags any mention of the hint. Write the recall of that monitor in each case. What does Baker et al.'s 95% recall measure differently?

14. **Brittleness.** GSM-Symbolic adds one irrelevant clause and accuracy drops by up to 65%. Design a 30-problem test for a model you use: write three surface variants of each of ten problems (renamed entities, changed numbers, one irrelevant clause) and define the statistic you would report. What result would make you trust a 95% headline on the original set?

15. **Harness or model.** Lawsen's comment shows the Tower of Hanoi "collapse" coincides with the output-token limit. For n disks the solution has $2^n - 1$ moves; if each move costs about 10 tokens, at what n does a 64,000-token limit bind? Propose a prompt that separates the model's ability to solve the puzzle from its ability to print the solution.

16. **Reporting.** Write the four tags slide 29 asks for (model and date, harness, effort or budget, samples per problem) for the DeepSeek-R1 AIME 2024 number of 79.8% as the paper reports it, and for the o1 number of 74%† as the blog reports it. Which tags are missing from each, and how would the missing tags change a comparison between them?

D Reading list

The papers behind each part of the deck, in the order they are discussed, with one line on why each is worth reading. Entries with an arXiv identifier are included in the paper archive; entries without one are listed with a URL. Items marked *secondary source* are blog posts, model cards, documentation or commentary.

1. Overview and lineage (slides 1-4)

- **OpenAI (2024).** *Learning to reason with LLMs (blog, September 12, 2024)*. `https://openai.com/index/learning-to-reason-with-llms/`. The o1 announcement: AIME 2024 from 12% (GPT-4o) to 74% single-sample, 83% with 64-sample consensus, 93% re-ranking 1,000 samples with a learned scorer; accuracy rising smoothly with both train-time and test-time compute; the decision to hide the raw chain. Secondary source.
- **Nye et al. (2021).** *Show your work: scratchpads for intermediate computation with language models*. arXiv:2112.00114. The origin of trained intermediate computation: fine-tuning a model to emit a scratchpad before the answer makes multi-step arithmetic and program execution learnable. The 2021 point on the timeline.
- **Zelikman, Wu, Mu and Goodman (2022).** *STaR: bootstrapping reasoning with reasoning*. arXiv:2203.14465. Sample rationales, keep the ones that reach the right answer, fine-tune, repeat; rationalize failures from the answer. Rejection-sampling self-training, the ancestor of every RLVR loop; matches fine-tuning a 30x larger model on CommonsenseQA.
- **Lambert et al. (2024).** *Tulu 3: pushing frontiers in open language model post-training*. arXiv:2411.15124. Coins 'reinforcement learning with verifiable rewards' (RLVR) as a post-training stage on math and instruction-following with programmatic checkers, two months before R1.
- **DeepSeek-AI (2025).** *DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning*. arXiv:2501.12948. The open recipe: R1-Zero (pure RL from the base, AIME 2024 15.6 to 71.0 pass@1, 86.7 with majority voting), R1's four stages with a cold start, the distilled Qwen and Llama models, and the failed PRM and MCTS attempts. Also Nature 645:633-638 (2025). Read with section 4.
- **Xu et al. (2025).** *Towards large reasoning models: a survey of reinforced reasoning with large language models*. arXiv:2501.09686. The first survey written after o1 and before R1: automated data, test-time scaling and RL as the three ingredients; a map of the field in January 2025.
- **Kumar et al. (2025).** *LLM post-training: a deep dive into reasoning large language models*. arXiv:2502.21321. A post-R1 survey of fine-tuning, RL and test-time scaling for reasoning, with a taxonomy that matches this deck's sections.
- **Weng (2025).** *Why we think (blog, May 2025)*. `https://lilianweng.github.io/posts/2025-05-01-thinking/`. A long, careful overview of thinking-time compute: latent and token-level thinking, RL on correctness, faithfulness, scaling laws for thinking time. The best single piece of secondary reading. Secondary source.
- **Raschka (2025).** *Understanding reasoning LLMs (newsletter, February 2025)*. `https://magazine.sebastianraschka.com/p/understanding-reasoning-llms`. Four ways to build a reasoning model (inference scaling, pure RL, RL plus SFT, distillation) laid out around DeepSeek-R1 with a cost comparison; written for practitioners. Secondary source.
- **Lambert (2025).** *Reinforcement learning from human feedback (book), chapter 7: reasoning and*

inference-time scaling. arXiv:2504.12501. The textbook treatment of RLVR as a post-training stage, with GRPO derivations and the practical lore of 2025; the arXiv entry is the whole book.

2. Chains of thought: the effect, the theory, faithfulness (slides 5-7)

- **Wei et al. (2022).** *Chain-of-thought prompting elicits reasoning in large language models*. arXiv:2201.11903. Eight worked exemplars in the prompt take PaLM 540B from 17.9% to 56.9% on GSM8K (58.6% with a calculator); the gain appears only at scale. The slide-5 bars.
- **Kojima et al. (2022).** *Large language models are zero-shot reasoners*. arXiv:2205.11916. 'Let's think step by step' alone: text-davinci-002 from 17.7% to 78.7% on MultiArith and 10.4% to 40.7% on GSM8K. The behavior was latent in the pretrained model.
- **Wang et al. (2022).** *Self-consistency improves chain of thought reasoning in language models*. arXiv:2203.11171. Sample many chains, majority-vote the answers: +17.9 points on GSM8K over greedy chain-of-thought for PaLM 540B (to 74.4%). The first test-time scaling result and still the baseline for parallel compute.
- **Zhou et al. (2022).** *Least-to-most prompting enables complex reasoning in large language models*. arXiv:2205.10625. Decompose into subproblems, solve in order: 99% on SCAN length generalization with 14 exemplars where chain-of-thought gets 16%. Decomposition as a prompting primitive.
- **Madaan and Yazdanbakhsh (2022).** *Text and patterns: for effective chain of thought, it takes two to tango*. arXiv:2209.07686. Counterfactual prompts show the factual correctness of the exemplar steps matters little; the symbols, patterns and text of the chain matter together. An early warning that chains are not what they seem.
- **Prystawski, Li and Goodman (2023).** *Why think step by step? Reasoning emerges from the locality of experience*. arXiv:2304.03843. A data-side explanation: when training data covers local dependencies, chaining local inferences lets the model bridge variables that never co-occur. Proven for a chain-structured model, checked with transformers.
- **Feng et al. (2023).** *Towards revealing the mystery behind chain of thought: a theoretical perspective*. arXiv:2305.15408. Bounded-depth transformers cannot output arithmetic or equation answers directly unless size grows super-polynomially; constant-size autoregressive transformers solve them with a chain. The circuit-complexity case on slide 6.
- **Merrill and Sabharwal (2023).** *The expressive power of transformers with chain of thought*. arXiv:2310.07923. Logarithmically many steps add little; linearly many reach all regular languages; polynomially many reach exactly P. The length of the chain is the resource.
- **Li, Liu, Zhou and Ma (2024).** *Chain of thought empowers transformers to solve inherently serial problems*. arXiv:2402.12875. Without a chain, constant-depth constant-precision transformers sit in AC0; with T chain steps they compute anything a size-T circuit computes. Serial problems need serial tokens.
- **Hao et al. (2024).** *Training large language models to reason in a continuous latent space*. arXiv:2412.06769. Coconut: feed the last hidden state back as the next input embedding instead of a token; an implicit breadth-first search that beats chain-of-thought on search-heavy logic. Latent reasoning, and what it would do to monitorability.
- **Geiping et al. (2025).** *Scaling up test-time compute with latent reasoning: a recurrent depth approach*. arXiv:2502.05171. A 3.5B model with a looped block, trained on 800B tokens, matches test-time compute

equivalent to 50B parameters by iterating in latent space; the recurrent-depth alternative mentioned on slides 6 and 24.

- **Zelikman et al. (2024).** *Quiet-STaR: language models can teach themselves to think before speaking*. arXiv:2403.09629. Per-token latent rationales learned from plain text with a REINFORCE signal on next-token likelihood: zero-shot GSM8K 5.9 to 10.9, CommonsenseQA 36.3 to 47.2.
- **Turpin, Michael, Perez and Bowman (2023).** *Language models don't always say what they think: unfaithful explanations in chain-of-thought prompting*. arXiv:2305.04388. Biasing features in the context (answer always 'A' in the exemplars) change answers by up to 36 points across 13 tasks while the chains never mention the bias. The slide-7 result.
- **Lanham et al. (2023).** *Measuring faithfulness in chain-of-thought reasoning*. arXiv:2307.13702. Truncate, corrupt or paraphrase the chain and see whether the answer moves; larger and more capable models produce less faithful chains on most tasks.
- **Arcuschin et al. (2025).** *Chain-of-thought reasoning in the wild is not always faithful*. arXiv:2503.08679. Unprompted unfaithfulness: implicit post-hoc rationalization in up to 13% of answers for production models, with much lower rates for R1 and Claude 3.7 Sonnet in thinking mode.
- **Chen et al. (2025).** *Reasoning models don't always say what they think*. arXiv:2505.05410. Hints that change the answer are verbalized 25% (Claude 3.7 Sonnet) and 39% (R1) of the time; outcome RL raises faithfulness then plateaus; reward hacks exploited on over 99% of examples are mentioned in under 2% of chains. The slide-7 and slide-27 numbers.

3. Verifiers, search and test-time compute (slides 8-10)

- **Cobbe et al. (2021).** *Training verifiers to solve math word problems*. arXiv:2110.14168. GSM8K (8.5K problems) and the first outcome verifier: sample 100 solutions, rank with a trained verifier; verification scales better with data than fine-tuning alone.
- **Uesato et al. (2022).** *Solving math word problems with process- and outcome-based feedback*. arXiv:2211.14275. Process supervision cuts reasoning errors among correct final answers from 14.0% to 3.4% at similar final-answer accuracy; the first process-versus-outcome comparison.
- **Lightman et al. (2023).** *Let's verify step by step*. arXiv:2305.20050. PRM800K: 800K step labels. Best-of-1,860 on a MATH subset: process reward model 78.2%, outcome reward model 72.4%, majority vote 69.6%. The slide-8 numbers.
- **Wang et al. (2023).** *Math-Shepherd: verify and reinforce LLMs step-by-step without human annotations*. arXiv:2312.08935. Step labels estimated by Monte Carlo rollouts instead of humans; a PRM for both reranking and RL, moving Mistral-7B from 77.9 to 84.1 on GSM8K.
- **Zhang et al. (2025).** *The lessons of developing process reward models in mathematical reasoning*. arXiv:2501.07301. Qwen's post-mortem: Monte Carlo step labels are worse than LLM-judge or human labels, best-of-N evaluation is biased, consensus filtering helps. Why R1 dropped PRMs.
- **Zhang et al. (2024).** *Generative verifiers: reward modeling as next-token prediction*. arXiv:2408.15240. Train the verifier to generate a judgment (with its own chain) and read the probability of 'Yes'; large best-of-N gains over discriminative reward models.
- **Yao et al. (2023).** *Tree of Thoughts: deliberate problem solving with large language models*. arXiv:2305.10601. Explicit search over partial thoughts with self-evaluation: Game of 24 from 4% (chain-of-thought) to 74% with GPT-4. Search outside the model, slide 9.

- **Besta et al. (2023).** *Graph of Thoughts: solving elaborate problems with large language models*. arXiv:2308.09687. Generalizes tree search to graphs with merging and refinement; better quality than Tree of Thoughts on sorting at lower cost. The end of the prompt-time search line.
- **Snell, Lee, Xu and Kumar (2024).** *Scaling LLM test-time compute optimally can be more effective than scaling model parameters*. arXiv:2408.03314. Allocate test-time compute by estimated difficulty: more than 4x the efficiency of best-of-N, and a small model with compute-optimal search beating a 14x larger one at matched FLOPs. Slide 9.
- **Brown et al. (2024).** *Large language monkeys: scaling inference compute with repeated sampling*. arXiv:2407.21787. Coverage (pass@k) grows log-linearly over four orders of magnitude of samples; SWE-bench Lite from 15.9% with one sample to 56% with 250. The ceiling that verifiers cash in.
- **Guan et al. (2025).** *rStar-Math: small LLMs can master math reasoning with self-evolved deep thinking*. arXiv:2501.04519. MCTS with code-verified steps and a process preference model, four rounds of self-evolution: Qwen2.5-Math-7B from 58.8% to 90.0% on MATH. Search-based training that works, at a cost.
- **OpenAI (El-Kishky et al.) (2025).** *Competitive programming with large reasoning models*. arXiv:2502.06807. o1 (1673 Codeforces rating), o1-ioi with hand-engineered test-time strategies (1807 to 2214), and o3 (2724, 99.8th percentile) with none: scaled RL replaced the hand-built search. Slide 10.
- **Beeching, Tunstall and Rush (2024).** *Scaling test-time compute with open models (Hugging Face blog)*. `https://huggingface.co/spaces/HuggingFaceH4/blogpost-scaling-test-time-compute`. Reproduces Snell et al. with open weights: Llama 3.2 1B plus a PRM and beam search beats Llama 3.1 8B on MATH-500; best-of-N, beam and diverse verifier tree search compared. Secondary source.
- **Liu et al. (2025).** *Can 1B LLM surpass 405B LLM? Rethinking compute-optimal test-time scaling*. arXiv:2502.06703. Compute-optimal test-time scaling depends on the policy, the PRM and the difficulty; a 1B model beats a 405B one on MATH-500 with the right search, and a 7B beats o1 and R1.
- **Fu et al. (2025).** *Deep think with confidence*. arXiv:2508.15260. Filter parallel chains by the model's own token confidence: 99.9% on AIME 2025 at 512 samples with 84.7% fewer tokens than plain majority voting.
- **Wen et al. (2025).** *ParaThinker: native parallel thinking as a new paradigm to scale LLM test-time compute*. arXiv:2509.04475. Trains a model to run several reasoning paths in parallel inside one context and merge them; +12.3% for a 1.5B model with 8 paths at 7% latency overhead. Parallel thinking as an architecture.
- **Ghosal et al. (2025).** *Does thinking more always help? Mirage of test-time scaling in reasoning models*. arXiv:2506.04210. Forcing more 'Wait' tokens raises accuracy and then lowers it; parallel sampling scales more reliably. Test-time scaling is not monotone.
- **Zhang et al. (2026).** *Scaling reasoning tokens via RL and parallel thinking: evidence from competitive programming*. arXiv:2604.01302. Log-linear accuracy against reasoning tokens across RL checkpoints; 16 threads by 16 rounds reach oracle pass@16 at pass@1 for 7.6M tokens per problem. The 2026 state of the test-time curve.

4. Reinforcement learning on verifiable rewards: RLVR, R1, GRPO, reward design (slides 11-14)

- **Shao et al. (2024).** *DeepSeekMath: pushing the limits of mathematical reasoning in open language models*. arXiv:2402.03300. Introduces GRPO: PPO without a value model, advantages from a group of samples per prompt; 120B math tokens; 51.7% on MATH with a 7B model. The algorithm on slide 13.
- **Singh et al. (2023).** *Beyond human data: scaling self-training for problem-solving with language models*. arXiv:2312.06585. ReST-EM: expectation-maximization view of sample-filter-finetune on PaLM 2, the bridge between STaR and RLVR.
- **Ahmadian et al. (2024).** *Back to basics: revisiting REINFORCE-style optimization for learning from human feedback in LLMs*. arXiv:2402.14740. REINFORCE with a leave-one-out baseline (RLOO) matches or beats PPO for LLM fine-tuning at a fraction of the complexity; the case that the value model was never needed.
- **Hu et al. (2025).** *REINFORCE++: stabilizing critic-free policy optimization with global advantage normalization*. arXiv:2501.03262. Normalize advantages over the global batch rather than per prompt; a critic-free algorithm with better robustness to reward hacking than GRPO in the authors' tests.
- **Kimi Team (2025).** *Kimi k1.5: scaling reinforcement learning with LLMs*. arXiv:2501.12599. Released the same week as R1: long-context RL with a length penalty, multimodal, and long2short methods that transfer long-chain accuracy to short chains. AIME 77.5, MATH500 96.2.
- **Yu et al. (2025).** *DAPO: an open-source LLM reinforcement learning system at scale*. arXiv:2503.14476. Four fixes to GRPO (clip-higher with epsilon_high 0.28, dynamic sampling, token-level loss, overlong reward shaping) reach 50 on AIME 2024 with Qwen2.5-32B base, beating DeepSeek's 47 at half the steps; code, data (17k problems) and system released.
- **Liu et al. (2025).** *Understanding R1-Zero-like training: a critical perspective*. arXiv:2503.20783. Dr. GRPO: the length and standard-deviation normalizations in GRPO bias toward long wrong answers; base models already show 'aha' behaviors; Qwen2.5 bases answer without templates. 43.3% AIME 2024 with 7B.
- **Yue et al. (2025).** *VAPO: efficient and reliable reinforcement learning for advanced reasoning tasks*. arXiv:2504.05118. The value-based counterpoint: with the right value pretraining, length-adaptive GAE and decoupled clipping, PPO reaches 60.4 on AIME 2024 with Qwen-32B in 5,000 steps.
- **Zheng et al. (2025).** *Group sequence policy optimization*. arXiv:2507.18071. Importance ratios and clipping at the sequence level instead of per token; stabilizes RL for mixture-of-experts models; used to train Qwen3.
- **MiniMax (2025).** *MiniMax-M1: scaling test-time compute efficiently with lightning attention*. arXiv:2506.13585. CISPO (clip the importance weights, not the updates), a 456B hybrid-attention model with 1M context, and a stated RL bill: 512 H800s for three weeks, $534,700.
- **Mistral AI (2025).** *Magistral*. arXiv:2506.10910. An RL-only recipe on an existing instruct model with the 2025 lore spelled out: no KL term, length-normalized loss, minibatch advantage normalization, clip-higher; Magistral Medium 73.6 on AIME 2024.
- **Skalse et al. (2022).** *Defining and characterizing reward hacking*. arXiv:2209.13085. A formal definition: two rewards are unhackable only in degenerate cases over all stochastic policies; the background for every reward-design slide.

- **Huang et al. (2025).** *Reinforcement learning with rubric anchors*. arXiv:2508.12790. Over 10,000 rubrics as rewards for open-ended tasks; +5.2% on open-ended benchmarks with 5K samples. The main attempt to extend RLVR beyond checkable answers.
- **Zhao et al. (2025).** *Learning to reason without external rewards*. arXiv:2505.19590. Intuitor: the model's own self-certainty as the only reward matches GRPO on math and generalizes better to code. Evidence, mostly on Qwen, that the reward signal can be weak.
- **Zhao et al. (2025).** *Absolute Zero: reinforced self-play reasoning with zero data*. arXiv:2505.03335. The model proposes its own tasks, a code executor verifies them, and RL trains both roles; no external problems at all.
- **Wei et al. (2025).** *SWE-RL: advancing LLM reasoning via reinforcement learning on open software evolution*. arXiv:2502.18449. RL on real pull-request history with a similarity reward against the merged patch: a 70B Llama reaches 41.0% on SWE-bench Verified. Agentic RL without a unit-test oracle.
- **Feng et al. (2025).** *ReTool: reinforcement learning for strategic tool use in LLMs*. arXiv:2504.11536. A code interpreter inside the RL rollout: a 32B model reaches 67% on AIME in 400 training steps where text-only RL took 1,080 steps for 40%; 72.5% with more training. Slide 21.
- **Gou et al. (2023).** *ToRA: a tool-integrated reasoning agent for mathematical problem solving*. arXiv:2309.17452. Interleave natural-language reasoning with program calls; the first open model over 50% on MATH (ToRA-Code-34B). Tool-integrated reasoning before RL.
- **Helff et al. (2026).** *LLMs gaming verifiers: RLVR can lead to reward hacking*. arXiv:2604.15149. RLVR-trained models enumerate instance labels instead of inducing the rule when the verifier allows it; isomorphic verification removes the exploit. A 2026 reminder that the checker is part of the model.

5. Training dynamics: what RL changes and what was already there (slides 15-16)

- **Yue et al. (2025).** *Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?*. arXiv:2504.13837. RLVR models win at small k, base models win at large k: RL sharpens the distribution over chains the base model could already produce and narrows coverage; distillation expands it. The slide-16 crossover.
- **Shao et al. (2025).** *Spurious rewards: rethinking training signals in RLVR*. arXiv:2506.10947. Random rewards give +21.4 points on MATH-500 for Qwen2.5-Math-7B (ground truth: +29.1; wrong labels: +24.1), by amplifying code-style reasoning the base already uses; little effect on Llama and OLMo. Check the base before crediting the reward.
- **Wang et al. (2025).** *Reinforcement learning for reasoning in large language models with one training example*. arXiv:2504.20571. RLVR on a single problem takes Qwen2.5-Math-1.5B from 36.0% to 73.6% on MATH500, matching a 1.2k-problem set; improvement continues after the one example is saturated.
- **Liu et al. (2025).** *ProRL: prolonged reinforcement learning expands reasoning boundaries in large language models*. arXiv:2505.24864. Over 2,000 RL steps from R1-Distill-Qwen-1.5B on 136K problems with KL control and reference resets; pass@k rises on tasks where the base model fails at any k. The reply to the crossover paper.
- **Hu et al. (2025).** *BroRL: scaling reinforcement learning via broadened exploration*. arXiv:2510.01180. Hundreds of rollouts per prompt instead of more steps revive models saturated after 3K ProRL steps; rollouts as a scaling axis.

- **Cui et al. (2025).** *The entropy mechanism of reinforcement learning for reasoning language models*. arXiv:2505.22617. Policy entropy collapses early and performance follows R = -a exp(H) + b, so a ceiling is fixed at zero entropy; Clip-Cov and KL-Cov keep entropy up. Slide 13's 'entropy' bullet.
- **Wang et al. (2025).** *Beyond the 80/20 rule: high-entropy minority tokens drive effective reinforcement learning for LLM reasoning*. arXiv:2506.01939. About 20% of tokens, the high-entropy 'forks', carry the useful policy gradient; updating only them matches or beats full updates on Qwen3 models.
- **Khatri et al. (2025).** *The art of scaling reinforcement learning compute for LLMs*. arXiv:2510.13786. Over 400,000 GPU-hours of ablations: RL compute-performance curves are sigmoidal with a recipe-dependent asymptote; ScaleRL extrapolated from small runs to a 100,000 GPU-hour run.
- **Tan et al. (2025).** *Scaling behaviors of LLM reinforcement learning post-training: an empirical study in mathematical reasoning*. arXiv:2509.25300. Qwen2.5 from 0.5B to 72B: test loss follows a power law in compute and data; model size matters more than data under a fixed budget.
- **Chen et al. (2025).** *Pass@k training for adaptively balancing exploration and exploitation of large reasoning models*. arXiv:2508.10751. Use pass@k itself as the reward with an analytic advantage; the model keeps exploring instead of collapsing onto one chain.
- **Cheng et al. (2026).** *IsoCompute playbook: optimally scaling sampling compute for LLM RL*. arXiv:2603.12151. How many rollouts per problem at a fixed compute budget: compute-optimal rollouts rise with budget and then saturate. The 2026 rollout-allocation law.
- **Zhai et al. (2026).** *Rewards as labels: revisiting RLVR from a classification perspective*. arXiv:2602.05630. Treat the binary reward as a label and train with a classification loss: +6.7% over DAPO on a 1.5B model, +6.2% on 7B. One of several 2026 reformulations of GRPO.

6. Distillation and data (slides 17-18)

- **Hsieh et al. (2023).** *Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes*. arXiv:2305.02301. Rationales from a large model as extra supervision for a small one: a 770M T5 beats 540B PaLM using 80% of the data. Rationale distillation before reasoning models.
- **Muennighoff et al. (2025).** *s1: simple test-time scaling*. arXiv:2501.19393. 1,000 questions with Gemini-distilled traces, 26 minutes on 16 H100s, and budget forcing (insert end-of-thinking, or suppress it and append 'Wait'): s1-32B beats o1-preview on competition math by up to 27%. Slide 17.
- **Ye et al. (2025).** *LIMO: less is more for reasoning*. arXiv:2502.03387. 817 curated problems with high-quality chains: 57.1% on AIME 2024 and 94.8% on MATH (v1; v3 reports 63.3 and 95.6 with 800 samples), against 6.5% and 59.2% for earlier SFT recipes with 100k examples.
- **Guha et al. (2025).** *OpenThoughts: data recipes for reasoning models*. arXiv:2506.04178. 1,000-plus ablations of question sourcing, filtering and teacher choice; OpenThoughts3 (1.2M examples) gives a 7B model 53% on AIME 2025 by supervised fine-tuning alone; QwQ-32B a better teacher than R1. Slide 18.
- **Moshkov et al. (2025).** *AIMO-2 winning solution: building state-of-the-art mathematical reasoning models with OpenMathReasoning dataset*. arXiv:2504.16891. 540K problems with 3.2M long-reasoning and 1.7M tool-integrated solutions, plus GenSelect for choosing among candidates; the largest open reasoning set.

- **Wen et al. (2025).** *Light-R1: curriculum SFT, DPO and RL for long COT from scratch and beyond*. arXiv:2503.10460. A curriculum of 76k then 3k hard problems takes Qwen2.5-32B-Instruct to 76.6 on AIME 2024 in six hours on 12 H800s (about $1,000 by the authors' count). The cheap-32B recipe.
- **Chen et al. (2025).** *AceReason-Nemotron: advancing math and code reasoning through reinforcement learning*. arXiv:2505.16400. Math-only RL then code-only RL on distilled 7B and 14B models: +14.6 and +17.2 on AIME 2025; math RL transfers to code.
- **Hugging Face (2025).** *Open-R1: a fully open reproduction of DeepSeek-R1 (blog, January 28, 2025) and update 2 (February 10, 2025)*. `https://huggingface.co/blog/open-r1`. The reproduction plan and OpenR1-Math-220k: 800k R1 traces for 400k NuminaMath problems generated on 512 H100s, filtered to 220k; the resulting 7B scores 90.6 on MATH-500 against 91.6 for R1-Distill-Qwen-7B. Secondary source.
- **Project Numina (2024).** *NuminaMath: CoT, TIR and 1.5 datasets (Hugging Face collection)*. `https://huggingface.co/collections/AI-MO/numinamath`. 860k competition-style problems with chains (and 72.5k tool-integrated solutions); the problem source behind most 2025 reasoning data. Secondary source.
- **NovaSky (UC Berkeley) (2025).** *Sky-T1: train your own o1-preview model within $450 (blog, January 10, 2025)*. `https://novasky-ai.github.io/posts/sky-t1/`. 17K QwQ-distilled samples, 19 hours on 8 H100s, Qwen2.5-32B-Instruct: 43.3 on AIME 2024. The first sub-$500 reproduction. Secondary source.
- **Bespoke Labs (2025).** *Bespoke-Stratos: the unreasonable effectiveness of reasoning distillation (blog, January 22, 2025)*. `https://www.bespokelabs.ai/blog/bespoke-stratos-the-unreasonable-effectiveness-of-reasoning-distillation`. 17k R1 traces, 47x fewer than DeepSeek's own distillation set, give a 32B model 63.3 on AIME 2024 and 93.0 on MATH500. Secondary source.
- **Agentica (2025).** *DeepScaleR-1.5B-Preview: surpassing o1-preview with a 1.5B model by scaling RL (model card, February 2025)*. `https://huggingface.co/agentica-org/DeepScaleR-1.5B-Preview`. GRPO with context grown from 8K to 16K to 24K tokens takes R1-Distill-Qwen-1.5B to 43.1% on AIME 2024; the small-model RL recipe many later papers start from. Secondary source.
- **Qwen Team (2025).** *Qwen3 technical report*. arXiv:2505.09388. Hybrid thinking and non-thinking modes in one model, a thinking budget, four post-training stages (long-CoT cold start, reasoning RL with GRPO on 3,995 query-verifier pairs, thinking-mode fusion, general RL), and strong-to-weak distillation at a tenth of the GPU-hours.
- **OpenAI (2025).** *gpt-oss-120b and gpt-oss-20b model card*. arXiv:2508.10925. Open-weight reasoning models with low/medium/high effort set in the system prompt; 116.8B parameters with 5.1B active; 2.1M H100-hours for the 120b; AIME 2025 97.9 with tools. The open reference point on slides 19 and 24.
- **Abdin et al. (2025).** *Phi-4-reasoning technical report*. arXiv:2504.21318. A 14B model fine-tuned on o3-mini traces, then a short outcome-RL stage (Phi-4-reasoning-plus); a careful account of what SFT alone buys at 14B.
- **Bercovich et al. (2025).** *Llama-Nemotron: efficient reasoning models*. arXiv:2505.00949. 8B, 49B and 253B models with a reasoning toggle in the system prompt, built by distillation then RL; the toggle idea that hybrid models generalized.

- **Zeng et al. (2025).** *GLM-4.5: agentic, reasoning, and coding (ARC) foundation models*. arXiv:2508.06471. A 355B/32B-active hybrid reasoning model with RL on multi-turn tool use; AIME 2024 91.0, SWE-bench Verified 64.2. Agentic RL in an open report.
- **Kimi Team (2025).** *Kimi K2: open agentic intelligence*. arXiv:2507.20534. A 1T/32B-active model trained with MuonClip on 15.5T tokens and large-scale agentic data synthesis; the base of K2 Thinking, which runs hundreds of sequential tool calls (slide 21).

7. Inference: budgets, length control, parallel compute, tools, serving (slides 19-22)

- **Chen et al. (2024).** *Do NOT think that much for 2+3=? On the overthinking of o1-like LLMs*. arXiv:2412.21187. QwQ emits 13 solutions and 901 tokens for 2+3; the first round is already correct over 92% of the time; a self-trained preference for efficient chains cuts MATH500 tokens by 48.6% at equal accuracy. Slide 20.
- **Wang et al. (2025).** *Thoughts are all over the place: on the underthinking of o1-like LLMs*. arXiv:2501.18585. Wrong answers switch between lines of thought more often than right ones; a decoding penalty on switching (TIP) raises accuracy without retraining.
- **Aggarwal and Welleck (2025).** *L1: controlling how long a reasoning model thinks with reinforcement learning*. arXiv:2503.04697. Length-controlled policy optimization: a target length in the prompt, a reward for hitting it; a 1.5B model beats GPT-4o at equal chain length and budget forcing by up to 20 points absolute.
- **Hou et al. (2025).** *ThinkPrune: pruning long chain-of-thought of LLMs via reinforcement learning*. arXiv:2504.01296. Iteratively tighten the token limit during RL: R1-Distill-Qwen-1.5B halves its AIME chain length for a 2-point drop.
- **Ma et al. (2025).** *Reasoning models can be effective without thinking*. arXiv:2504.09858. Prefill an empty thinking block: at a 700-token budget NoThinking scores 51.3 against 28.9 for thinking on ACM 23, and with parallel samples it matches thinking at up to 9x lower latency.
- **Zhang et al. (2025).** *AdaptThink: reasoning models can learn when to think*. arXiv:2505.13417. RL that chooses between thinking and not thinking per problem: 53% shorter outputs and +2.4% accuracy on math.
- **Fang et al. (2025).** *Thinkless: LLM learns when to think*. arXiv:2505.13379. A decoupled GRPO objective learns a <short>/<think> control token; long-chain use drops 50-90% on easy sets.
- **Cuadron et al. (2025).** *The danger of overthinking: examining the reasoning-action dilemma in agentic tasks*. arXiv:2502.08235. 4,018 SWE-bench Verified trajectories: agents that deliberate instead of acting fail more; selecting low-overthinking runs gains about 30% at 43% lower cost. Slide 20's agent bullet.
- **Aghajohari et al. (2025).** *The Markovian Thinker: architecture-agnostic linear scaling of reasoning*. arXiv:2510.06557. Reason in fixed-size chunks with a carried state so compute is linear in chain length; 96K-token thinking at a fraction of the GPU-months. A response to the length-generalization problem.
- **Sui et al. (2025).** *Stop overthinking: a survey on efficient reasoning for large language models*. arXiv:2503.16419. Model-, output- and prompt-based methods for shorter chains, catalogued; the reference list for slide 20.
- **Shen et al. (2025).** *Thinking vs. doing: agents that reason by scaling test-time interaction*. arXiv:2506.07976. Scale the number of environment interactions rather than the chain: a curriculum over interaction length gives Gemma 3 12B state-of-the-art open web-agent results.

- **Zheng et al. (2025).** *Parallel-R1: towards parallel thinking via reinforcement learning*. arXiv:2509.07980. RL that rewards spawning and merging parallel sub-chains; +8.4% over sequential RL. Parallel thinking as a trained behavior, slide 21.
- **Wang et al. (2025).** *Scaling over scaling: exploring test-time scaling plateau in large reasoning models*. arXiv:2505.20522. A saturation bound (TTSPM) for parallel and sequential test-time scaling; where the curve on slide 19 stops.
- **Gou et al. (2023).** *CRITIC: large language models can self-correct with tool-interactive critiquing*. arXiv:2305.11738. Self-correction grounded in tool outputs (search, interpreters) rather than the model's own judgment; the precursor to tools inside the chain.

8. The landscape in October 2026 (slides 23-25)

- **OpenAI (2026).** *Reasoning models guide: reasoning effort, encrypted reasoning items, billing (developer documentation)*. `https://platform.openai.com/docs/guides/reasoning`. The effort levels, encrypted reasoning items and the billing of reasoning tokens as output; the GPT-6 Astra and GPT-5.6 entries on slide 23 come from the model and pricing pages linked here. Secondary source, read October 2026.
- **Anthropic (2026).** *Thinking: adaptive thinking, effort, display and signed thinking blocks (developer documentation)*. `https://platform.claude.com/docs/en/build-with-claude/thinking`. Effort levels, the deprecated budget_tokens parameter, summarized or omitted display, and signed thinking blocks that must be passed back across tool calls; the Claude rows on slide 23. Secondary source, read October 2026.
- **Google (2026).** *Gemini thinking: thinking levels, thought signatures, Deep Think (developer documentation)*. `https://ai.google.dev/gemini-api/docs/thinking`. thinking_level, thought signatures that carry reasoning across calls, and the retirement of the 2025 thinkingBudget parameter; the Gemini row on slide 23. Secondary source, read October 2026.
- **DeepSeek (2026).** *DeepSeek V4 model card and API documentation*. `https://api-docs.deepseek.com/`. Non-thinking, thinking-high and thinking-max modes for the 1.6T mixture-of-experts V4-Pro with 49B active parameters; a previous turn's reasoning_content is ignored in plain chat but must be passed back across tool calls. Secondary source, read October 2026.
- **Qwen Team (Alibaba) (2026).** *Qwen3.8 model cards (Hugging Face)*. `https://huggingface.co/Qwen`. Effort levels xhigh/medium/low with thinking that cannot be disabled, preserve-thinking on by default, and the revenue-gated license of the 2.4T model; slide 23. Secondary source, read October 2026.
- **Moonshot AI (2026).** *Kimi K3 model card (Hugging Face)*. `https://huggingface.co/moonshotai`. 2.8T parameters with 104B active, effort levels low/high/max with thinking always on, and the Kimi K3 license; the training recipe (nine specialists, three domains by three effort levels, merged by on-policy distillation with a -1 reward for overshooting the budget) is in the technical report, arXiv:2607.24653; slides 23-24. Secondary source, read October 2026.
- **Rohani (OpenAI) (2025).** *OpenAI's result at the 2025 ICPC World Finals (X thread, September 17, 2025)*. `https://x.com/mostafarohani/status/1968361152741826849`. 12 of 12 problems solved under contest rules: GPT-5 solved 11 on the first try and an experimental model the hardest on its ninth submission. OpenAI published no blog post; the thread is the primary statement, with Google DeepMind's blog covering Gemini 2.5 Deep Think's 10 of 12. Secondary source.

- **Google DeepMind (2025).** *Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad (blog, July 21, 2025)*. https://deepmind.google/discover/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/. 35 of 42 in natural language within the 4.5-hour limit, graded by IMO coordinators; the parallel-thinking Deep Think mode. The 2025 IMO row on slide 25. Secondary source.
- **Balunovic et al. (2025).** *MathArena: evaluating LLMs on uncontaminated math competitions*. arXiv:2505.23281. Evaluate on competitions as they happen: strong signs of contamination on AIME 2024, proof-writing below 40% on IMO 2025 at the time; the live leaderboard behind slide 25's saturation claims.
- **Chollet et al. (2025).** *ARC-AGI-2: a new challenge for frontier AI reasoning systems*. arXiv:2505.11831. Every task solved by at least two humans in testing; o3-medium at 3.0% at launch; cost per task reported beside accuracy. The benchmark that put cost on the axis.
- **Phan et al. (2025).** *Humanity's Last Exam*. arXiv:2501.14249. 2,500 expert questions from about 1,000 contributors; o1 at 8.8% and R1 at 8.6% in January 2025 with calibration errors of 81-93%; the column on slide 23.
- **Pan, Zhang et al. (Berkeley) (2025).** *TinyZero: reproduce DeepSeek R1-Zero's countdown and multiplication tasks for under $30 (GitHub, January 2025)*. https://github.com/Jiayi-Pan/TinyZero. A 3B Qwen base learns search and self-verification on the countdown game with plain RL in veRL; the $30 figure on slide 24. Secondary source.
- **ByteDance Seed and community (2025).** *verl: volcano engine reinforcement learning for LLMs (GitHub)*. https://github.com/volcengine/verl. The most used open RL stack for reasoning models (GRPO, DAPO, PPO; vLLM and SGLang rollouts; FSDP and Megatron training); where DAPO, ReTool and most 2025-26 reproductions ran. Secondary source.

9. Limits, evaluation and safety (slides 26-27)

- **Mirzadeh et al. (2024).** *GSM-Symbolic: understanding the limitations of mathematical reasoning in large language models*. arXiv:2410.05229. Templated variants of GSM8K: scores fall and scatter when names and numbers change; one irrelevant clause (GSM-NoOp) drops accuracy by up to 65%. Slide 26.
- **Zhang et al. (2024).** *A careful examination of large language model performance on grade school arithmetic*. arXiv:2405.00332. GSM1k, fresh problems matched to GSM8K: drops of up to 8% for some model families and none for others; contamination measured directly.
- **Shojaee et al. (2025).** *The illusion of thinking: understanding the strengths and limitations of reasoning models via the lens of problem complexity*. arXiv:2506.06941. Controllable puzzles show three regimes and a collapse beyond a complexity threshold, with effort falling as problems get harder. Read with the reply below.
- **Lawsen (2025).** *Comment on 'The illusion of thinking: understanding the strengths and limitations of reasoning models via the lens of problem complexity'*. arXiv:2506.09250. The Hanoi collapse coincides with output-token limits, river crossing with n>5 is unsolvable as posed, and asking for a generating function instead of the move list succeeds. The harness, not only the model.
- **Hendrycks et al. (2021).** *Measuring mathematical problem solving with the MATH dataset*.

arXiv:2103.03874. 12,500 competition problems with step-by-step solutions; the benchmark every reasoning model saturated by 2025.

- **Rein et al. (2023).** *GPQA: a graduate-level Google-proof Q&A benchmark*. arXiv:2311.12022. 448 questions where PhD experts score 65% and skilled non-experts 34% with web access; GPT-4 scored 39% at launch and frontier models sit in the nineties by 2026. The GPQA column on slide 23.
- **Glazer et al. (2024).** *FrontierMath: a benchmark for evaluating advanced mathematical reasoning in AI*. arXiv:2411.04872. Research-level problems with automatically checkable answers; under 2% at launch. One of the few math benchmarks with headroom in 2026.
- **Jain et al. (2024).** *LiveCodeBench: holistic and contamination free evaluation of large language models for code*. arXiv:2403.07974. Problems collected continuously from contests with release dates, so contamination can be measured by date window; the code analogue of MathArena.
- **OpenAI (2024).** *Introducing SWE-bench Verified (blog, August 13, 2024)*. `https://openai.com/index/introducing-swe-bench-verified/`. 500 human-validated tasks from SWE-bench after 93 developers screened 1,699; GPT-4o from 16% on the original to 33.2%. The agentic benchmark quoted on slides 12 and 21. Secondary source.
- **Yoon et al. (2025).** *Reasoning models better express their confidence*. arXiv:2505.14489. Reasoning models are better calibrated in 33 of 36 settings, traced to slow-thinking behaviors such as exploring alternatives and backtracking; slide 26.
- **Mei et al. (2025).** *Reasoning about uncertainty: do reasoning models know when they don't know?*. arXiv:2506.18183. Stated confidence often above 85% on wrong answers, with longer chains more overconfident; introspection helps some models and hurts others. The other half of slide 26.
- **Zeng et al. (2025).** *Thinking out loud: do reasoning models know when they're right?*. arXiv:2504.06564. Reasoning models verbalize confidence better but abstain less, saying 'I don't know' at lower rates: a reasoning tax on honesty.
- **Baker et al. (2025).** *Monitoring reasoning models for misbehavior and the risks of promoting obfuscation*. arXiv:2503.11926. A GPT-4o monitor reading o3-mini's chain catches reward hacking in agentic coding with 95% recall against 60% from actions alone; training against the monitor hides the hacking (recall near zero) without stopping it. The monitorability tax, slide 27.
- **Korbak et al. (2025).** *Chain of thought monitorability: a new and fragile opportunity for AI safety*. arXiv:2507.11473. A 41-author position paper across labs: chains of thought are monitorable today for reasons that may not last; keep them legible and report monitorability in model cards.
- **Guan et al. (2025).** *Monitoring monitorability*. arXiv:2512.18311. Thirteen evaluations in 24 environments: current models 'fairly, but not perfectly, monitorable'; longer chains more monitorable; RL at current scale did not materially degrade monitorability in two runs.
- **OpenAI (2025).** *OpenAI o3 and o4-mini system card (April 2025)*. `https://openai.com/index/o3-o4-mini-system-card/`. PersonQA hallucination rates of 33% (o3) and 48% (o4-mini) against 16% for o1, as reported in the card and widely covered; the slide-26 tile. Secondary source.
- **OpenAI (2026).** *Reasoning models struggle to control their chains of thought, and that's good (research post, March 5, 2026)*. `https://openai.com/index/reasoning-models-chain-of-thought-controllability/`. Models asked to steer their own chain content succeed between 0.1% and 15.4% of the time at baseline, and the rate falls with further RL training; slide 27. Secondary source.

- **OpenAI (2026).** *GPT-6 Astra system card (September 3, 2026)*. https://deploymentsafety.openai.com/gpt-6-astra. Reports chain-of-thought monitorability below earlier models and evaluation awareness in 9.6% of chains against 2.8% for GPT-5.6 Sol in a Codex deployment simulation, and says the authors take the trend seriously; slide 27. Secondary source.

10. Practice, summary and surveys (slides 28-31)

- **Sun et al. (2023).** *A survey of reasoning with foundation models*. arXiv:2312.11562. The pre-o1 map of reasoning methods, useful for seeing which 2023 ideas survived RL (verifiers, tools, search) and which did not (prompt-time graph search).
- **Zhu and Li (2025).** *Towards concise and adaptive thinking in large reasoning models: a survey*. arXiv:2507.09662. Methods for shorter and adaptive chains, organized by where the control sits; complements Sui et al. for the slide-29 'using' column.
- **Raschka (2025).** *The state of LLM reasoning models (newsletter, March 2025)*. https://magazine.sebastianraschka.com/p/state-of-llm-reasoning-and-inference-scaling. A roundup of inference-time scaling methods published after R1, grouped by what they change; a practitioner's checklist. Secondary source.
- **Hariri et al. (2026).** *Test-time scaling in reasoning LLMs: inference regimes, evaluation, and reproducibility*. arXiv:2608.04001. A 2026 taxonomy of test-time scaling (sequential, leaf, prefix) with 1.4M released attempts; the evaluation protocol slide 29 asks for, spelled out.
- **OpenAI (2025).** *Introducing GPT-5 (blog, August 7, 2025)*. https://openai.com/index/introducing-gpt-5/. The first frontier release with a router choosing when to think and a claim of about six times fewer factual errors than o3 in thinking mode; the hybrid default that 2026 models made universal. Secondary source.
- **Gibney (Scientific American, reprinted from Nature news) (2025).** *Secrets of DeepSeek AI model revealed in landmark paper (September 17, 2025)*. https://www.scientificamerican.com/article/secrets-of-chinese-ai-model-deepseek-revealed-in-landmark-paper/. The peer-reviewed R1 paper's supplement puts the RL phase at about $294,000 on H800s (at an assumed $2 per GPU-hour) on top of a roughly $6M base model; the figure quoted on slides 12 and 24. Secondary source.

Index of terms

Absolute Zero, 38
AC0, 12
accuracy–length frontier, 42
adaptive thinking, 42, 48
advantage, 22
agentic RL, 44
aha moment, 30
AIME, 7, 24, 36
always-on thinking, 48
answer match, 28
Apache-2.0 and MIT, 50
ARC-AGI-2, 52
asynchronous rollouts, 50
auto effort, 40

base model, 30
baseline, 22
bat-and-ball problem, 4
beam search over steps, 18
best-of-N, 16
biasing feature, 14
BIG-Bench Hard, 14
BroRL, 33
budget forcing, 35
budget tokens, 40

cap enforcement, 40
chain of thought, 4
chain-of-thought control, 56
chain-of-thought prompting, 10
chat model, 5
chat template, 31
circuit class, 12
CISPO, 27
class P, 12
Clip-Cov and KL-Cov, 33
clip-higher, 26, 60
clipped surrogate, 26
Coconut, 12
code reasoning, 31
Codeforces rating, 24
cold start, 24
community harness, 52
compute-optimal test-time scaling, 18
compute-performance curve, 34
consensus, 7
consensus at 64, 19
contamination, 52
contested result, 58
cost per task, 52
CoT pressure, 56
coverage, 7, 18, 33
crossover, 33
curriculum, 38

dagger, 4
DAPO, 26
DAPO-Math-17k, 38
decode, 46
decode rate, 46
decontamination, 37, 52
DeepConf, 44
DeepScaleR, 38
DeepSeek-R1, 8, 23
DeepSeek-V3-Base, 23
depth, 12
distillation, 9, 24, 35
Dr. GRPO, 26, 30
dynamic sampling, 26, 59

effort level, 5, 40, 48
effort per task, 59
encrypted thinking, 57
end-of-thinking token, 35
entropy collapse, 27, 33
episode, 21
evaluation awareness, 56

faithfulness, 14
few-shot exemplars, 10
fork token, 33
fork tokens, 27
format reward, 24
forward pass, 5

Game of 24, 18
generative verifier, 16
GPQA Diamond, 24, 36, 48
greedy decoding, 10
group, 26
group baseline, 58

GRPO, 8, 26
GSM-NoOp, 54
GSM-Symbolic, 54
GSM1k, 54
GSM8K, 10
GSPO, 27

hallucination rate, 54
harness, 54
harness sensitivity, 52
hidden chain, 46
hint test, 14, 60
Humanity's Last Exam, 49
hybrid model, 9, 40

ICPC World Finals, 52
identification problem, 18
importance ratio, 26
in-context search, 12
inference budget, 61
International Mathematical Olympiad, 52
isomorphic verification, 29

Kimi K2 Thinking, 45
KL penalty, 22
knee, 40
KV cache, 46

L1, 42
language-consistency reward, 24
latency floor, 46, 59
latent reasoning, 61
LCPO, 42
learned judge, 28
least-to-most prompting, 10
length generalization, 55
length normalization, 31
less-is-more hypothesis, 35
Light-R1, 38
LIMO, 35
list price, 49
locality of experience, 12
log precision, 12
long2short, 42

majority vote, 7, 16
majority voting, 44
MATH-500, 24
Math-Verify, 37
MATH500, 36
MathArena, 52
monitor, 14, 56
monitorability, 14, 56
monitorability tax, 14, 56
Monte Carlo step label, 16
Monte Carlo tree search, 20
multi-head latent attention, 46
MultiArith, 10

n-gram overlap, 37
neutral grading, 61
NoThinking, 42
NuminaMath, 38

o1, 8, 20
o1-ioi, 20
o1-preview, 36
o3, 20
obfuscated reward hacking, 56
obfuscation, 14
official grading, 52
on-policy, 21
on-policy distillation, 50
one-example RLVR, 31
open weights, 50
OpenMathReasoning, 38
OpenR1-Math-220k, 38
OpenThoughts3, 38
oracle pass at 16, 58
outcome reward model, 16
output tokens, 5
output-token limit, 54
over-optimization, 16
overlong reward shaping, 27, 42
overlong shaping, 60
overthinking, 42

parallel axis, 7
parallel sampling, 44
ParaThinker, 44
pass at 1 as a mean, 60
pass at k, 7, 18, 33
pass at k crossover, 58
pass-rate filtering, 38
PersonQA, 54
policy, 21
policy entropy, 31
position paper, 56

post-hoc rationalization, 14
post-saturation generalization, 31
PPO, 26
prefix cache, 46
preserved thinking, 46
PRM800K, 16
pro tier, 44
process reward model, 8, 16
ProRL, 33
proxy reward, 28
puzzle collapse, 54

Quiet-STaR, 12
Qwen-heavy evidence, 58

R1-Zero, 19, 30
random-reward control, 31
re-ranking, 7, 44
reasoning content, 46
reasoning effort, 40
reasoning model, 4, 61
reasoning parser, 46
reasoning tax, 55
reasoning–action dilemma, 42
recall, 14
recurrent depth, 12, 50
reference reset, 33, 58
regular language, 12
REINFORCE, 22
REINFORCE++, 26
reinforcement learning with verifiable rewards, 21
rejection sampling, 10, 24
ReST-EM, 22
ReTool, 29, 44
revenue-gated license, 50
reward hacking, 14, 16, 56
RLHF, 22
RLOO, 26
RLVR, 8
RMS calibration error, 55
rollout, 21, 33
rollout engine, 50
rStar-Math, 20
rubric reward, 16
rule reward, 24

s1, 35
s1K, 35
sandbox, 28, 44
saturation, 52
ScaleRL, 33
SCAN, 10
scratch cell, 12
scratchpad, 8
search inside the chain, 19
search outside the model, 19
self-certainty reward, 28
self-consistency, 10
self-correction, 10
sequential revision, 18
serial axis, 7
serial computation, 5
settled result, 58
sharpening, 33
signed thinking block, 48
Sky-T1, 38
specialist merging, 50
spurious reward, 31
STaR, 8, 22
summarized thinking, 48
supervised fine-tuning, 35
surface form, 54
SWE-bench Verified, 24
SWE-Gym and SWE-smith, 38
SWE-RL, 29, 44

TC0, 12
teacher model, 36
test-time compute, 6
test-time scaling, 7
think and no-think switches, 40
think tags, 5
thinking budget, 40
thinking budget (Google), 40
thinking span, 4
Thinkless, 42
ThinkPrune, 42
thought signature, 48
thought-switching penalty, 42
TinyZero, 50
token budget, 5, 48
token-level loss, 26
token-minutes, 46
tokens per answer, 60
tool-integrated reasoning, 8, 44
training recipe, 61
training stack, 50

trajectory, 44
Tree of Thoughts, 18

underthinking, 42
unit-test reward, 28

value model, 20
value network, 26
VAPO, 27
verbalized confidence, 54
verifiable problem, 37
verifier, 16
verifier gaming, 29, 57

zero-shot chain of thought, 10